Homework 3 Report

Hande Eryılmaz, 21902678

Use of GenAI Tools

This homework assignment was completed with the assistance of **Claude Sonnet 4.5.** The following describes how the tool was used throughout the assignment:

Claude was used to help implement the MATLAB code for all parts of this assignment, assist in structuring and drafting the written commentary and observations sections of this report. All written content was reviewed, understood, and verified by the student. When errors occurred during code execution (for example, array indexing errors in Part 4.1), the error messages were shared with Claude to identify and correct the issues. The student remains fully responsible for all submitted code, results, and written content. All MATLAB code was executed and verified locally by the student. All numerical outputs, figures, and conclusions were checked for correctness and consistency. The student is able to explain and justify all code, results, and interpretations presented in this report.

Part 1 - System Matrix

1.1 - Viewing Calibration Data

Objective and Methodology

The objective of this section is to visualize the frequency-domain calibration measurements stored in the system matrix file `SM_25nm_50x50.mat`. The system matrix `SM` has size `2×1632×2500`, where the first index corresponds to the receive coil number, the second index corresponds to the frequency axis, and the third index corresponds to the grid position (50×50 = 2500 locations).

For both coils, the magnitude spectrum of the calibration measurements was plotted at three different grid locations (5, 100, and 500), resulting in a 2×3 figure layout. The magnitude of the complex-valued frequency-domain signal was computed using the `abs()` function and plotted against the frequency index.

Observations and Comments

The magnitude spectrum plots reveal several important characteristics of the MPI calibration data:

Sparse frequency-domain representation: The signal energy is concentrated at only a small number of discrete frequency indices, with large flat zero regions in between. This confirms that working in the frequency domain is advantageous for MPI, as the signal is highly sparse.

Two clusters of signal energy: The spectra show two distinct groups of non-zero frequency components — one cluster at low frequency indices (approximately 1–200) and another cluster around index ~1550. These correspond to the higher harmonics of the drive field frequencies used to induce the nanoparticle signal in MPI.

Position-dependent magnitudes: Different grid locations (5, 100, 500) show different magnitude levels, reflecting the spatially varying sensitivity of the receive coils. The signal strength depends on where the point source is placed within the scanner field of view.

Conjugate symmetry: Since the time-domain calibration signal is real-valued, the frequency-domain signal exhibits conjugate symmetry. This means only the first half of the spectrum (indices 1–816) contains non-redundant information, which motivates the preprocessing step in Part 1.2.

Both coils show similar patterns: The two receive coils exhibit qualitatively similar spectral patterns but with different magnitude values, as expected from coils placed at different positions relative to the scanner.

Code Snippet and Results

Code:

```
% Task 1.1 - Viewing Calibration Data
locations = [5, 100, 500];

figure;
for coil = 1:2
    for i = 1:3
        subplot(2, 3, (coil-1)*3 + i);
        plot(abs(SM(coil, :, locations(i))));
        title(sprintf('Coil #%d, Grid Location %d', coil, locations(i)));
        xlabel('Frequency Index');
        ylabel('Magnitude');
        grid on;
    end
end
sgtitle('Magnitude Spectrum of Calibration Measurements');
```

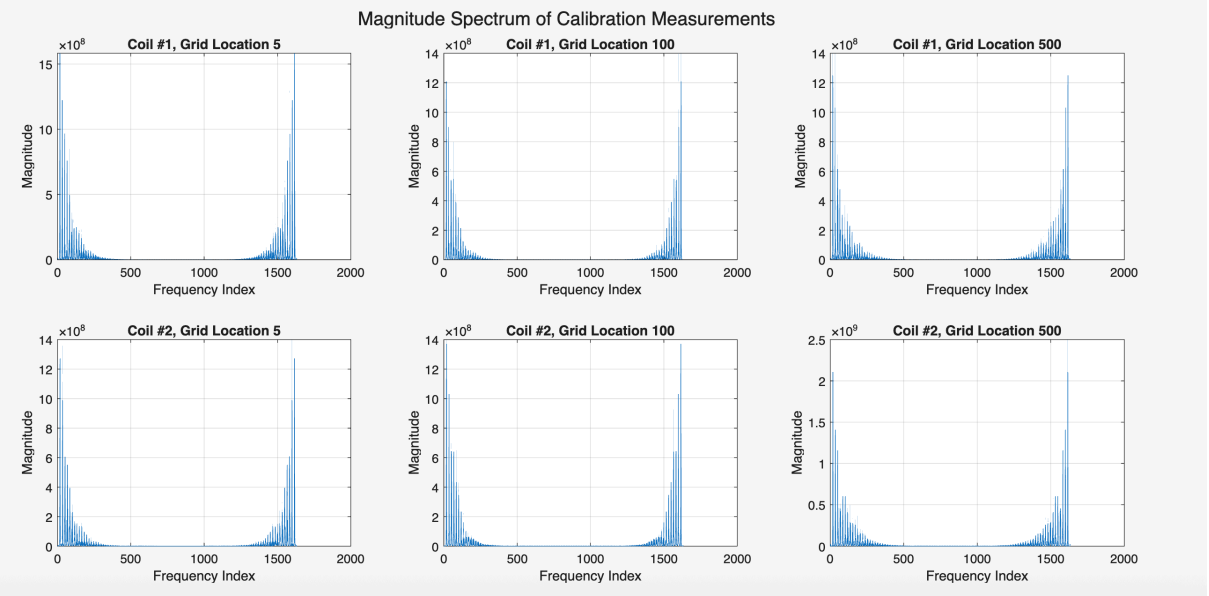


Figure 1: Part 1.1

1.2 - Preparing the System Matrix

Objective and Methodology

The objective of this section is to reformat the raw calibration data `SM` into a proper system matrix `S` suitable for image reconstruction. This involves three sequential preprocessing steps:

Step 1 - Discard redundant frequency components: Since the time-domain calibration signal is real-valued, its Fourier transform has conjugate symmetry. Therefore, only the first half of the frequency axis

contains non-redundant information. The second half is discarded:

```
S = SM(:, 1:end/2, :);
```

After this step, `S` has size `2×816×2500`.

Step 2 - Concatenate coil measurements: The calibration measurements from the two receive coils are concatenated along the frequency axis by reshaping:

```
S = reshape(S, 2*816, 2500);
```

After this step, `S` has size `1632×2500`.

Step 3 - Separate real and imaginary parts: Since the MPI image represents nanoparticle concentration which is real-valued, it is beneficial to work with a purely real-valued system matrix. The real and imaginary parts of `S` are concatenated along the frequency axis:

```
S = [real(S); imag(S)];
```

After this step, `S` has final size `3264×2500` and is purely real-valued. Each column of `S` contains the complete calibration data for a given grid position. The resulting system matrix is saved to a `.mat` file for use in subsequent parts.

Observations and Comments

Final system matrix dimensions: The system matrix `S` has size `3264×2500`. Since the number of rows (3264) exceeds the number of columns (2500), the system is overdetermined in principle. However, as will be seen from the singular value analysis, the effective rank is much lower, making the system ill-conditioned.

Column structure: The plotted columns of `S` show two clusters of non-zero amplitude values — one around indices 1–200 (real parts of the low harmonics) and another around indices 1600–1900 (imaginary parts). Large flat zero regions exist in between, confirming the sparsity of the MPI signal in the frequency domain.

Position-dependent patterns: Different grid locations show different amplitude patterns in their system matrix columns, reflecting the spatially varying sensitivity of the scanner. This position-dependence is what enables spatial encoding in MPI.

File size: The saved `.mat` file is **60.15 MB**, consistent with the expected size for a `3264×2500` double-precision matrix (3264 × 2500 × 8 bytes ≈ 65 MB uncompressed, with MATLAB compression yielding ~60 MB).

Code Snippet and Command Window Output

Code:

```
% Step 1: Discard second half of frequency-domain data
S = SM(:, 1:end/2, :);

% Step 2: Reshape to concatenate two coils along frequency axis
S = reshape(S, 2*816, 2500);

% Step 3: Concatenate real and imaginary parts
S = [real(S); imag(S)];
```

```
% Verify size
disp(size(S));

% Plot three columns
locations = [5, 100, 500];
figure;
for i = 1:3
    subplot(1, 3, i);
    plot(S(:, locations(i)));
    title(sprintf('System Matrix Column - Grid Location %d', locations(i)));
    xlabel('Frequency Index');
    ylabel('Amplitude');
    grid on;
end
sgtitle('System Matrix Columns at Different Grid Locations');

% Save and check file size
save('/Users/handeeryilmaz/Downloads/medikal/homework_475_3/system_matrix_S.ma
t', 'S');
fileInfo = dir('/Users/handeeryilmaz/Downloads/medikal/homework_475_3/system_mat
rix_S.mat');
fprintf('File size: %.2f MB\n', fileInfo.bytes / (1024^2));
```

Command Window Output:

```
3264 2500
System matrix saved.
File size: 60.15 MB
```

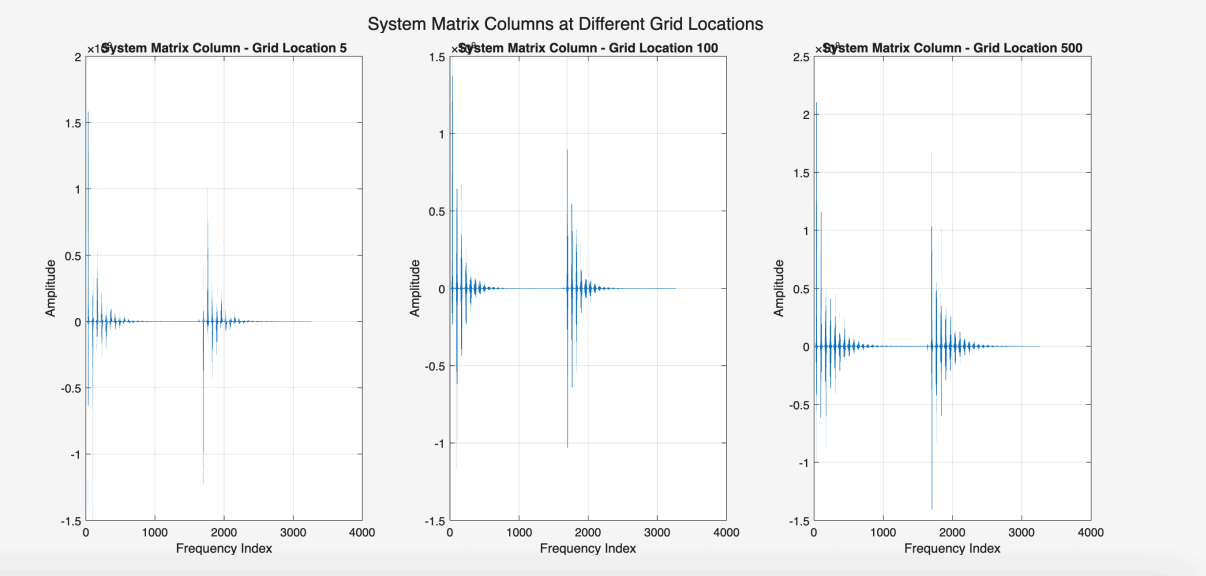


Figure 2: Part 1.2

1.3 - Preparing the Measurement Vector

Objective and Methodology

The objective of this section is to prepare the measurement vector `u` from the frequency-domain phantom measurement data stored in `meas_25nm_phantom.mat`, and to visualize the reference phantom image. The preparation steps are identical to those applied to the system matrix in Part 1.2, which is critical to ensure consistency between `u` and the columns of `S`.

Step 1 - Display the reference phantom: The reference phantom `phantom_ref` has size `50×50` and represents the ground truth spatial distribution of nanoparticle concentration in the field of view. It is displayed using `imagesc` with a grayscale colormap. The maximum pixel intensity is defined as:

```
max_val = max(phantom_ref(:));
```

This variable `max_val` is used throughout Parts 2 and 3 for consistent grayscale windowing `[0 max_val]` for reconstructed images and `[0 0.5*max_val]` for error images, as well as for PSNR and SSIM normalization.

Step 2 - Prepare measurement vector: Following the same steps as in Part 1.2:

```
u = meas(:, 1:end/2);
u = reshape(u, 2*816, 1);
u = [real(u); imag(u)];
```

After these steps, `u` has size `3264×1` and is purely real-valued, consistent with the system matrix `S`.

The resulting measurement vector is saved to a `.mat` file for use in subsequent parts.

Observations and Comments

Reference phantom: The phantom displays a **Y-shaped structure** representing a branching vascular geometry. The pixel intensities represent nanoparticle concentration, with maximum value `max_val = 0.2092`. The background is zero (no nanoparticles), and the Y-shape has varying intensity levels along its branches.

Measurement vector structure: The plot of `u` shows the same two-cluster sparse structure observed in the system matrix columns — signal concentrated at low frequency indices and around index ~1700–1900, with flat zero regions in between. This is consistent with the sparse frequency-domain representation of MPI signals.

Consistency with system matrix: It is critical that `u` is prepared using identical steps as the columns of `S`. Any mismatch in preprocessing would lead to incorrect reconstruction results. Both `u` and `S` now share the same frequency indexing, coil concatenation order, and real/imaginary separation.

File size: The saved measurement vector file is only **24.58 KB**, which is expected since `u` is a single vector of size `3264×1` (3264 × 8 bytes ≈ 26 KB uncompressed).

Grayscale windowing convention: As instructed, for all subsequent reconstructed images the window `[0 max_val]` is used, and for all error images the window `[0 0.5*max_val]` is used. PSNR and SSIM are computed after normalizing both the reconstructed image and `phantom_ref` by `max_val`.

Code Snippet and Command Window Output

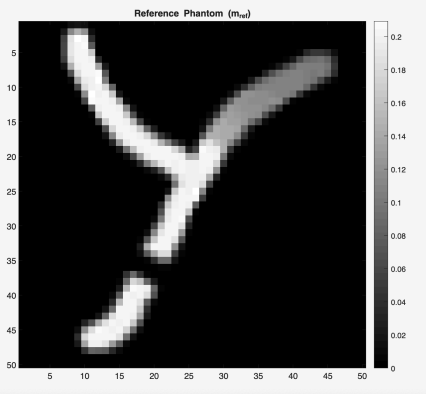
Code:

```
% Display the phantom
figure;
imagesc(phantom_ref);
colormap gray;
colorbar;
title('Reference Phantom (m_{ref})');
axis image;
```

```
% Define max_val
max_val = max(phantom_ref(:));
fprintf('max_val = %.4f\n', max_val);

% Prepare measurement vector u
u = meas(:, 1:end/2);
u = reshape(u, 2*816, 1);
u = [real(u); imag(u)];

% Verify size
fprintf('Size of u: %d x %d\n', size(u,1), size(u,2));

% Plot measurement vector
figure;
plot(u);
title('Measurement Vector u');
xlabel('Index');
ylabel('Amplitude');
grid on;

% Save and check file size
save('/Users/handeeryilmaz/Downloads/medikal/homework_475_3/measurement_vector_
u.mat', 'u');
fileInfo = dir('/Users/handeeryilmaz/Downloads/medikal/homework_475_3/measuremen
t_vector_u.mat');
fprintf('File size: %.4f KB\n', fileInfo.bytes / 1024);
```

Command Window Output:

```
max_val = 0.2092
Size of u: 3264 x 1
Measurement vector saved.
File size: 24.5771 KB
```



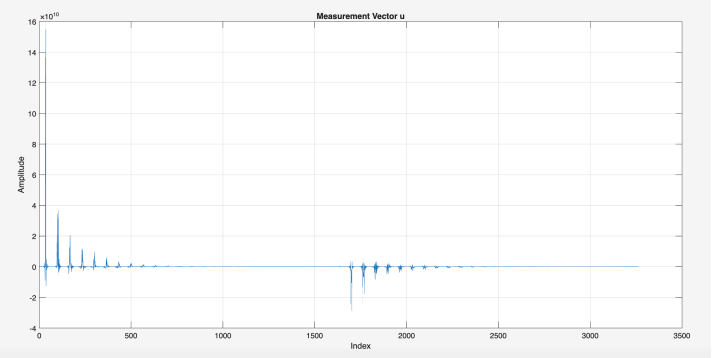


Figure 3: Part 1.3

Part 2 - SVD-Based Reconstruction

2.1 - SVD

Objective and Methodology

The objective of this section is to compute the Singular Value Decomposition (SVD) of the system matrix `S` and analyze its singular values to understand the conditioning of the reconstruction problem. The compact (economy) SVD is computed as:

```
[U, Sigma, V] = svd(S, 'econ');
```

This yields:

- `U`: left singular vectors, size `3264×2500`
- `Sigma`: diagonal matrix of singular values, size `2500×2500`
- `V`: right singular vectors, size `2500×2500`

The singular values are extracted from the diagonal of `Sigma` and plotted on both linear and logarithmic scales. The condition number is computed manually as the ratio of the largest to smallest singular value:

$$\text{Cond}(S) = \frac{\sigma_1}{\sigma_{2500}}$$

The result is verified against MATLAB's built-in `cond()` function.

Observations and Comments

Singular value decay: The linear scale plot shows a sharp drop near index 0, with singular values quickly approaching zero and remaining flat for the majority of indices. The logarithmic scale plot reveals the full dynamic range of singular values, decaying smoothly from ~10^{10} down to ~10^{-5}, with a steep cliff around index ~2200 where singular values essentially collapse to near zero.

Extremely large condition number: The condition number is approximately **6.73×10^{16}** (manual) and **4.40×10^{17}** (built-in). The small discrepancy between the two is due to differences in internal numerical precision when computing the smallest singular values. Both values confirm that the system matrix is **extremely ill-conditioned**.

Implications for reconstruction: The large condition number means that small perturbations in the measurement vector `u` (such as noise) will be amplified by a factor of ~10^{16}–10^{17} when computing the naive pseudoinverse solution. This makes regularization essential for any meaningful image reconstruction, as will be demonstrated in Part 2.2.

Rank deficiency: The steep cliff in singular values around index 2200 suggests that the effective rank of `S` is approximately 2200, even though the matrix has 2500 columns. The remaining ~300 singular values are essentially numerical noise.

Code Snippet and Command Window Output

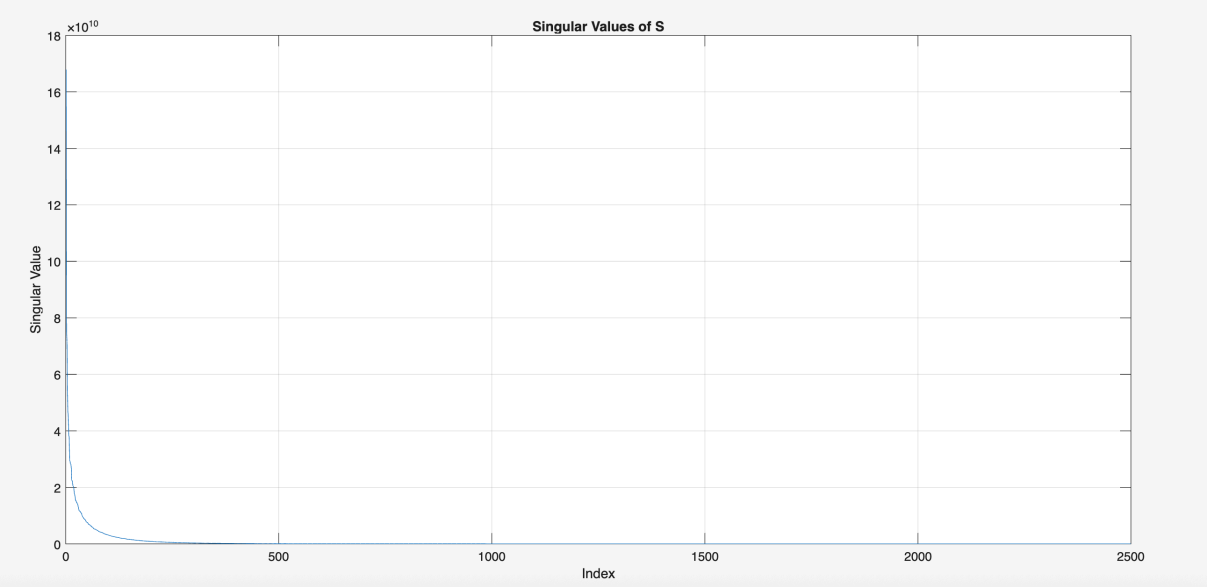
Code:

```
% Compute compact SVD
[U, Sigma, V] = svd(S, 'econ');

% Verify sizes
fprintf('Size of U: %d x %d\n', size(U,1), size(U,2));
```

```
fprintf('Size of Sigma: %d x %d\n', size(Sigma,1), size(Sigma,2));
fprintf('Size of V: %d x %d\n', size(V,1), size(V,2));

% Extract diagonal singular values
sigma_vals = diag(Sigma);

% Plot singular values
figure;
plot(sigma_vals);
title('Singular Values of S');
xlabel('Index');
ylabel('Singular Value');
grid on;

figure;
semilogy(sigma_vals);
title('Singular Values of S (Log Scale)');
xlabel('Index');
ylabel('Singular Value (log scale)');
grid on;

% Condition number
sigma_max = sigma_vals(1);
sigma_min = sigma_vals(end);
cond_manual = sigma_max / sigma_min;
fprintf('Condition number (manual): %.4e\n', cond_manual);

cond_builtin = cond(S);
fprintf('Condition number (built-in): %.4e\n', cond_builtin);
```

Command Window Output:

```
Size of U: 3264 x 2500
Size of Sigma: 2500 x 2500
Size of V: 2500 x 2500
Condition number (manual): 6.7344e+16
Condition number (built-in): 4.4041e+17
```



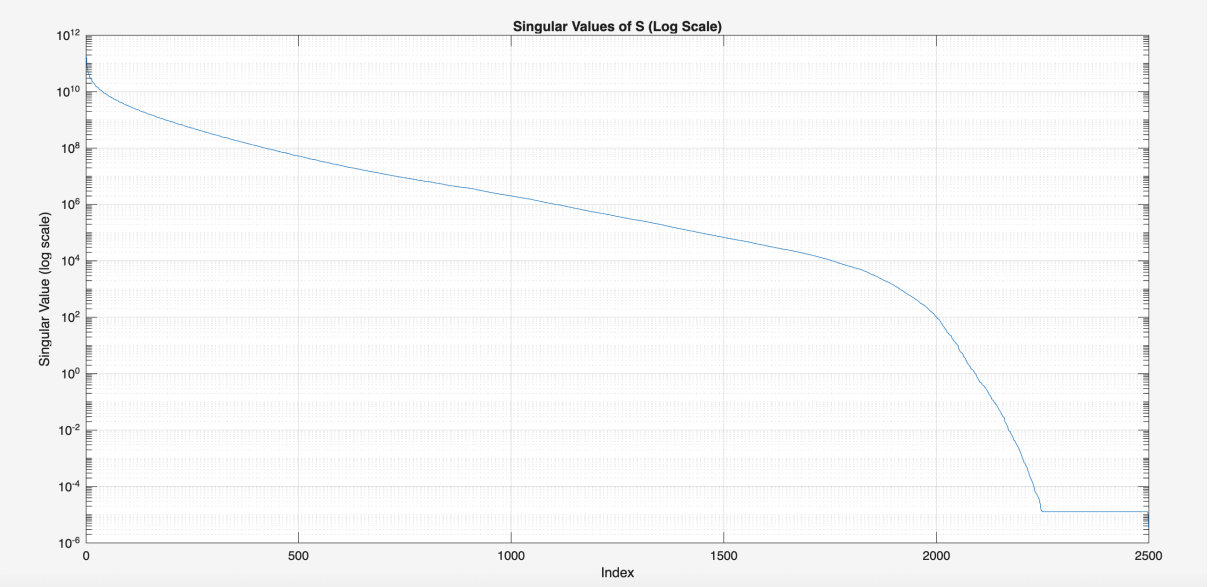


Figure 4: Part 2.1

2.2 - Image Reconstruction via SVD

Objective and Methodology

The objective of this section is to reconstruct the MPI image using the Moore-Penrose pseudoinverse, computed via the full SVD decomposition obtained in Part 2.1. The pseudoinverse solution is given by:

$$c = V\Sigma^{-1}U^{T}u$$

where Σ^{-1} is computed by inverting all diagonal entries of Σ. The reconstruction steps are:

1. Compute the pseudoinverse solution `c` of size `2500×1`
2. Reshape `c` into a 2D image of size `50×50`
3. Set all negative pixel values to zero (nanoparticle concentration cannot be negative)
4. Display the reconstructed image with grayscale window `[0 max_val]`

5. Compute and display the error image `|ima - phantom_ref|` with window `[0 0.5*max_val]`
6. Compute PSNR and SSIM after normalizing both images by `max_val`

Observations and Comments

Catastrophic failure of the pseudoinverse: The reconstructed image is completely dominated by a severe checkerboard-like noise pattern, making the Y-shaped phantom entirely unrecognizable. This is a direct consequence of the extreme ill-conditioning of `S`.

Root cause — noise amplification: The pseudoinverse computes $1/\sigma_i$ for all singular values, including the near-zero ones (~10^{-5}). This amplifies any numerical noise by a factor proportional to the condition number (~10^{16}–10^{17}), resulting in a completely corrupted reconstruction.

Extremely poor metrics: PSNR of **-119.15 dB** is severely negative, meaning the reconstruction error is orders of magnitude larger than the signal itself. SSIM of **0.0334** is near zero, indicating virtually no structural similarity to the reference phantom.

Circular artifact boundary: A circular boundary of high-amplitude noise is visible, corresponding to the coverage boundary of the Lissajous trajectory used during the scan. Pixels outside the field of view receive particularly large errors due to the poor conditioning at the edges.

Conclusion: The plain pseudoinverse is completely unusable for MPI image reconstruction due to the extreme ill-conditioning of the system matrix. This strongly motivates the use of regularization techniques such as Truncated SVD and Filtered SVD, which will be explored in Parts 2.3–2.8.

Code Snippet and Command Window Output

Code:

```
% Compute pseudo-inverse solution: c = V * Sigma^-1 * U' * u
sigma_vals = diag(Sigma);
Sigma_inv = diag(1 ./ sigma_vals);
c = V * Sigma_inv * (U' * u);

% Reshape into 2D image
ima = reshape(c, 50, 50);

% Set negative values to zero
ima(ima < 0) = 0;

% Display reconstructed image
figure;
subplot(1,2,1);
imshow(ima, [0 max_val]);
colormap gray; colorbar;
title('SVD Reconstruction');
axis image;

% Compute and display error image
error_img = abs(ima - phantom_ref);
subplot(1,2,2);
imshow(error_img, [0 0.5*max_val]);
colormap gray; colorbar;
title('Error Image');
axis image;
```

```
sgtitle('SVD Reconstruction vs Error');

% Compute PSNR and SSIM
ima_norm = ima / max_val;
ref_norm = phantom_ref / max_val;
psnr_val = psnr(ima_norm, ref_norm);
ssim_val = ssim(ima_norm, ref_norm);

fprintf('PSNR: %.4f dB\n', psnr_val);
fprintf('SSIM: %.4f\n', ssim_val);
```

Command Window Output:

```
PSNR: -119.1470 dB
SSIM: 0.0334
```

Figure 5: Part 2.2

2.3 - Truncated SVD

Objective and Methodology

The objective of this section is to improve the reconstruction quality by applying Truncated SVD (TSVD), which addresses the ill-conditioning problem by discarding the smallest singular values. Instead of inverting all 2500 singular values as in Part 2.2, only the largest M singular values are retained, such that the condition number of the truncated system is approximately 100:

$$\frac{\sigma_1}{\sigma_M} \approx 100$$

The value of M is determined by finding the largest index where the singular value is still at least $\sigma_1/100$:

```
M = sum(sigma_vals >= sigma_max / target_cond);
```

The truncated matrices are then:

- `U_trunc`: size `3264×M` (first M columns of U)
- `Sigma_trunc`: size `M×M` (top-left M×M block of Sigma)
- `V_trunc`: size `2500×M` (first M columns of V)

The reconstruction is computed as:

$$c = V_{\text{trunc}} \Sigma_{\text{trunc}}^{-1} U_{\text{trunc}}^{T} u$$

The same post-processing steps as Part 2.2 are applied: reshaping to 50×50, clipping negatives to zero, and computing PSNR and SSIM.

Observations and Comments

M = 143 singular values retained: To achieve a condition number of approximately 100, only the 143 largest singular values out of 2500 are kept. This means 2357 singular values — which were causing catastrophic noise amplification — are discarded.

Method	PSNR (dB)	SSIM
Full SVD (pseudoinverse)	-119.15	0.033
Truncated SVD (M=143)	19.77	0.422

Table 1: Part 2.3 Dramatic improvement over full pseudoinverse

Y-shape phantom clearly visible: The main structure of the phantom is successfully recovered, which was completely lost in the full pseudoinverse reconstruction.

Remaining problems:

- **Blurring:** The edges of the Y-shape are smoothed out. Discarding higher singular values removes fine spatial detail, resulting in a low-resolution reconstruction
- **Ringing artifacts:** Small bright and dark spots are visible around the phantom structure in both the reconstruction and error image, caused by the sharp truncation in singular value space
- **Loss of fine features:** The small bottom branch of the Y is less well-defined compared to the reference

Conclusion: TSVD is a major improvement over the full pseudoinverse, demonstrating that regularization by truncation is effective. However, the remaining blurring and ringing artifacts motivate exploration of smoother regularization methods such as Filtered SVD in Parts 2.5–2.8.

Code Snippet and Command Window Output

Code:

```
% Find M such that sigma(1)/sigma(M) ≈ 100
target_cond = 100;
sigma_max = sigma_vals(1);
M = sum(sigma_vals >= sigma_max / target_cond);
fprintf('M (number of kept singular values): %d\n', M);
fprintf('Achieved condition number: %.4f\n', sigma_max / sigma_vals(M));

% Truncate U, Sigma, V
U_trunc = U(:, 1:M);
Sigma_trunc = Sigma(1:M, 1:M);
V_trunc = V(:, 1:M);
```

```
fprintf('Size of U_trunc: %d x %d\n', size(U_trunc,1), size(U_trunc,2));
fprintf('Size of Sigma_trunc: %d x %d\n', size(Sigma_trunc,1), size(Sigma_trunc,
2));
fprintf('Size of V_trunc: %d x %d\n', size(V_trunc,1), size(V_trunc,2));

% Reconstruct
sigma_trunc_vals = diag(Sigma_trunc);
Sigma_trunc_inv = diag(1 ./ sigma_trunc_vals);
c = V_trunc * Sigma_trunc_inv * (U_trunc' * u);

% Reshape and clip
ima_tsvd = reshape(c, 50, 50);
ima_tsvd(ima_tsvd < 0) = 0;

% Display
figure;
subplot(1,3,1);
imshow(phantom_ref, [0 max_val]);
colormap gray; colorbar;
title('Reference Phantom'); axis image;

subplot(1,3,2);
imshow(ima_tsvd, [0 max_val]);
colormap gray; colorbar;
title(sprintf('TSVD Reconstruction (M=%d)', M)); axis image;

subplot(1,3,3);
error_tsvd = abs(ima_tsvd - phantom_ref);
imshow(error_tsvd, [0 0.5*max_val]);
colormap gray; colorbar;
title('Error Image'); axis image;

sgtitle('Truncated SVD Reconstruction');

% Metrics
ima_tsvd_norm = ima_tsvd / max_val;
ref_norm = phantom_ref / max_val;
psnr_tsvd = psnr(ima_tsvd_norm, ref_norm);
ssim_tsvd = ssim(ima_tsvd_norm, ref_norm);

fprintf('PSNR: %.4f dB\n', psnr_tsvd);
fprintf('SSIM: %.4f\n', ssim_tsvd);
```

Command Window Output:

```
M (number of kept singular values): 143
Achieved condition number: 98.8228
Size of U_trunc: 3264 x 143
Size of Sigma_trunc: 143 x 143
Size of V_trunc: 2500 x 143
```

```
PSNR: 19.7741 dB
SSIM: 0.4219
```

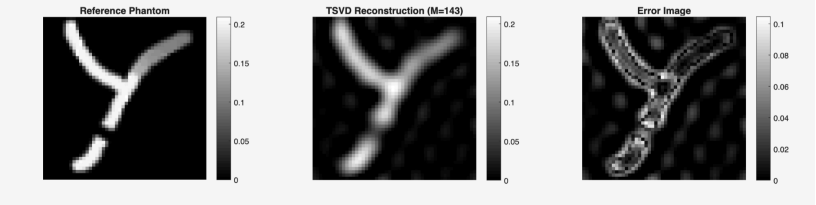


Figure 6: Part 2.3

2.4 - Truncated SVD for Different Condition Numbers

Objective and Methodology

The objective of this section is to systematically study the effect of the truncation threshold on reconstruction quality by repeating the TSVD reconstruction for a range of condition numbers: `[1, 5, 10, 10², 10³, 10⁴, 10⁵, 10⁶, 10⁷]`. For each condition number, the corresponding number of retained singular values M is determined, the TSVD reconstruction is computed at the 10th iteration, and PSNR and SSIM are recorded. All reconstructed images and error images are displayed in a single figure with a 2×9 subplot layout (top row: reconstructions, bottom row: error images). PSNR and SSIM are plotted as functions of condition number on a logarithmic x-axis.

Observations and Comments

Condition Number	M	PSNR (dB)	SSIM
10^0	1	11.60	0.015
10^1	8	12.13	0.144
10^1	21	13.59	0.328
10^2	143	19.77	0.422
10^3	364	26.46	0.666
10^4	648	27.92	0.859
10^5	1029	30.61	0.820
10^6	**1370**	**31.12**	**0.886**
10^7	1700	30.18	0.856

Table 2: Part 2.4 Summary of results

Low condition numbers (10^0–10^1): Severe over-regularization — only 1–21 singular values are kept, resulting in extremely blurry images where the Y-shape is barely recognizable. Both PSNR and SSIM are very low, indicating poor reconstruction quality.

Medium condition numbers (10^2–10^4): Progressive and significant improvement — the Y-shape becomes increasingly sharp and well-defined with each order of magnitude increase in condition number. Both PSNR and SSIM rise steadily as more singular values are included.

Optimal condition number = 10^6 (M=1370): The best reconstruction quality is achieved at condition number 10^6, with PSNR=31.12 dB and SSIM=0.886. This represents a good balance between retaining enough singular values for spatial detail while discarding the noisy small singular values.

Too high condition number (10^7): Both PSNR and SSIM slightly decrease at condition number 10^7 (M=1700), as including more small singular values begins to reintroduce noise into the reconstruction.

This demonstrates the classic bias-variance trade-off in regularization.

General trend: Both PSNR and SSIM increase with condition number up to 10^6, then slightly decrease at 10^7, revealing a clear optimal operating point. The PSNR curve shows a smooth unimodal shape, while the SSIM curve is slightly more irregular due to its sensitivity to structural features.

Code Snippet and Command Window Output

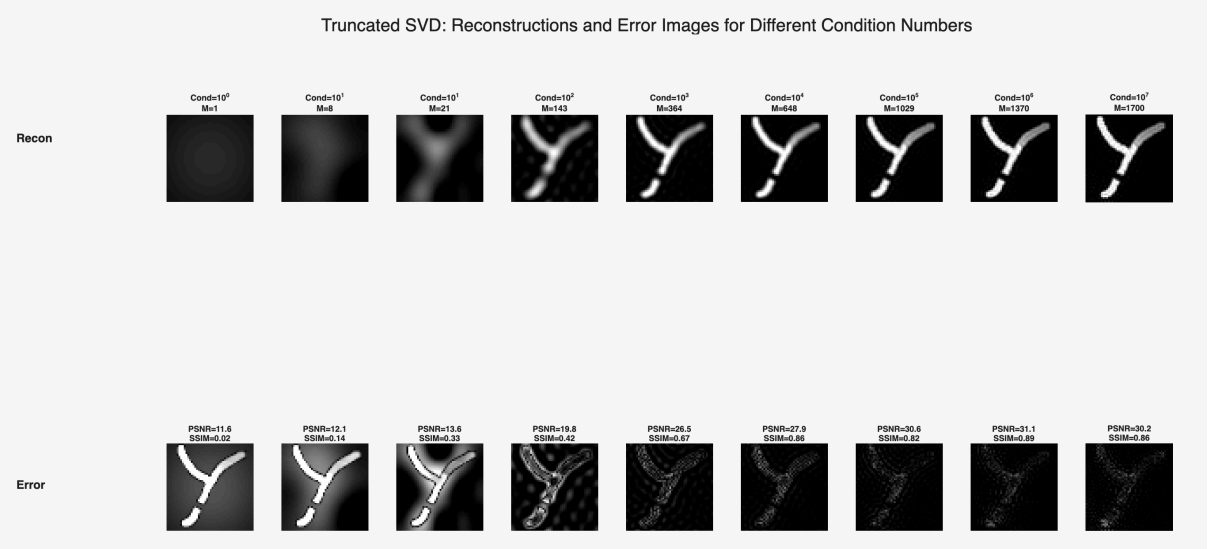
Code:

```
condition_numbers = [1, 5, 10, 100, 1e3, 1e4, 1e5, 1e6, 1e7];
n_conds = length(condition_numbers);
sigma_vals = diag(Sigma);
sigma_max = sigma_vals(1);

psnr_vals = zeros(1, n_conds);
ssim_vals = zeros(1, n_conds);
M_vals    = zeros(1, n_conds);
ref_norm  = phantom_ref / max_val;

figure('Units','normalized','Position',[0 0 1 0.6]);

for idx = 1:n_conds
    target_cond = condition_numbers(idx);
    M = sum(sigma_vals >= sigma_max / target_cond);
    if M < 1, M = 1; end
    M_vals(idx) = M;

    U_trunc = U(:, 1:M);
    Sigma_trunc_inv = diag(1 ./ sigma_vals(1:M));
    V_trunc = V(:, 1:M);

    c = V_trunc * Sigma_trunc_inv * (U_trunc' * u);
    ima = reshape(c, 50, 50);
    ima(ima < 0) = 0;

    ima_norm = ima / max_val;
    psnr_vals(idx) = psnr(ima_norm, ref_norm);
    ssim_vals(idx) = ssim(ima_norm, ref_norm);

    subplot(2, n_conds, idx);
    imshow(ima, [0 max_val]); colormap gray;
    title(sprintf('Cond=10^{%d}\nM=%d', round(log10(max(target_cond,1))), M), 'F
ontSize', 7);
    axis image;

    subplot(2, n_conds, n_conds + idx);
    error_img = abs(ima - phantom_ref);
    imshow(error_img, [0 0.5*max_val]); colormap gray;
    title(sprintf('PSNR=%.1f\nSSIM=%.2f', psnr_vals(idx), ssim_vals(idx)), 'Font
Size', 7);
    axis image;
```

```
    fprintf('Cond: %.0e | M: %4d | PSNR: %.4f dB | SSIM: %.4f\n', ...
        target_cond, M, psnr_vals(idx), ssim_vals(idx));
end

sgtitle('Truncated SVD: Reconstructions and Error Images for Different Condition
Numbers');

figure;
subplot(2,1,1);
semilogx(condition_numbers, psnr_vals, 'b-o', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('Condition Number (log scale)'); ylabel('PSNR (dB)');
title('PSNR vs Condition Number (TSVD)'); grid on;

subplot(2,1,2);
semilogx(condition_numbers, ssim_vals, 'r-o', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('Condition Number (log scale)'); ylabel('SSIM');
title('SSIM vs Condition Number (TSVD)'); grid on;
sgtitle('Image Quality vs Condition Number');

[~, best_psnr_idx] = max(psnr_vals);
[~, best_ssim_idx] = max(ssim_vals);
fprintf('\nBest PSNR: %.4f dB at Cond = %.0e (M=%d)\n', ...
    psnr_vals(best_psnr_idx), condition_numbers(best_psnr_idx), M_vals(best_psnr
_idx));
fprintf('Best SSIM: %.4f at Cond = %.0e (M=%d)\n', ...
    ssim_vals(best_ssim_idx), condition_numbers(best_ssim_idx), M_vals(best_ssim
_idx));
```

Command Window Output:

```
Cond: 1e+00 | M:    1 | PSNR: 11.6012 dB | SSIM: 0.0151
Cond: 5e+00 | M:    8 | PSNR: 12.1296 dB | SSIM: 0.1443
Cond: 1e+01 | M:   21 | PSNR: 13.5874 dB | SSIM: 0.3283
Cond: 1e+02 | M:  143 | PSNR: 19.7741 dB | SSIM: 0.4219
Cond: 1e+03 | M:  364 | PSNR: 26.4567 dB | SSIM: 0.6660
Cond: 1e+04 | M:  648 | PSNR: 27.9205 dB | SSIM: 0.8588
Cond: 1e+05 | M: 1029 | PSNR: 30.6093 dB | SSIM: 0.8196
Cond: 1e+06 | M: 1370 | PSNR: 31.1224 dB | SSIM: 0.8859
Cond: 1e+07 | M: 1700 | PSNR: 30.1781 dB | SSIM: 0.8560
Best PSNR: 31.1224 dB at Cond = 1e+06 (M=1370)
Best SSIM: 0.8859 at Cond = 1e+06 (M=1370)
```



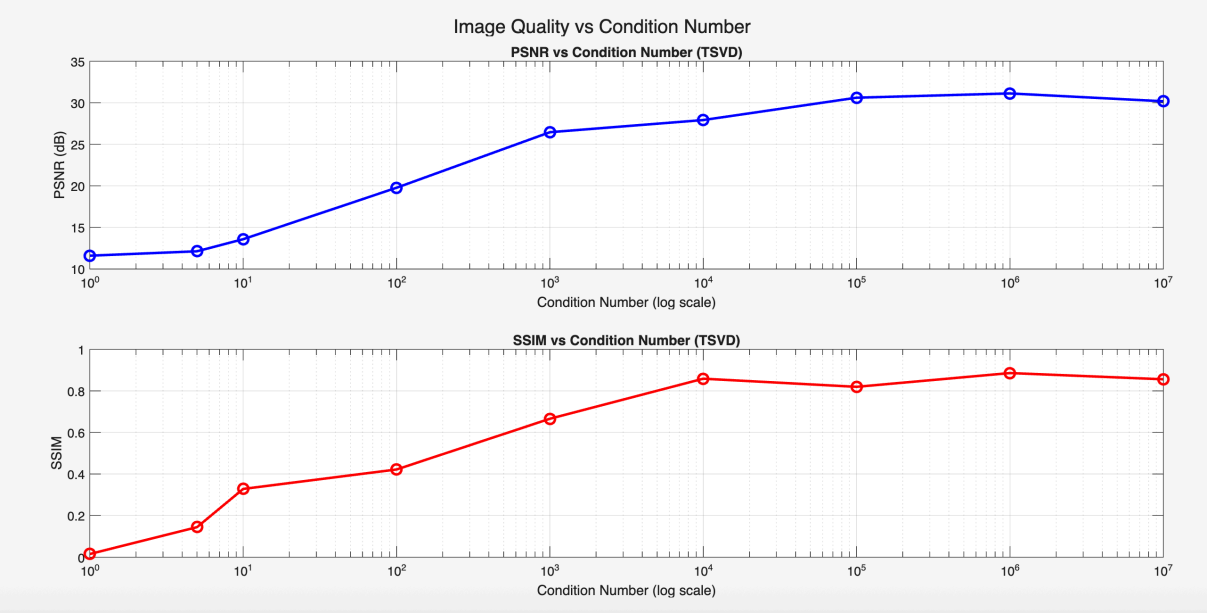


Figure 7: Part 2.4

2.5 - Filtered SVD

Objective and Methodology

The objective of this section is to implement Filtered SVD, a smoother alternative to TSVD that gradually attenuates small singular values rather than hard-truncating them. Each singular value σ_i is multiplied by a filter value y_i:

$$y_i = \frac{\sigma_i^2}{\sigma_i^2 + \lambda}$$

where λ is a regularization parameter. This is equivalent to replacing each singular value σ_i with a filtered singular value σ_i:

$$\sigma_i' = \frac{\sigma_i}{y_i} = \frac{\sigma_i^2 + \lambda}{\sigma_i} = \sigma_i \left(1 + \frac{\lambda}{\sigma_i^2}\right)$$

The reconstruction is then computed as:

$$c = V\tilde{\Sigma}^{-1}U^T u$$

where $\tilde{\Sigma}^{-1}$ is a diagonal matrix with entries y_i/σ_i. This approach is equivalent to solving the following regularized least squares problem:

$$(S^H S + \lambda I)c = S^H u$$

This equivalence can be shown by substituting the SVD decomposition $S = U\Sigma V^H$ into the normal equations, yielding the same filtered solution.

The reconstruction is tested for a range of λ values: `[10⁵, 10⁸, 10¹⁰, 10¹², 10¹⁴, 10¹⁶, 10¹⁸, 10²⁰, 10²²]`. For each value, the reconstructed image and error image are displayed in a single 2×9 figure, and PSNR and SSIM are recorded and plotted as functions of λ.

Observations and Comments

λ	PSNR (dB)	SSIM
10^5	17.05	0.466
10^8	29.28	0.894
10^{10}	31.67	**0.942**
10^{12}	**31.96**	0.940
10^{14}	29.00	0.939
10^{16}	27.46	0.890
10^{18}	22.26	0.776
10^{20}	15.34	0.520
10^{22}	11.90	0.033

Table 3: Part 2.5 Summary of results

Small λ (10^5): Under-regularized — the filter values y_i are close to 1 for most singular values, providing little suppression of small singular values. Checkerboard-like noise artifacts are visible in the reconstruction.

Optimal range (10^{10}–10^{12}): Best trade-off between noise suppression and detail preservation. The Y-shape is cleanly and sharply recovered with minimal artifacts. Best PSNR of **31.96 dB** at λ=10^{12} and best SSIM of **0.942** at λ=10^{10}.

Large λ (10^{18}–10^{22}): Over-regularized — the filter values y_i approach zero for all singular values, causing heavy smoothing and eventually complete image suppression at λ=10^{22}, where the image fades to black.

Smooth bell-shaped quality curve: Unlike TSVD which applies a hard cutoff, Filtered SVD produces a smooth transition in image quality as a function of λ. This is because the filter gradually attenuates rather than abruptly discards singular values.

Method	Best PSNR (dB)	Best SSIM
TSVD	31.12	0.886
Filtered SVD	**31.96**	**0.942**

Table 4: Part 2.5 Comparison with TSVD

Filtered SVD outperforms TSVD on both metrics, producing cleaner images with smoother edges and fewer ringing artifacts due to the gradual rather than hard truncation of singular values.

Code Snippet and Command Window Output

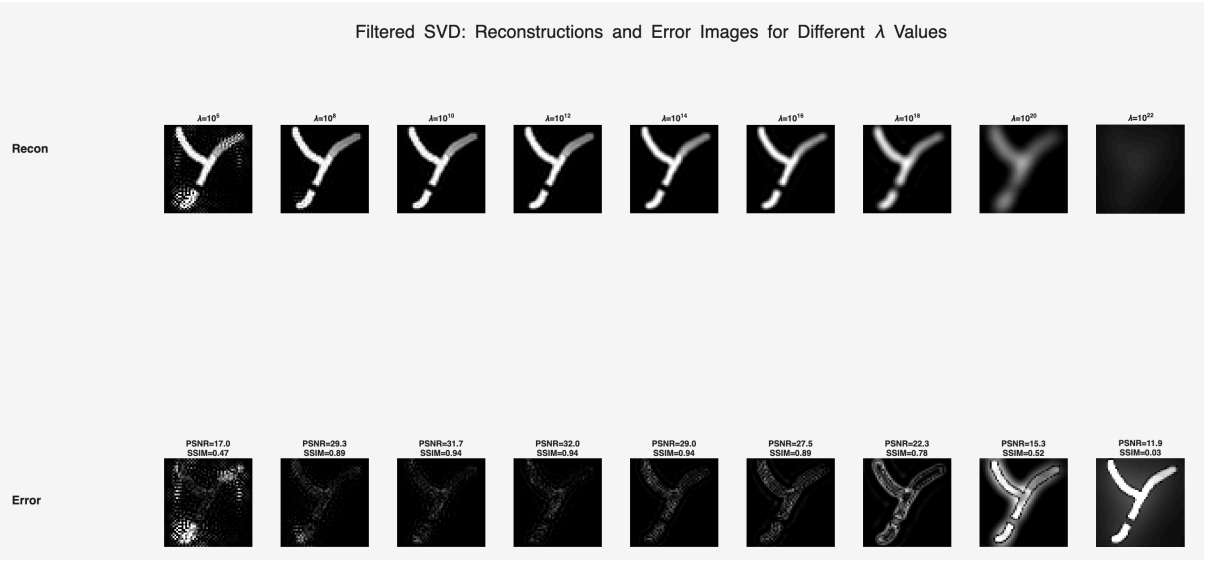
Code:

```
sigma_vals = diag(Sigma);
lambdas = [1e5, 1e8, 1e10, 1e12, 1e14, 1e16, 1e18, 1e20, 1e22];
n_lambdas = length(lambdas);

psnr_fsvd = zeros(1, n_lambdas);
ssim_fsvd = zeros(1, n_lambdas);
ref_norm  = phantom_ref / max_val;

figure('Units','normalized','Position',[0 0 1 0.6]);

for idx = 1:n_lambdas
    lambda = lambdas(idx);

    % Filter values
    y = sigma_vals.^2 ./ (sigma_vals.^2 + lambda);

    % Filtered inverse
    Sigma_tilde_inv = diag(y ./ sigma_vals);

    % Reconstruct
    c = V * Sigma_tilde_inv * (U' * u);
    ima = reshape(c, 50, 50);
    ima(ima < 0) = 0;

    ima_norm = ima / max_val;
    psnr_fsvd(idx) = psnr(ima_norm, ref_norm);
    ssim_fsvd(idx) = ssim(ima_norm, ref_norm);

    subplot(2, n_lambdas, idx);
    imshow(ima, [0 max_val]); colormap gray;
    title(sprintf('\\lambda=10^{%d}', round(log10(lambda))), 'FontSize', 7);
    axis image;

    subplot(2, n_lambdas, n_lambdas + idx);
    error_img = abs(ima - phantom_ref);
    imshow(error_img, [0 0.5*max_val]); colormap gray;
    title(sprintf('PSNR=%.1f\nSSIM=%.2f', psnr_fsvd(idx), ssim_fsvd(idx)), 'Font
Size', 7);
    axis image;

    fprintf('Lambda: %.0e | PSNR: %.4f dB | SSIM: %.4f\n', ...
        lambda, psnr_fsvd(idx), ssim_fsvd(idx));
end

sgtitle('Filtered SVD: Reconstructions and Error Images for Different \lambda Va
```

```
lues');

figure;
subplot(2,1,1);
semilogx(lambdas, psnr_fsvd, 'b-o', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('\lambda (log scale)'); ylabel('PSNR (dB)');
title('PSNR vs \lambda (Filtered SVD)'); grid on;

subplot(2,1,2);
semilogx(lambdas, ssim_fsvd, 'r-o', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('\lambda (log scale)'); ylabel('SSIM');
title('SSIM vs \lambda (Filtered SVD)'); grid on;
sgtitle('Image Quality vs \lambda (Filtered SVD)');

[~, best_psnr_idx] = max(psnr_fsvd);
[~, best_ssim_idx] = max(ssim_fsvd);
fprintf('\nBest PSNR: %.4f dB at lambda = %.0e\n', ...
    psnr_fsvd(best_psnr_idx), lambdas(best_psnr_idx));
fprintf('Best SSIM: %.4f at lambda = %.0e\n', ...
    ssim_fsvd(best_ssim_idx), lambdas(best_ssim_idx));
```

Command Window Output:

```
Lambda: 1e+05 | PSNR: 17.0460 dB | SSIM: 0.4661
Lambda: 1e+08 | PSNR: 29.2802 dB | SSIM: 0.8940
Lambda: 1e+10 | PSNR: 31.6741 dB | SSIM: 0.9423
Lambda: 1e+12 | PSNR: 31.9573 dB | SSIM: 0.9396
Lambda: 1e+14 | PSNR: 29.0009 dB | SSIM: 0.9391
Lambda: 1e+16 | PSNR: 27.4607 dB | SSIM: 0.8897
Lambda: 1e+18 | PSNR: 22.2581 dB | SSIM: 0.7762
Lambda: 1e+20 | PSNR: 15.3368 dB | SSIM: 0.5195
Lambda: 1e+22 | PSNR: 11.8952 dB | SSIM: 0.0333
Best PSNR: 31.9573 dB at lambda = 1e+12
Best SSIM: 0.9423 at lambda = 1e+10
```



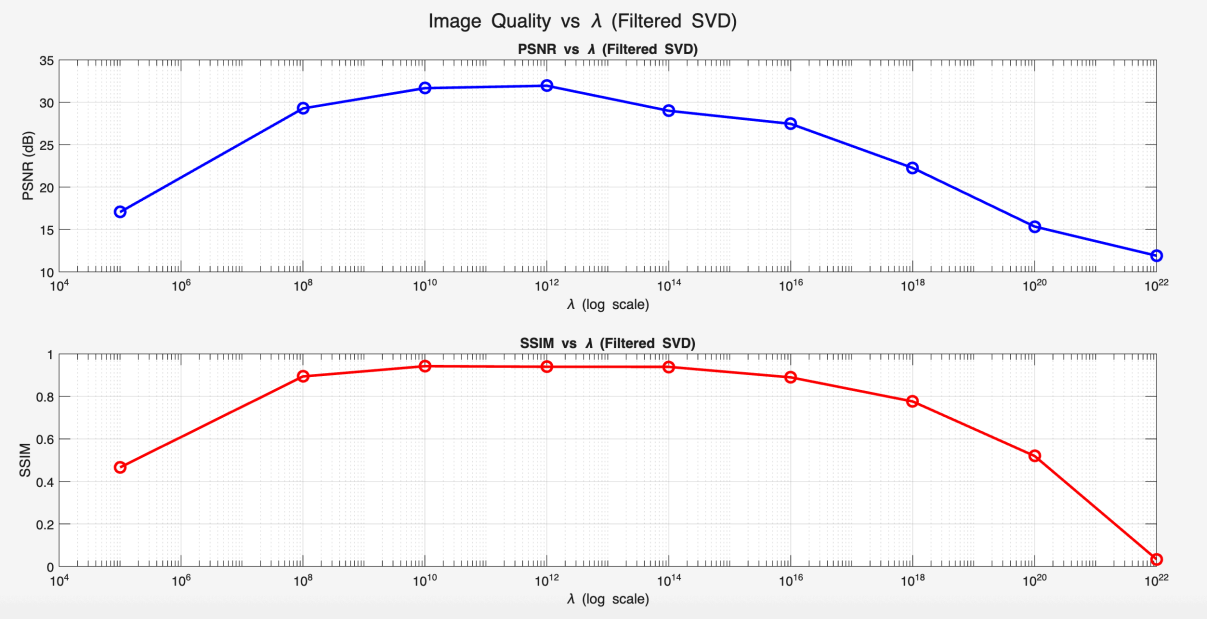


Figure 8: Part 2.5

2.6 - Filtered SVD with λ = σ₁σₙ

Objective and Methodology

The objective of this section is to analyze the theoretical motivation for choosing the regularization parameter λ in Filtered SVD, and to evaluate a specific principled choice: $\lambda = \sigma_1\sigma_n$, i.e., the product of the largest and smallest singular values.

The reasoning behind this choice is based on the dual requirement for λ:

- λ should satisfy $\boldsymbol{\lambda \ll \sigma_i^2}$ for the largest singular values, so that they are not affected by the regularization
- λ should satisfy $\boldsymbol{\lambda \gg \sigma_i^2}$ for the smallest singular values, so that their inversion does not cause noise amplification

The choice $\lambda = \sigma_1\sigma_n$ places the regularization parameter as a geometric mean between the largest squared singular value σ_1^2 and the smallest squared singular value σ_n^2. The filtered singular values are plotted alongside the original singular values for comparison. The reconstruction is then computed using this λ value, and PSNR and SSIM are evaluated.

Observations and Comments

Parameter values:

- σ_1 = $\mathbf{1.6791 \times 10^{11}}$ (largest singular value)
- σ_n = $\mathbf{2.4933 \times 10^{-6}}$ (smallest singular value)
- $\lambda = \sigma_1 \cdot \sigma_n$ = $\mathbf{4.19 \times 10^{5}}$

Filter behavior confirmed:

- Filter value at largest singular value: $\mathbf{y_1 = 1.000000}$ → largest singular values are completely unaffected ✓
- Filter value at smallest singular value: $\mathbf{y_n = 0.000000}$ → smallest singular values are completely suppressed ✓

Singular value plot analysis: The filtered singular values (red dashed line) closely follow the original singular values (blue solid line) for indices 1–~2000, confirming that large singular values are unaffected. Only around index ~2000 do the filtered singular values begin to diverge upward, reflecting the inflation of small singular values by the filter. The green dashed vertical line marks the transition point where $\sigma_i^2 \approx \lambda$.

Reconstruction quality is poor: Despite the theoretically motivated choice of λ, the reconstruction shows visible checkerboard noise artifacts, particularly in the lower portion of the image. PSNR = **19.997 dB** and SSIM = **0.5955** are significantly worse than the optimal filtered SVD results from Part 2.5 (PSNR = 31.96 dB, SSIM = 0.942).

Root cause — λ is too small: $\lambda = 4.19 \times 10^5$ falls far below the optimal range identified in Part 2.5 (10^{10}–10^{12}). This means the system is under-regularized — many low-energy singular values are still being inverted with insufficient suppression. The extreme dynamic range of singular values (~10^{17}) means that the geometric mean $\lambda = \sigma_1\sigma_n$ is still far too small to provide effective regularization.

Conclusion: While $\lambda = \sigma_1\sigma_n$ is a theoretically intuitive starting point, it is not optimal for this dataset due to the extreme ill-conditioning. A more systematic search over a range of λ values, as performed in Parts 2.5 and 2.7, is necessary to find the optimal regularization parameter.

Code Snippet and Command Window Output

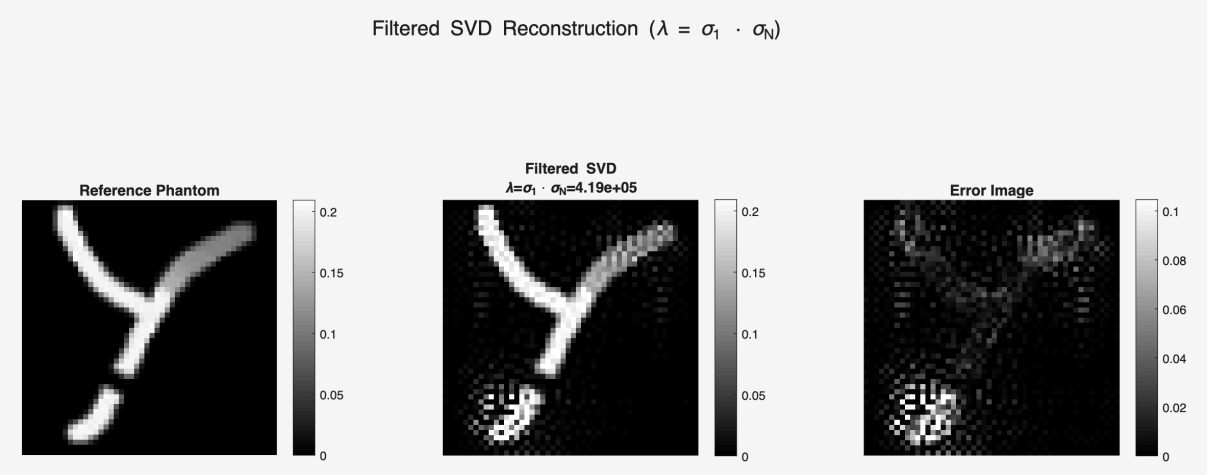
Code:

```
sigma_vals = diag(Sigma);
sigma_1 = sigma_vals(1);
sigma_N = sigma_vals(end);

% Choose lambda
lambda = sigma_1 * sigma_N;
fprintf('sigma_1 = %.4e\n', sigma_1);
fprintf('sigma_N = %.4e\n', sigma_N);
fprintf('lambda = sigma_1 * sigma_N = %.4e\n', lambda);

% Filter values
y = sigma_vals.^2 ./ (sigma_vals.^2 + lambda);

% Filtered singular values
sigma_filtered = sigma_vals ./ y;

% Plot original vs filtered singular values
figure;
subplot(1,2,1);
semilogy(sigma_vals, 'b-', 'LineWidth', 2); hold on;
semilogy(sigma_filtered, 'r--', 'LineWidth', 2);
xlabel('Index'); ylabel('Singular Value (log scale)');
title('Original vs Filtered Singular Values');
legend('\sigma_i (original)', '\sigma_i'' (filtered)');
grid on;

subplot(1,2,2);
semilogy(sigma_vals, 'b-', 'LineWidth', 2); hold on;
semilogy(sigma_filtered, 'r--', 'LineWidth', 2);
xline(find(sigma_vals.^2 >= lambda, 1, 'last'), 'g--', 'LineWidth', 1.5);
xlabel('Index'); ylabel('Singular Value (log scale)');
```

```
title('Zoom: Transition Region');
legend('\sigma_i (original)', '\sigma_i'' (filtered)');
grid on;
sgtitle(sprintf('Singular Value Filtering with \\lambda = %.2e', lambda));

% Reconstruct
Sigma_tilde_inv = diag(y ./ sigma_vals);
c = V * Sigma_tilde_inv * (U' * u);
ima_fsvd = reshape(c, 50, 50);
ima_fsvd(ima_fsvd < 0) = 0;

% Display
figure;
subplot(1,3,1);
imshow(phantom_ref, [0 max_val]); colormap gray; colorbar;
title('Reference Phantom'); axis image;

subplot(1,3,2);
imshow(ima_fsvd, [0 max_val]); colormap gray; colorbar;
title(sprintf('Filtered SVD\n\\lambda=%.2e', lambda)); axis image;

subplot(1,3,3);
error_img = abs(ima_fsvd - phantom_ref);
imshow(error_img, [0 0.5*max_val]); colormap gray; colorbar;
title('Error Image'); axis image;

sgtitle('Filtered SVD Reconstruction (\lambda = \sigma_1 \cdot \sigma_N)');

% Metrics
ima_fsvd_norm = ima_fsvd / max_val;
ref_norm = phantom_ref / max_val;
psnr_fsvd = psnr(ima_fsvd_norm, ref_norm);
ssim_fsvd = ssim(ima_fsvd_norm, ref_norm);

fprintf('PSNR: %.4f dB\n', psnr_fsvd);
fprintf('SSIM: %.4f\n', ssim_fsvd);
fprintf('Filter value at largest singular value (i=1):   y_1 = %.6f\n', y(1));
fprintf('Filter value at smallest singular value (i=N): y_N = %.6f\n', y(end));
```

Command Window Output:

```
sigma_1 = 1.6791e+11
sigma_N = 2.4933e-06
lambda = sigma_1 * sigma_N = 4.1866e+05
PSNR: 19.9971 dB
SSIM: 0.5955
Filter value at largest singular value (i=1):   y_1 = 1.000000
Filter value at smallest singular value (i=N): y_N = 0.000000
```



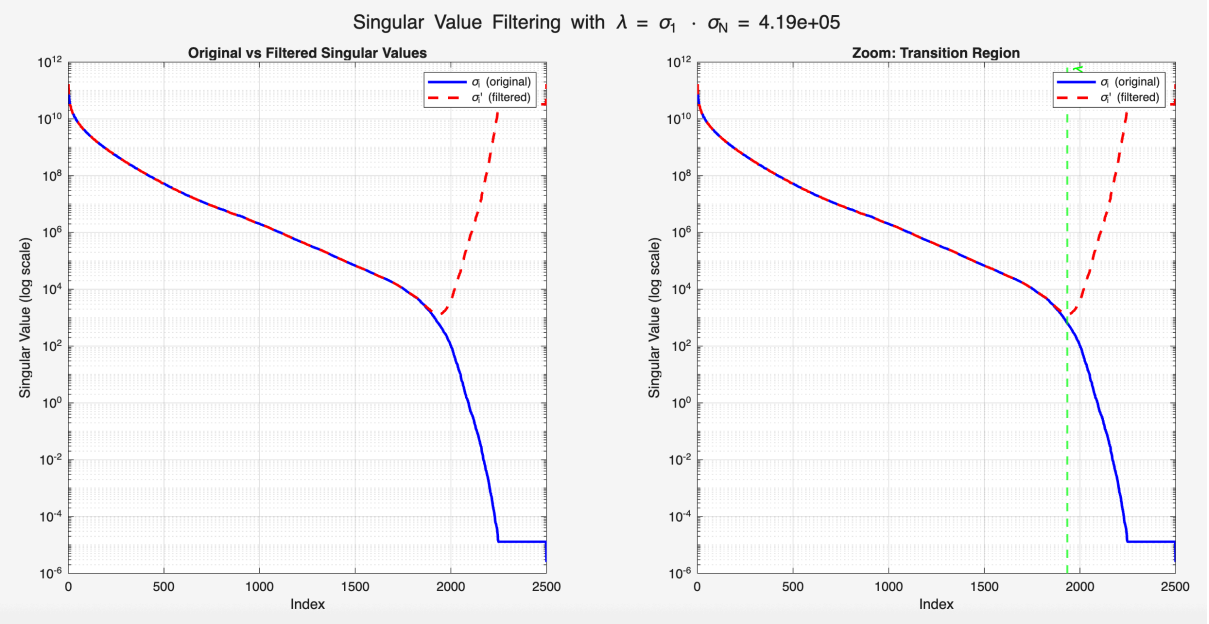


Figure 9: Part 2.6

2.7 - Finding Optimal λ*

Objective and Methodology

The objective of this section is to systematically search for the optimal regularization parameter λ* for Filtered SVD by evaluating a dense grid of λ values in the reasonable range identified from Parts 2.5 and 2.6. The reasonable range is defined as the range where λ is large enough to suppress small singular values but small enough to leave large singular values unaffected.

λ values are tested at half-decade intervals from 10^8 to 10^{16}:

```
lambdas = 10.^(8:0.5:16);
```

For each λ, the Filtered SVD reconstruction is computed and PSNR and SSIM are recorded. The optimal λ* is identified as the value yielding the highest PSNR. The reconstruction quality is then visualized for three specific cases: λ*/10 (under-regularized), λ* (optimal), and 10λ* (over-regularized), displayed in a single 2×3 figure.

Observations and Comments

λ	PSNR (dB)	SSIM
$10^{8.0}$	29.28	0.894
$10^{8.5}$	30.36	0.921
$10^{9.0}$	31.05	0.937
$10^{9.5}$	31.44	0.942
$10^{10.0}$	31.67	0.942
$10^{10.5}$	31.87	0.941
$10^{11.0}$	32.06	0.938
$10^{11.5}$	**32.16**	**0.939**
$10^{12.0}$	31.96	0.940
$10^{12.5}$	31.41	0.938
$10^{13.0}$	30.64	0.939
$10^{13.5}$	29.79	0.940
$10^{14.0}$	29.00	0.939
$10^{14.5}$	28.42	0.934
$10^{15.0}$	28.05	0.924
$10^{15.5}$	27.79	0.909
$10^{16.0}$	27.46	0.890

Table 5: Part 2.7 Summary of results

*Optimal $\lambda = 3.16 \times 10^{11}$:** The best PSNR of **32.16 dB** is achieved at $\lambda^* = 3.16 \times 10^{11}$. The best SSIM of **0.9423** is achieved at $\lambda = 10^{10}$, which is slightly different from the best PSNR point but within the same optimal range.

SSIM is relatively flat: SSIM remains above 0.93 for a wide range of λ values from 10^9 to 10^{14}, indicating that structural similarity is well-preserved across a broad range of regularization strengths. This suggests the reconstruction is robust to the exact choice of λ within this range.

PSNR has a sharper peak: PSNR peaks more sharply around $\lambda^* = 3.16 \times 10^{11}$ and decays more steeply on both sides, making it a more sensitive indicator of the optimal regularization point.

Case	λ	PSNR (dB)	SSIM
λ*/10	3.16×10^{10}	31.87	0.9405
λ*	**3.16×10^{11}**	**32.16**	**0.9391**
10λ*	3.16×10^{12}	31.41	0.9383

Table 6: Part 2.7 Effect of $\lambda/10$, λ, 10λ*:**

- *λ/10:*Slightly under-regularized — mild noise artifacts visible, PSNR slightly lower than optimal
- *λ:*Best reconstruction — clean Y-shape with sharp edges and minimal error
- *10λ:*Slightly over-regularized — mild blurring of edges, PSNR slightly lower than optimal

The relatively small differences between these three cases confirm the robustness of the reconstruction within the optimal λ range.

Code Snippet and Command Window Output

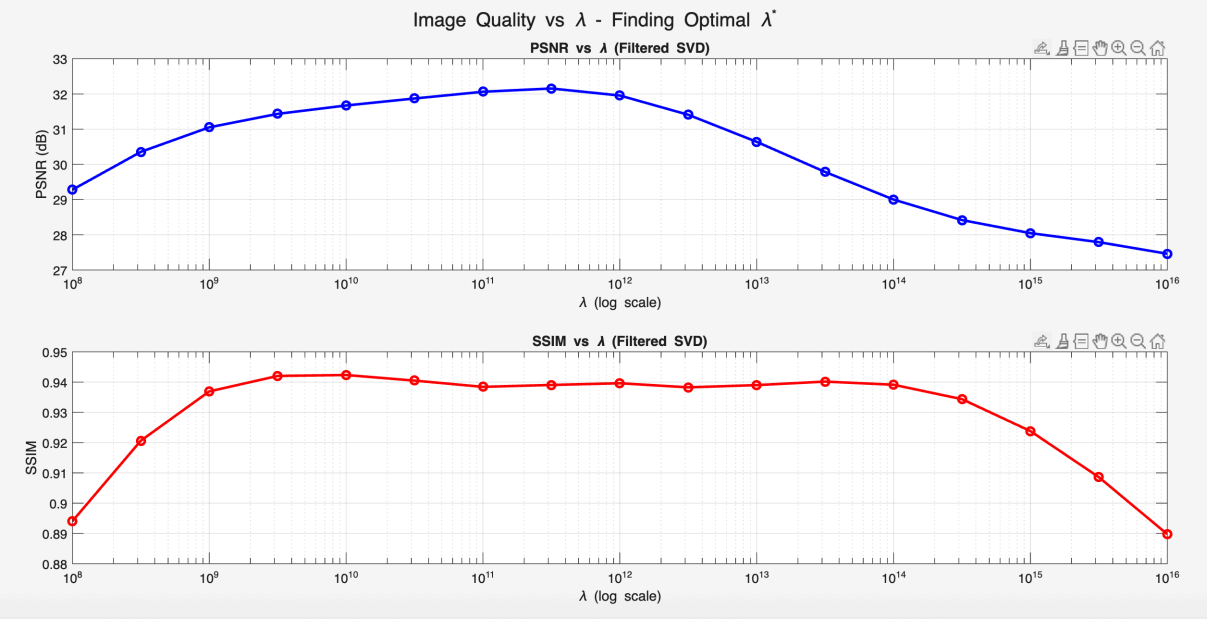
Code:

```
% Search for optimal lambda
lambdas = 10.^(8:0.5:16);
n_lambdas = length(lambdas);
psnr_vals = zeros(1, n_lambdas);
ssim_vals = zeros(1, n_lambdas);
ref_norm  = phantom_ref / max_val;
sigma_vals = diag(Sigma);

for idx = 1:n_lambdas
    lambda = lambdas(idx);
    y = sigma_vals.^2 ./ (sigma_vals.^2 + lambda);
    Sigma_tilde_inv = diag(y ./ sigma_vals);
    c = V * Sigma_tilde_inv * (U' * u);
    ima = reshape(c, 50, 50);
    ima(ima < 0) = 0;
    ima_norm = ima / max_val;
    psnr_vals(idx) = psnr(ima_norm, ref_norm);
    ssim_vals(idx) = ssim(ima_norm, ref_norm);
    fprintf('Lambda: %.2e | PSNR: %.4f dB | SSIM: %.4f\n', ...
        lambda, psnr_vals(idx), ssim_vals(idx));
end

% Plot
figure;
subplot(2,1,1);
semilogx(lambdas, psnr_vals, 'b-o', 'LineWidth', 2, 'MarkerSize', 6);
xlabel('\lambda (log scale)'); ylabel('PSNR (dB)');
title('PSNR vs \lambda (Filtered SVD)'); grid on;

subplot(2,1,2);
semilogx(lambdas, ssim_vals, 'r-o', 'LineWidth', 2, 'MarkerSize', 6);
xlabel('\lambda (log scale)'); ylabel('SSIM');
title('SSIM vs \lambda (Filtered SVD)'); grid on;
sgtitle('Image Quality vs \lambda - Finding Optimal \lambda^*');

[~, best_psnr_idx] = max(psnr_vals);
lambda_star = lambdas(best_psnr_idx);
fprintf('\nBest PSNR: %.4f dB at lambda* = %.2e\n', psnr_vals(best_psnr_idx), la
mbda_star);

% Display lambda*/10, lambda*, 10*lambda*
test_lambdas = [lambda_star/10, lambda_star, lambda_star*10];
test_labels  = {'\lambda^*/10', '\lambda^*', '10\lambda^*'};

figure('Units','normalized','Position',[0 0 0.9 0.6]);
for idx = 1:3
    lambda = test_lambdas(idx);
    y = sigma_vals.^2 ./ (sigma_vals.^2 + lambda);
    Sigma_tilde_inv = diag(y ./ sigma_vals);
```

```
    c = V * Sigma_tilde_inv * (U' * u);
    ima = reshape(c, 50, 50);
    ima(ima < 0) = 0;
    ima_norm = ima / max_val;
    p = psnr(ima_norm, ref_norm);
    s = ssim(ima_norm, ref_norm);

    subplot(2, 3, idx);
    imshow(ima, [0 max_val]); colormap gray; colorbar;
    title(sprintf('%s = %.2e\nPSNR=%.2f dB, SSIM=%.4f', ...
        test_labels{idx}, lambda, p, s), 'FontSize', 8);
    axis image;

    subplot(2, 3, 3 + idx);
    error_img = abs(ima - phantom_ref);
    imshow(error_img, [0 0.5*max_val]); colormap gray; colorbar;
    title(sprintf('Error - %s', test_labels{idx}), 'FontSize', 8);
    axis image;

    fprintf('%s = %.2e | PSNR: %.4f dB | SSIM: %.4f\n', ...
        test_labels{idx}, lambda, p, s);
end
sgtitle(sprintf('Filtered SVD: \\lambda^*/10, \\lambda^*, 10\\lambda^* (\\lambda
^* = %.2e)', ...
    lambda_star));
```

Command Window Output:

```
Lambda: 1.00e+08 | PSNR: 29.2802 dB | SSIM: 0.8940
Lambda: 3.16e+08 | PSNR: 30.3561 dB | SSIM: 0.9206
Lambda: 1.00e+09 | PSNR: 31.0534 dB | SSIM: 0.9369
Lambda: 3.16e+09 | PSNR: 31.4358 dB | SSIM: 0.9420
Lambda: 1.00e+10 | PSNR: 31.6741 dB | SSIM: 0.9423
Lambda: 3.16e+10 | PSNR: 31.8731 dB | SSIM: 0.9405
Lambda: 1.00e+11 | PSNR: 32.0639 dB | SSIM: 0.9384
Lambda: 3.16e+11 | PSNR: 32.1563 dB | SSIM: 0.9391
Lambda: 1.00e+12 | PSNR: 31.9573 dB | SSIM: 0.9396
Lambda: 3.16e+12 | PSNR: 31.4101 dB | SSIM: 0.9383
Lambda: 1.00e+13 | PSNR: 30.6410 dB | SSIM: 0.9390
Lambda: 3.16e+13 | PSNR: 29.7851 dB | SSIM: 0.9402
Lambda: 1.00e+14 | PSNR: 29.0009 dB | SSIM: 0.9391
Lambda: 3.16e+14 | PSNR: 28.4175 dB | SSIM: 0.9344
Lambda: 1.00e+15 | PSNR: 28.0465 dB | SSIM: 0.9238
Lambda: 3.16e+15 | PSNR: 27.7907 dB | SSIM: 0.9086
Lambda: 1.00e+16 | PSNR: 27.4607 dB | SSIM: 0.8897
Best PSNR: 32.1563 dB at lambda* = 3.16e+11
Best SSIM: 0.9423 at lambda = 1.00e+10
\lambda^*/10 = 3.16e+10 | PSNR: 31.8730 dB | SSIM: 0.9405
\lambda^*    = 3.16e+11 | PSNR: 32.1563 dB | SSIM: 0.9391
10\lambda^*  = 3.16e+12 | PSNR: 31.4105 dB | SSIM: 0.9383
```



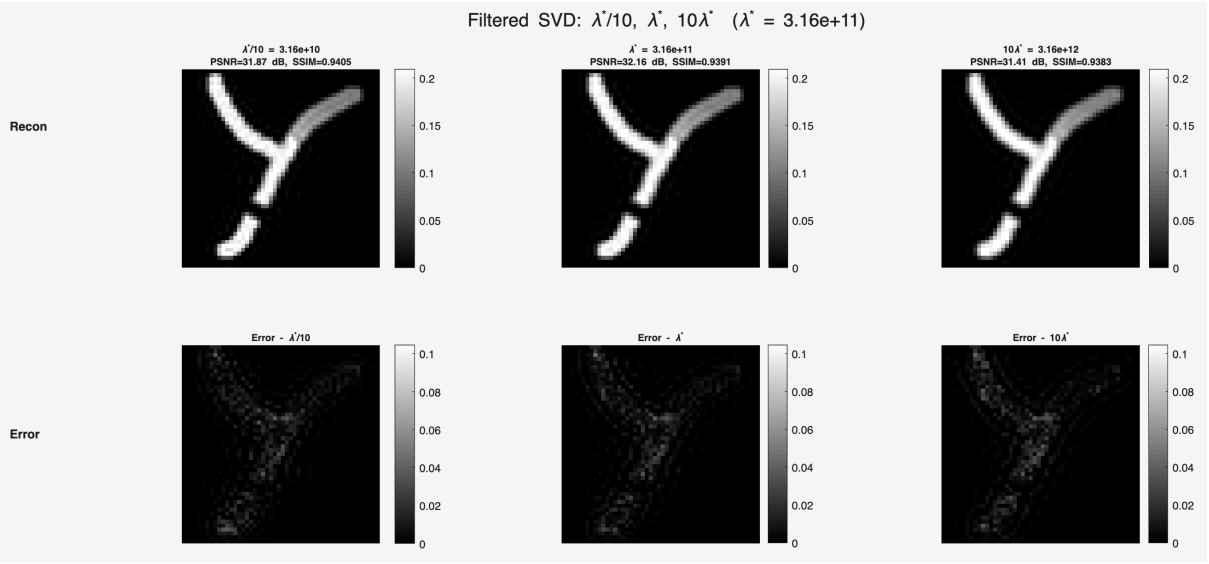


Figure 10: Part 2.7

2.8 - L-Curve

Objective and Methodology

The objective of this section is to determine the optimal regularization parameter λ* using the L-curve method, which does not require knowledge of the reference image. The L-curve is a log-log plot of the residual norm `||Sc - u||` (x-axis) versus the solution norm `||c||` (y-axis), computed for a range of λ values. The optimal λ* is typically located at the corner of this L-shaped curve, representing the best trade-off between data fidelity and solution regularity.

The L-curve is computed for λ values at quarter-decade intervals from 10^6 to 10^{20}:

```
lambdas = 10.^(6:0.25:20);
```

Two approaches are used to identify the corner:

1. **Automatic detection:** The corner is identified by finding the point of maximum curvature in the log-log space, computed numerically using the gradient of the curve
2. **Manual detection:** The corner is identified by visual inspection of the L-curve plot

Both λ values are used to reconstruct the image, and PSNR and SSIM are compared.

Observations and Comments

L-curve shape: The L-curve shows a characteristic shape with two distinct regions:

- **Flat horizontal region** ($\lambda \sim 10^8$ to 10^{14}): The solution norm remains relatively constant while the residual norm decreases slowly. In this region, the reconstruction is well-regularized and the solution norm is stable
- **Steep vertical drop** ($\lambda > 10^{16}$): The solution norm drops rapidly as the reconstruction becomes heavily over-regularized and approaches zero

Automatic corner detection fails: The numerical curvature maximization identifies $\lambda =$ **3.16×10^6** as the corner, yielding PSNR = 24.49 dB and SSIM = 0.7585. This is significantly worse than the optimal results from Part 2.7. The failure occurs because the L-curve has a very gradual, smooth bend rather than a sharp corner, making the numerical curvature computation unreliable.

Manual corner identification: Visual inspection of the L-curve suggests the true corner is around $\lambda =$ **3.16×10^{11}**, where the curve transitions from the flat horizontal region to the steeper descent. This is consistent with the optimal λ^* found in Part 2.7 using PSNR and SSIM directly.

Method	λ	PSNR (dB)	SSIM
Automatic L-curve corner	3.16×10^6	24.49	0.7585
Manual L-curve corner	**3.16×10^{11}**	**32.16**	**0.9391**

Table 7: Part 2.8 Comparison of results

Key insight — L-curve limitations: The L-curve method is useful as a reference-free approach to parameter selection, but its effectiveness depends on the sharpness of the corner. For this MPI system matrix, the gradual curvature makes automatic corner detection unreliable. However, visual inspection of the L-curve still provides a useful guide, confirming that λ should be in the range 10^{10}–10^{12} where the curve begins to bend.

Flat SSIM region confirms robustness: The flat horizontal region of the L-curve corresponds to the range where SSIM remains above 0.93 in Part 2.7, confirming that the reconstruction is structurally robust across a wide range of λ values in this region.

Code Snippet and Command Window Output

Code:

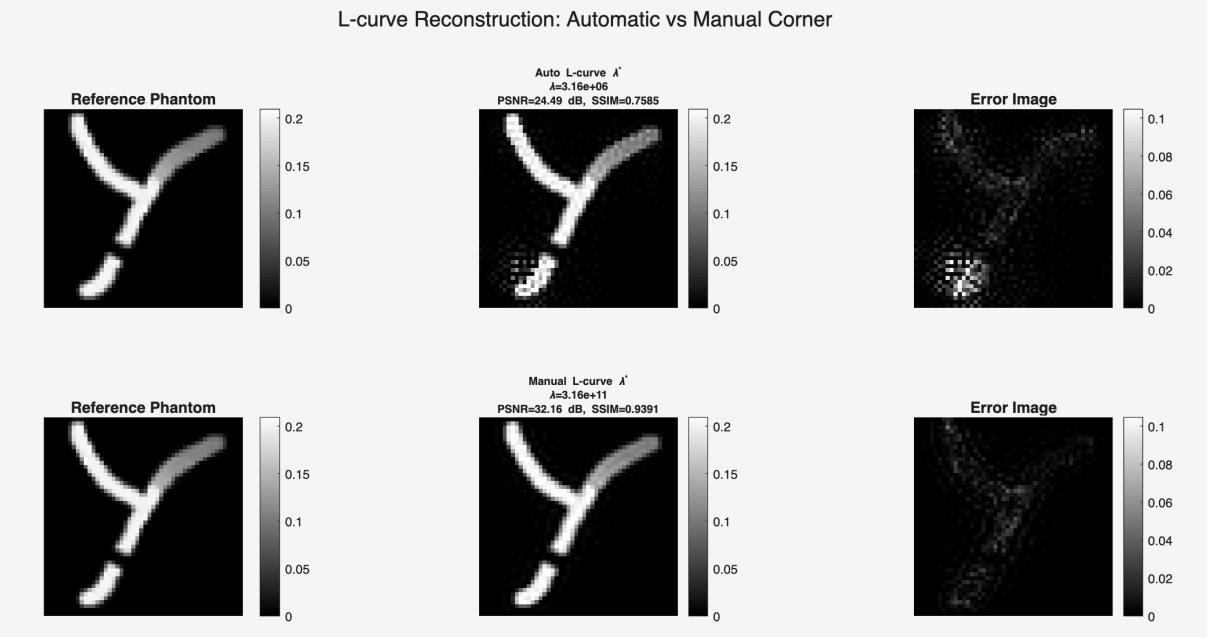
```
lambdas = 10.^(6:0.25:20);
n_lambdas = length(lambdas);
residual_norms = zeros(1, n_lambdas);
solution_norms = zeros(1, n_lambdas);
sigma_vals = diag(Sigma);
ref_norm = phantom_ref / max_val;

for idx = 1:n_lambdas
    lambda = lambdas(idx);
    y = sigma_vals.^2 ./ (sigma_vals.^2 + lambda);
```

```
    Sigma_tilde_inv = diag(y ./ sigma_vals);
    c = V * Sigma_tilde_inv * (U' * u);
    residual_norms(idx) = norm(S*c - u);
    solution_norms(idx) = norm(c);
end

% Plot L-curve
figure;
loglog(residual_norms, solution_norms, 'b-o', 'LineWidth', 2, 'MarkerSize', 5);
xlabel('Residual Norm ||Sc - u||');
ylabel('Solution Norm ||c||');
grid on;

label_indices = 1:4:n_lambdas;
for i = label_indices
    text(residual_norms(i), solution_norms(i), ...
        sprintf('  10^{%.1f}', log10(lambdas(i))), ...
        'FontSize', 7, 'Color', 'red');
end

% Automatic corner detection
log_res = log10(residual_norms);
log_sol = log10(solution_norms);
dx = gradient(log_res); dy = gradient(log_sol);
ddx = gradient(dx);     ddy = gradient(dy);
curvature = abs(dx.*ddy - dy.*ddx) ./ (dx.^2 + dy.^2).^1.5;
[~, corner_idx] = max(curvature);
lambda_auto = lambdas(corner_idx);

% Manual corner
lambda_lcurve_manual = 3.16e+11;

% Mark both on L-curve
hold on;
plot(residual_norms(corner_idx), solution_norms(corner_idx), ...
    'r*', 'MarkerSize', 15, 'LineWidth', 2);
manual_idx = find(abs(lambdas - lambda_lcurve_manual) == ...
    min(abs(lambdas - lambda_lcurve_manual)));
plot(residual_norms(manual_idx), solution_norms(manual_idx), ...
    'g*', 'MarkerSize', 15, 'LineWidth', 2);
legend('L-curve', ...
    sprintf('Auto corner: \\lambda = %.2e', lambda_auto), ...
    sprintf('Manual corner: \\lambda = %.2e', lambda_lcurve_manual), ...
    'Location', 'best');
title('L-Curve');

fprintf('Automatic L-curve corner at lambda = %.2e\n', lambda_auto);
fprintf('Manual L-curve corner at lambda = %.2e\n', lambda_lcurve_manual);

% Reconstruct for both
lambdas_to_test = [lambda_auto, lambda_lcurve_manual];
```

```
labels = {'Auto L-curve \lambda^*', 'Manual L-curve \lambda^*'};

figure('Units','normalized','Position',[0 0 0.9 0.6]);
for idx = 1:2
    lambda = lambdas_to_test(idx);
    y = sigma_vals.^2 ./ (sigma_vals.^2 + lambda);
    Sigma_tilde_inv = diag(y ./ sigma_vals);
    c = V * Sigma_tilde_inv * (U' * u);
    ima_lc = reshape(c, 50, 50);
    ima_lc(ima_lc < 0) = 0;
    ima_lc_norm = ima_lc / max_val;
    p = psnr(ima_lc_norm, ref_norm);
    s = ssim(ima_lc_norm, ref_norm);

    fprintf('%s: lambda = %.2e | PSNR: %.4f dB | SSIM: %.4f\n', ...
        labels{idx}, lambda, p, s);

    subplot(2, 3, (idx-1)*3 + 1);
    imshow(phantom_ref, [0 max_val]); colormap gray; colorbar;
    title('Reference Phantom'); axis image;

    subplot(2, 3, (idx-1)*3 + 2);
    imshow(ima_lc, [0 max_val]); colormap gray; colorbar;
    title(sprintf('%s\n\\lambda=%.2e\nPSNR=%.2f dB, SSIM=%.4f', ...
        labels{idx}, lambda, p, s), 'FontSize', 8);
    axis image;

    subplot(2, 3, (idx-1)*3 + 3);
    error_img = abs(ima_lc - phantom_ref);
    imshow(error_img, [0 0.5*max_val]); colormap gray; colorbar;
    title('Error Image'); axis image;
end
sgtitle('L-curve Reconstruction: Automatic vs Manual Corner');
```

Command Window Output:

```
Automatic L-curve corner at lambda = 3.16e+06
Manual L-curve corner at lambda = 3.16e+11
Auto L-curve lambda*:   lambda = 3.16e+06 | PSNR: 24.4948 dB | SSIM: 0.7585
Manual L-curve lambda*: lambda = 3.16e+11 | PSNR: 32.1563 dB | SSIM: 0.9391
```



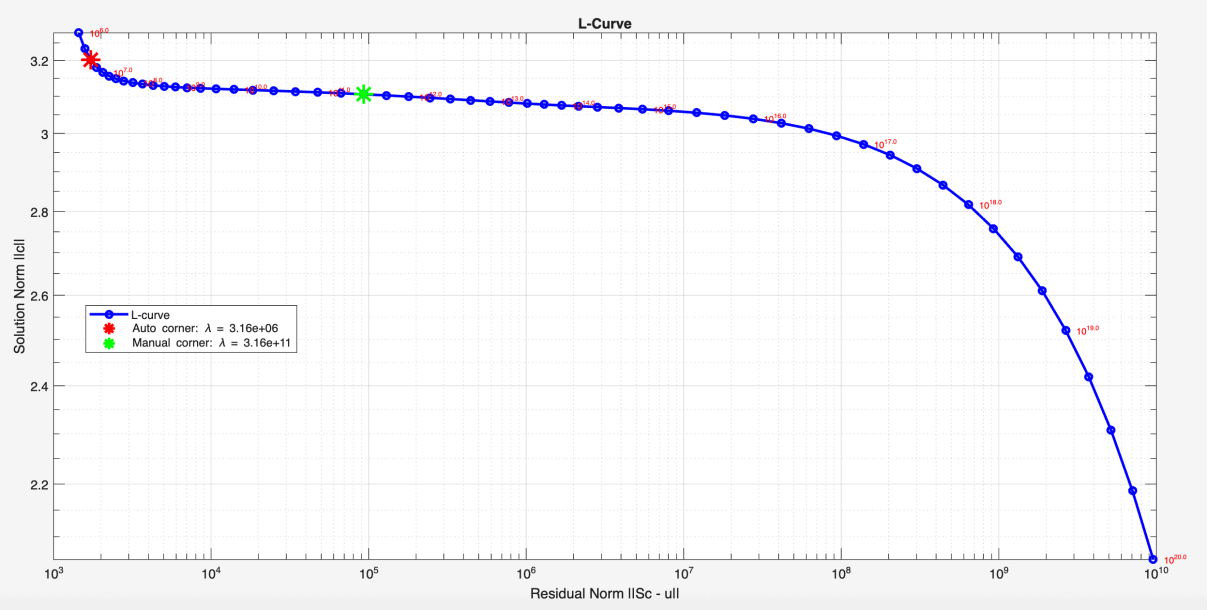


Figure 11: Part 2.8

Part 3 - Kaczmarz Method

3.1 - Row-Norm Thresholding

Objective and Methodology

The objective of this section is to preprocess the system matrix `S` by removing rows with small energy before applying the Kaczmarz method. This step is necessary because the standard Kaczmarz algorithm inherently normalizes each row by its squared norm during updates. Rows with near-zero energy produce very small denominators, causing severe noise amplification that degrades reconstruction quality.

The preprocessing steps are as follows:

Step 1 - Compute row norms: The 2-norm of each row of `S` is computed:

```
row_norms = vecnorm(S, 2, 2);
```

Step 2 - Define threshold: The maximum row norm is defined as `max_norm = max(row_norms)`. The threshold is set at `max_norm/10`, meaning any row with norm smaller than 10% of the maximum row norm is discarded.

Step 3 - Threshold rows: Rows of `S` and corresponding entries of `u` with norm below the threshold are discarded:

```
keep_rows = row_norms >= threshold;
S_thresh = S(keep_rows, :);
u_thresh = u(keep_rows);
```

The row norms are plotted against row index, with the threshold line overlaid for visual reference.

Observations and Comments

max_norm = 1.2031×10^{11}: The maximum row norm is very large, consistent with the large magnitude of the MPI signal at certain frequency components.

Only 44 out of 3264 rows retained: After thresholding, only **44 rows (1.35%)** pass the threshold. The remaining **3220 rows (98.65%)** are discarded. This reflects the extreme sparsity of the MPI signal in the frequency domain — only a small fraction of frequency components carry significant signal energy.

Row norm plot reveals sparse signal structure: The plot shows two clusters of high-norm rows:

- First cluster around row indices 1–300: corresponding to the real parts of the low-frequency harmonics
- Second cluster around indices 1600–2100: corresponding to the imaginary parts

The vast majority of rows have near-zero norms, confirming the sparsity of the MPI signal.

Threshold at max_norm/10 is aggressive but effective: The red dashed threshold line at 1.20×10^{10} cleanly separates the 44 signal-carrying rows from the 3220 noise-dominated rows. Including low-norm rows would cause the Kaczmarz update `(u_i - s_i'*c) / ||s_i||²` to have a very small denominator, amplifying noise by factors proportional to `1/||s_i||²`.

Sizes after thresholding:

- `S_thresh`: **44 × 2500**
- `u_thresh`: **44 × 1**

This dramatically reduces the problem size from 3264×2500 to 44×2500, making Kaczmarz iterations much faster and more stable.

Code Snippet and Command Window Output

Code:

```
% Compute norm of each row of S
row_norms = vecnorm(S, 2, 2);

% Define max_norm
max_norm = max(row_norms);
fprintf('max_norm = %.4e\n', max_norm);

% Plot row norms
figure;
```

```
plot(row_norms, 'b-', 'LineWidth', 1);
hold on;
yline(max_norm/10, 'r--', 'LineWidth', 2, 'Label', 'max\_norm/10 threshold');
xlabel('Row Index');
ylabel('Row Norm');
title('Row Norms of System Matrix S');
legend('Row norms', 'Threshold (max\_norm/10)', 'Location', 'best');
grid on;

% Find rows above threshold
threshold = max_norm / 10;
keep_rows = row_norms >= threshold;
fprintf('Number of rows kept: %d out of %d\n', sum(keep_rows), size(S,1));
fprintf('Number of rows discarded: %d\n', sum(~keep_rows));

% Threshold S and u
S_thresh = S(keep_rows, :);
u_thresh = u(keep_rows);

fprintf('Size of S after thresholding: %d x %d\n', size(S_thresh,1), size(S_thre
sh,2));
fprintf('Size of u after thresholding: %d x %d\n', size(u_thresh,1), size(u_thre
sh,2));
```

Command Window Output:

```
max_norm = 1.2031e+11
Number of rows kept: 44 out of 3264
Number of rows discarded: 3220
Size of S after thresholding: 44 x 2500
Size of u after thresholding: 44 x 1
```

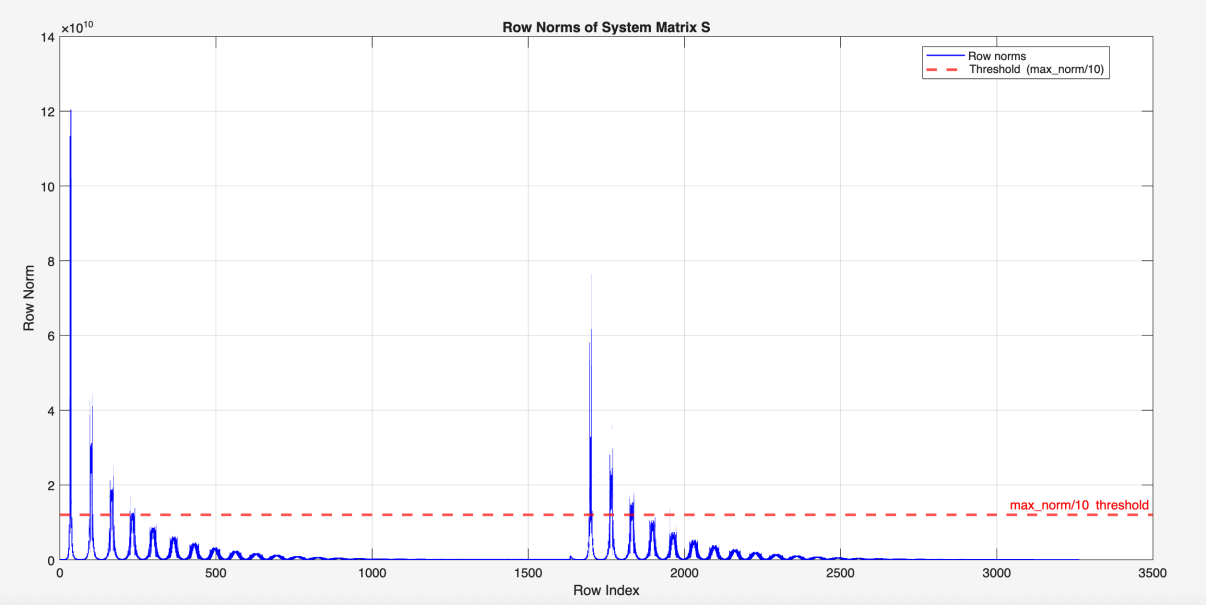


Figure 12: Part 3.1

3.2 - Standard Kaczmarz Method

Objective and Methodology

The objective of this section is to implement the standard Kaczmarz method using the thresholded system matrix `S_thresh` (44×2500) and measurement vector `u_thresh` (44×1) from Part 3.1. The Kaczmarz method is an iterative row-action algorithm that projects the current solution estimate onto the hyperplane defined by each row equation in sequence.

The update rule for the standard Kaczmarz method is:

$$c^{(k+1)} = c^{(k)} + \frac{u_i - s_i^T c^{(k)}}{|s_i|^2} s_i$$

where s_i is the i^{th} row of `S_thresh` (as a column vector), u_i is the corresponding entry of `u_thresh`, and $|s_i|^2$ is the squared row norm.

The implementation details are:

- The solution vector `c` is initialized as a zero vector of size 2500×1
- At each iteration, all 44 rows are visited in a **randomized order** using `randperm()`
- After each full iteration (all 44 rows visited), negative entries of `c` are set to zero
- The algorithm is run for 10 iterations
- At each iteration, the reconstructed image is displayed alongside its error image in a single 2×10 figure
- PSNR and SSIM are computed and plotted as functions of iteration count

Observations and Comments

Iteration	PSNR (dB)	SSIM
1	13.31	0.326
2	13.61	0.360
3	13.75	0.362
4	13.87	0.380
5	13.97	0.395
6	14.06	0.403
7	14.14	0.432
8	14.22	0.451
9	14.29	0.443
10	14.34	0.465

Table 8: Part 3.2 Summary of results

Monotonically increasing PSNR: PSNR increases steadily from 13.31 to 14.34 dB over 10 iterations, showing consistent convergence with each pass through the rows.

Slow convergence: The improvement per iteration is small (~0.1 dB/iter), suggesting many more iterations would be needed to fully converge. The reconstruction is still far from the reference quality achieved by SVD methods.

Heavy blurring: The Y-shape is visible but very smooth and blurry, with no sharp edges. This is because only 44 rows constrain 2500 unknowns — the system is highly underdetermined, meaning many solutions

are consistent with the measurements and the Kaczmarz method converges to the minimum norm solution.

SSIM fluctuations: SSIM shows slight non-monotonic fluctuations (e.g., iteration 9 dips slightly compared to iteration 8) due to the random row ordering, which introduces stochasticity in each iteration.

Error images: The error concentrates along the edges of the Y-shape, indicating the reconstruction captures the rough shape but misses fine structural details and sharp boundaries.

Poor compared to SVD methods: Best PSNR of 14.34 dB after 10 iterations is far below the best filtered SVD result of 32.16 dB, demonstrating that standard Kaczmarz with aggressive thresholding (only 44 rows) is insufficient for high-quality reconstruction.

Note on randomization: Because of the random row ordering via `randperm()`, results will differ slightly between runs. The values reported here are from one specific run.

Code Snippet and Command Window Output

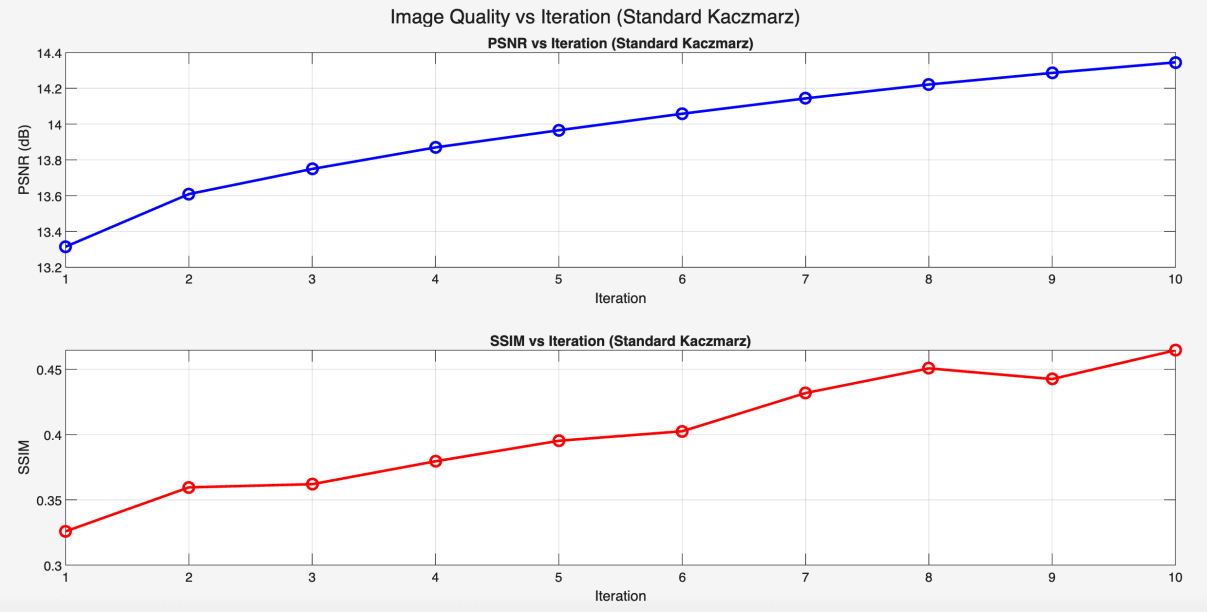
Code:

```
n_iter = 10;
n_rows = size(S_thresh, 1);   % 44
n_cols = size(S_thresh, 2);   % 2500

% Precompute row norms squared
row_norms_thresh = vecnorm(S_thresh, 2, 2);
row_norms_sq = row_norms_thresh.^2;

% Initialize c
c = zeros(n_cols, 1);
ref_norm = phantom_ref / max_val;

psnr_kacz = zeros(1, n_iter);
ssim_kacz = zeros(1, n_iter);

figure('Units','normalized','Position',[0 0 1 0.6]);

for iter = 1:n_iter
    % Randomize row order
    row_order = randperm(n_rows);

    % Sub-iterations
    for k = 1:n_rows
        i = row_order(k);
        s_i = S_thresh(i, :)';
        u_i = u_thresh(i);
        c = c + ((u_i - s_i'*c) / row_norms_sq(i)) * s_i;
    end

    % Set negatives to zero
    c(c < 0) = 0;

    ima = reshape(c, 50, 50);
    ima_norm = ima / max_val;
    psnr_kacz(iter) = psnr(ima_norm, ref_norm);
```

```
    ssim_kacz(iter) = ssim(ima_norm, ref_norm);

    fprintf('Iter %2d | PSNR: %.4f dB | SSIM: %.4f\n', ...
        iter, psnr_kacz(iter), ssim_kacz(iter));

    subplot(2, n_iter, iter);
    imshow(ima, [0 max_val]); colormap gray;
    title(sprintf('Iter %d', iter), 'FontSize', 7); axis image;

    subplot(2, n_iter, n_iter + iter);
    error_img = abs(ima - phantom_ref);
    imshow(error_img, [0 0.5*max_val]); colormap gray;
    title(sprintf('PSNR=%.1f\nSSIM=%.2f', ...
        psnr_kacz(iter), ssim_kacz(iter)), 'FontSize', 7); axis image;
end

sgtitle('Standard Kaczmarz: Iterations 1-10');

figure;
subplot(2,1,1);
plot(1:n_iter, psnr_kacz, 'b-o', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('Iteration'); ylabel('PSNR (dB)');
title('PSNR vs Iteration (Standard Kaczmarz)');
xticks(1:n_iter); grid on;

subplot(2,1,2);
plot(1:n_iter, ssim_kacz, 'r-o', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('Iteration'); ylabel('SSIM');
title('SSIM vs Iteration (Standard Kaczmarz)');
xticks(1:n_iter); grid on;

sgtitle('Image Quality vs Iteration (Standard Kaczmarz)');
```

Command Window Output:

```
Iter  1 | PSNR: 13.3146 dB | SSIM: 0.3258
Iter  2 | PSNR: 13.6089 dB | SSIM: 0.3595
Iter  3 | PSNR: 13.7492 dB | SSIM: 0.3620
Iter  4 | PSNR: 13.8695 dB | SSIM: 0.3795
Iter  5 | PSNR: 13.9655 dB | SSIM: 0.3953
Iter  6 | PSNR: 14.0578 dB | SSIM: 0.4025
Iter  7 | PSNR: 14.1435 dB | SSIM: 0.4318
Iter  8 | PSNR: 14.2211 dB | SSIM: 0.4508
Iter  9 | PSNR: 14.2860 dB | SSIM: 0.4426
Iter 10 | PSNR: 14.3449 dB | SSIM: 0.4646
```



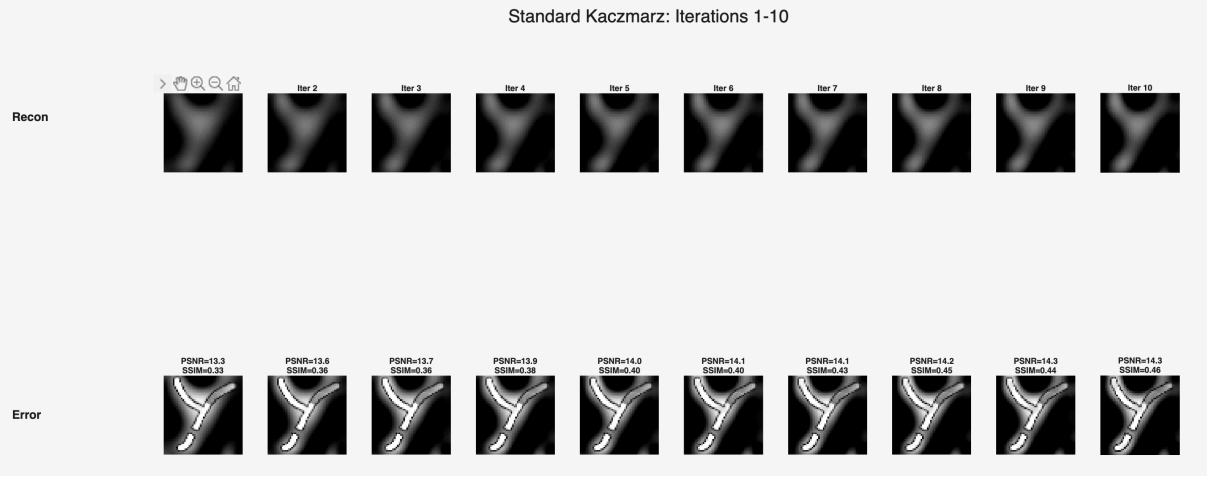


Figure 13: Part 3.2

3.3 - Standard Kaczmarz with Different Row-Norm Thresholds

Objective and Methodology

The objective of this section is to investigate the effect of the row-norm threshold on reconstruction quality by repeating the standard Kaczmarz method for four different threshold factors: `max_norm × [10⁻⁴, 10⁻³, 10⁻², 10⁻¹]`. For each threshold, the corresponding rows of `S` and entries of `u` are retained, and 10 iterations of standard Kaczmarz are performed. Only the reconstruction at the 10th iteration is displayed and evaluated. All reconstructed images and error images are shown in a single 2×4 figure. PSNR and SSIM are plotted as functions of threshold factor on a logarithmic x-axis.

Note that the threshold factor `10⁻¹` corresponds to the case studied in Part 3.2 (44 rows kept).

Observations and Comments

Threshold	Rows Kept	PSNR (dB)	SSIM
max_norm × 10^{-4}	3114	28.95	0.917
max_norm × 10^{-3}	1749	27.93	**0.931**

Threshold	Rows Kept	PSNR (dB)	SSIM
max_norm × 10^{-2}	486	23.07	0.744
max_norm × 10^{-1}	44	14.30	0.449

Table 9: Part 3.3 Summary of results

High threshold (10^{-1}, 44 rows): Worst quality — only 44 rows retained, the system is extremely underdetermined (44 equations, 2500 unknowns). Images are heavily blurred with no fine detail. PSNR = 14.30 dB, SSIM = 0.449.

Medium threshold (10^{-2}, 486 rows): Moderate quality — more rows provide better constraints, the Y-shape is more recognizable but still somewhat blurry. PSNR = 23.07 dB, SSIM = 0.744.

Low threshold (10^{-3}, 1749 rows): Best SSIM of 0.931 — sharp edges, clean reconstruction with good structural accuracy. Including more rows significantly improves the quality as more frequency components constrain the solution.

Very low threshold (10^{-4}, 3114 rows): Highest PSNR of 28.95 dB but slightly lower SSIM (0.917) than 10^{-3}. Including very low-norm rows introduces mild noise amplification that slightly degrades structural similarity despite improving overall intensity accuracy.

Optimal threshold is around 10^{-3}: Best overall trade-off between keeping enough signal rows and excluding noisy low-energy rows, achieving SSIM = 0.931.

Key insight: Unlike SVD methods where all frequency components are used simultaneously, the Kaczmarz method is highly sensitive to which rows are included. Too few rows leads to an underdetermined system with blurry reconstructions, while too many rows introduces noise amplification from low-energy rows.

Dramatic improvement over Part 3.2: Using a lower threshold (10^{-3} or 10^{-4}) instead of the default 10^{-1} dramatically improves quality — PSNR improves from 14.30 dB to 27.93–28.95 dB, and SSIM from 0.449 to 0.917–0.931.

Code Snippet and Command Window Output

Code:

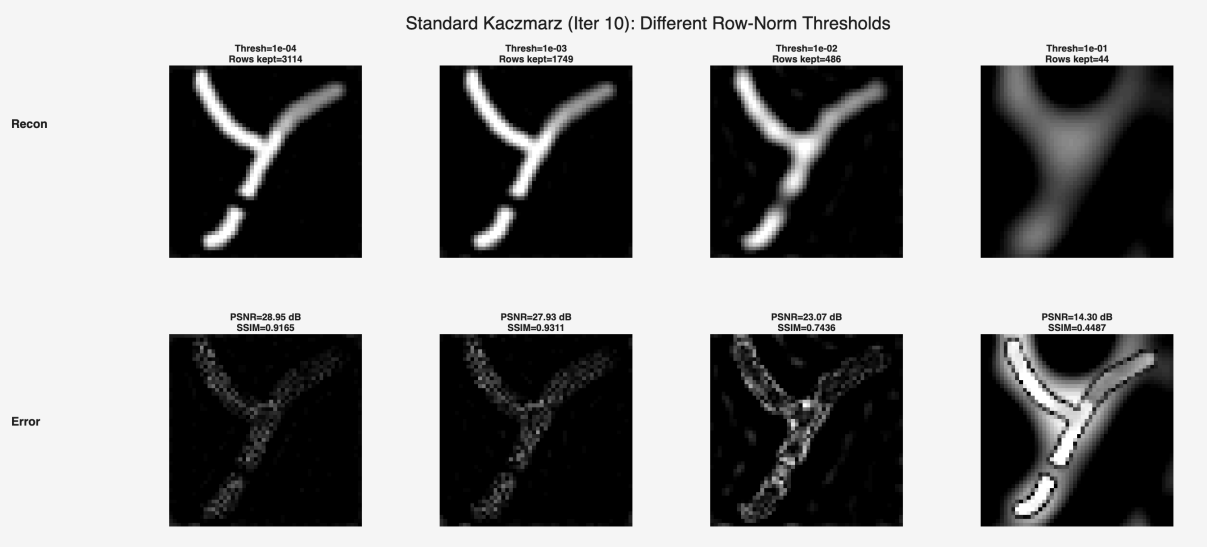
```
threshold_factors = [1e-4, 1e-3, 1e-2, 1e-1];
n_thresh = length(threshold_factors);
n_iter = 10;

row_norms_full = vecnorm(S, 2, 2);
max_norm = max(row_norms_full);

psnr_thresh = zeros(1, n_thresh);
ssim_thresh = zeros(1, n_thresh);
rows_kept   = zeros(1, n_thresh);
ref_norm    = phantom_ref / max_val;

figure('Units','normalized','Position',[0 0 1 0.6]);

for t = 1:n_thresh
    factor = threshold_factors(t);
    threshold = max_norm * factor;
```

```
    keep_rows = row_norms_full >= threshold;
    S_t = S(keep_rows, :);
    u_t = u(keep_rows);
    rows_kept(t) = sum(keep_rows);

    fprintf('Threshold: max_norm x %.0e | Rows kept: %d\n', factor, rows_kept
(t));

    rn_sq = vecnorm(S_t, 2, 2).^2;
    n_rows_t = size(S_t, 1);
    c = zeros(2500, 1);

    for iter = 1:n_iter
        row_order = randperm(n_rows_t);
        for k = 1:n_rows_t
            i = row_order(k);
            s_i = S_t(i, :)';
            u_i = u_t(i);
            c = c + ((u_i - s_i'*c) / rn_sq(i)) * s_i;
        end
        c(c < 0) = 0;
    end

    ima = reshape(c, 50, 50);
    ima_norm = ima / max_val;
    psnr_thresh(t) = psnr(ima_norm, ref_norm);
    ssim_thresh(t) = ssim(ima_norm, ref_norm);

    fprintf('  Iter 10 | PSNR: %.4f dB | SSIM: %.4f\n', psnr_thresh(t), ssim_thr
esh(t));

    subplot(2, n_thresh, t);
    imshow(ima, [0 max_val]); colormap gray;
    title(sprintf('Thresh=%.0e\nRows kept=%d', factor, rows_kept(t)), 'FontSiz
e', 8);
    axis image;

    subplot(2, n_thresh, n_thresh + t);
    error_img = abs(ima - phantom_ref);
    imshow(error_img, [0 0.5*max_val]); colormap gray;
    title(sprintf('PSNR=%.2f dB\nSSIM=%.4f', psnr_thresh(t), ssim_thresh(t)), 'F
ontSize', 8);
    axis image;
end

sgtitle('Standard Kaczmarz (Iter 10): Different Row-Norm Thresholds');

figure;
subplot(2,1,1);
semilogx(threshold_factors, psnr_thresh, 'b-o', 'LineWidth', 2, 'MarkerSize',
8);
```

```
xlabel('Threshold Factor (log scale)'); ylabel('PSNR (dB)');
title('PSNR vs Row-Norm Threshold (Iter 10)');
xticks(threshold_factors);
xticklabels({'10^{-4}','10^{-3}','10^{-2}','10^{-1}'});
grid on;

subplot(2,1,2);
semilogx(threshold_factors, ssim_thresh, 'r-o', 'LineWidth', 2, 'MarkerSize',
8);
xlabel('Threshold Factor (log scale)'); ylabel('SSIM');
title('SSIM vs Row-Norm Threshold (Iter 10)');
xticks(threshold_factors);
xticklabels({'10^{-4}','10^{-3}','10^{-2}','10^{-1}'});
grid on;

sgtitle('Image Quality vs Row-Norm Threshold (Standard Kaczmarz, Iter 10)');

fprintf('\nSummary at Iteration 10:\n');
fprintf('%-20s %-15s %-15s %-15s\n','Threshold','Rows Kept','PSNR (dB)','SSIM');
for t = 1:n_thresh
    fprintf('max_norm x %.0e   %-15d %-15.4f %-15.4f\n', ...
        threshold_factors(t), rows_kept(t), psnr_thresh(t), ssim_thresh(t));
end
```

Command Window Output:

```
Threshold: max_norm x 1e-04 | Rows kept: 3114
  Iter 10 | PSNR: 28.9529 dB | SSIM: 0.9165
Threshold: max_norm x 1e-03 | Rows kept: 1749
  Iter 10 | PSNR: 27.9286 dB | SSIM: 0.9311
Threshold: max_norm x 1e-02 | Rows kept: 486
  Iter 10 | PSNR: 23.0736 dB | SSIM: 0.7436
Threshold: max_norm x 1e-01 | Rows kept: 44
  Iter 10 | PSNR: 14.2973 dB | SSIM: 0.4487

Summary at Iteration 10:
Threshold           Rows Kept       PSNR (dB)       SSIM
max_norm x 1e-04    3114            28.9529         0.9165
max_norm x 1e-03    1749            27.9286         0.9311
max_norm x 1e-02    486             23.0736         0.7436
max_norm x 1e-01    44              14.2973         0.4487
```



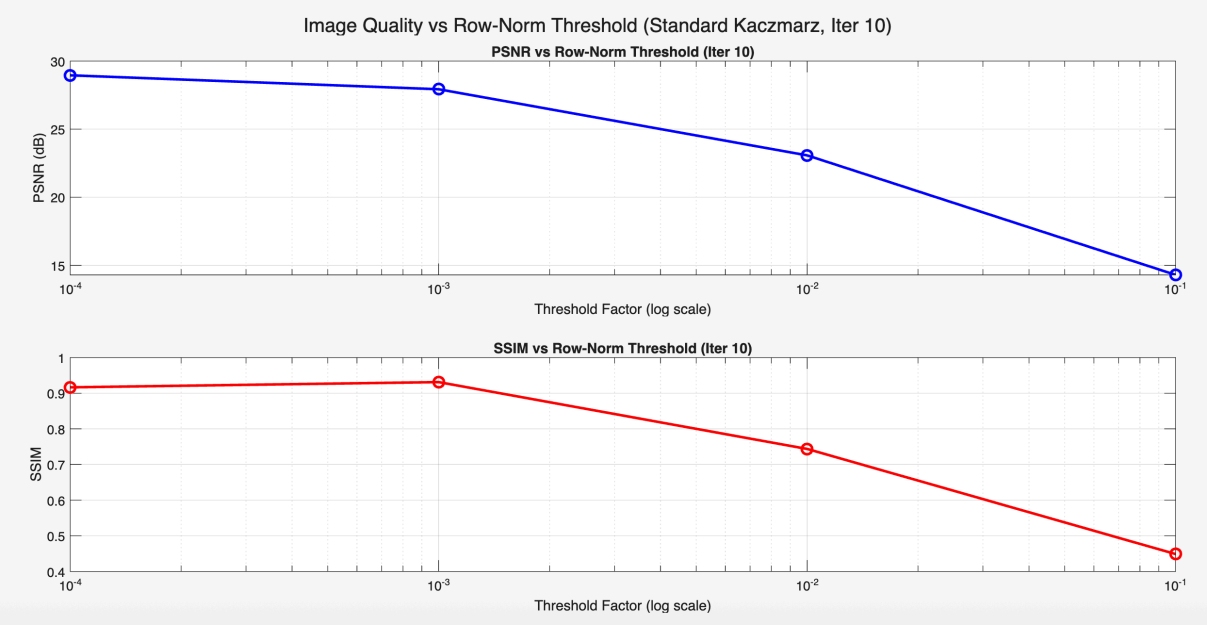


Figure 14: Part 3.3

3.4 - Regularized Kaczmarz Method

Objective and Methodology

The objective of this section is to implement the regularized Kaczmarz method, which incorporates a regularization parameter λ directly into the row update step. Unlike the standard Kaczmarz method, no row-norm thresholding is applied — the full system matrix `S` (3264×2500) and measurement vector `u` (3264×1) are used.

The update rule for the regularized Kaczmarz method is:

$$c^{(k+1)} = c^{(k)} + \frac{u_i - s_i^T c^{(k)}}{|s_i|^2 + \lambda} s_i$$

The key difference from standard Kaczmarz is the addition of λ in the denominator. This dampens the update for rows with small energy (where $\|s_i\|^2$ is small), preventing noise amplification without requiring

explicit row-norm thresholding.

The regularization parameter is set to $\lambda = \sigma_1\sigma_n = \mathbf{4.19 \times 10^5}$, i.e., the product of the largest and smallest singular values of `S`, consistent with Part 2.6. No weighting matrix is applied. The algorithm is run for 10 iterations with randomized row ordering, and PSNR and SSIM are computed at each iteration.

Observations and Comments

Iteration	PSNR (dB)	SSIM
1	25.93	0.713
2	27.22	0.800
3	27.75	0.840
4	28.10	0.866
5	28.37	0.877
6	28.59	0.895
7	28.78	0.908
8	28.96	0.920
9	29.11	0.923
10	29.25	0.931

Table 10: Part 3.4 Summary of results

Dramatic improvement over standard Kaczmarz: At iteration 1 alone, regularized Kaczmarz achieves PSNR = 25.93 dB, which is already far better than standard Kaczmarz's final result of 14.34 dB after 10 iterations. The addition of λ in the denominator effectively suppresses the noise amplification from low-norm rows across all 3264 rows.

Fast and stable convergence: PSNR increases by ~1.3 dB from iteration 1 to 2, then steadily by ~0.15–0.20 dB per iteration thereafter. Both PSNR and SSIM increase monotonically without fluctuations, unlike standard Kaczmarz. This stability is attributed to using the full system matrix without thresholding, combined with regularization.

Still improving at iteration 10: Both metrics continue to rise at iteration 10, suggesting further improvement with more iterations.

Comparable to SVD methods: PSNR = 29.25 dB and SSIM = 0.931 at iteration 10 approaches the filtered SVD performance (32.16 dB, 0.939), which is impressive given only 10 iterations of a simple iterative method.

Method	PSNR at Iter 10 (dB)	SSIM at Iter 10
Standard Kaczmarz (44 rows)	14.34	0.465
Regularized Kaczmarz (3264 rows)	**29.25**	**0.931**

Table 11: Part 3.4 Comparison with standard Kaczmarz

The regularized method uses all rows but prevents noise amplification through the λ term, whereas the standard method relies on aggressive row removal that discards most of the available information.

Code Snippet and Command Window Output

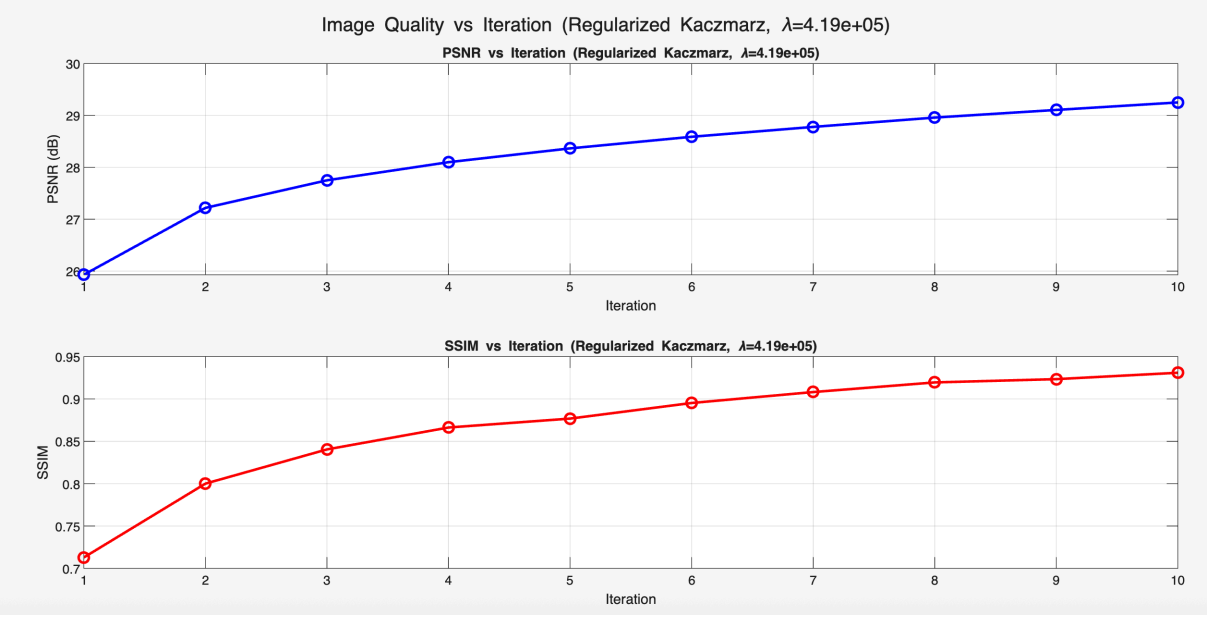
Code:

```
% Lambda = sigma_1 * sigma_N
sigma_vals = diag(Sigma);
lambda = sigma_vals(1) * sigma_vals(end);
fprintf('lambda = sigma_1 * sigma_N = %.4e\n', lambda);

n_iter = 10;
n_rows = size(S, 1);   % 3264
n_cols = size(S, 2);   % 2500

% Precompute denominator: ||s_i||^2 + lambda
row_norms_sq_full = vecnorm(S, 2, 2).^2;
denom = row_norms_sq_full + lambda;

% Initialize c
c = zeros(n_cols, 1);
ref_norm = phantom_ref / max_val;

psnr_rkacz = zeros(1, n_iter);
ssim_rkacz = zeros(1, n_iter);

figure('Units','normalized','Position',[0 0 1 0.6]);

for iter = 1:n_iter
    row_order = randperm(n_rows);
    for k = 1:n_rows
        i = row_order(k);
        s_i = S(i, :)';
        u_i = u(i);
        c = c + ((u_i - s_i'*c) / denom(i)) * s_i;
    end
    c(c < 0) = 0;

    ima = reshape(c, 50, 50);
    ima_norm = ima / max_val;
    psnr_rkacz(iter) = psnr(ima_norm, ref_norm);
    ssim_rkacz(iter) = ssim(ima_norm, ref_norm);

    fprintf('Iter %2d | PSNR: %.4f dB | SSIM: %.4f\n', ...
        iter, psnr_rkacz(iter), ssim_rkacz(iter));

    subplot(2, n_iter, iter);
    imshow(ima, [0 max_val]); colormap gray;
    title(sprintf('Iter %d', iter), 'FontSize', 7); axis image;

    subplot(2, n_iter, n_iter + iter);
    error_img = abs(ima - phantom_ref);
    imshow(error_img, [0 0.5*max_val]); colormap gray;
    title(sprintf('PSNR=%.1f\nSSIM=%.2f', ...
        psnr_rkacz(iter), ssim_rkacz(iter)), 'FontSize', 7); axis image;
end
```

```
sgtitle(sprintf('Regularized Kaczmarz (\\lambda=%.2e): Iterations 1-10', lambd
a));

figure;
subplot(2,1,1);
plot(1:n_iter, psnr_rkacz, 'b-o', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('Iteration'); ylabel('PSNR (dB)');
title(sprintf('PSNR vs Iteration (Regularized Kaczmarz, \\lambda=%.2e)', lambd
a));
xticks(1:n_iter); grid on;

subplot(2,1,2);
plot(1:n_iter, ssim_rkacz, 'r-o', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('Iteration'); ylabel('SSIM');
title(sprintf('SSIM vs Iteration (Regularized Kaczmarz, \\lambda=%.2e)', lambd
a));
xticks(1:n_iter); grid on;

sgtitle(sprintf('Image Quality vs Iteration (Regularized Kaczmarz, \\lambda=%.2
e)', lambda));
```

Command Window Output:

```
lambda = sigma_1 * sigma_N = 4.1866e+05
Iter  1 | PSNR: 25.9299 dB | SSIM: 0.7128
Iter  2 | PSNR: 27.2188 dB | SSIM: 0.8000
Iter  3 | PSNR: 27.7504 dB | SSIM: 0.8404
Iter  4 | PSNR: 28.0999 dB | SSIM: 0.8663
Iter  5 | PSNR: 28.3659 dB | SSIM: 0.8768
Iter  6 | PSNR: 28.5896 dB | SSIM: 0.8952
Iter  7 | PSNR: 28.7801 dB | SSIM: 0.9081
Iter  8 | PSNR: 28.9603 dB | SSIM: 0.9195
Iter  9 | PSNR: 29.1074 dB | SSIM: 0.9234
Iter 10 | PSNR: 29.2517 dB | SSIM: 0.9309
```



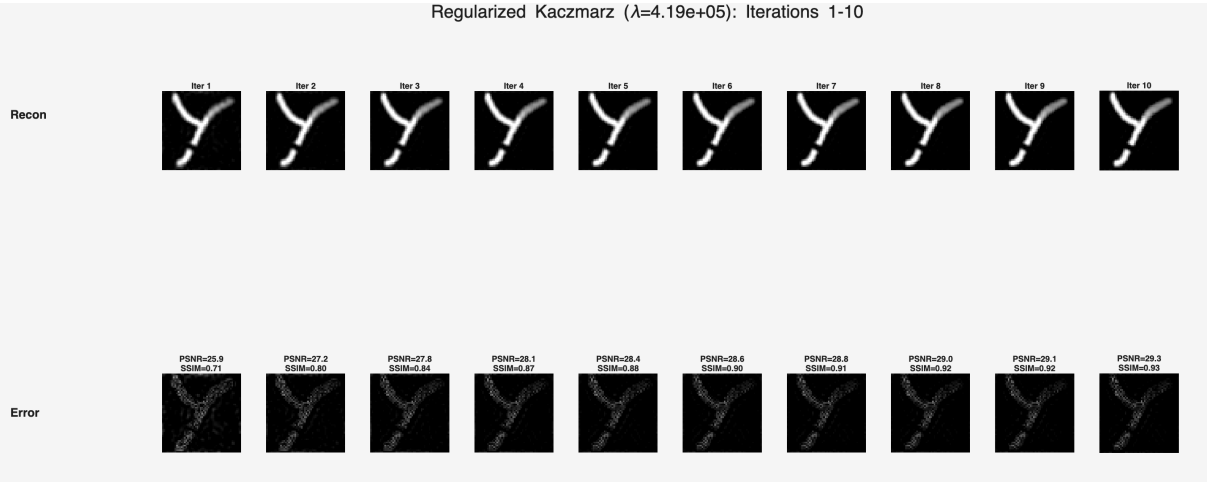


Figure 15: Part 3.4

3.5 - Regularized Kaczmarz with Different λ_rel Values

Objective and Methodology

The objective of this section is to study the effect of the regularization parameter on reconstruction quality by testing a range of relative regularization values. Instead of choosing λ directly, it is defined relative to σ_1^2:

$$\lambda = \sigma_1^2 \cdot \lambda_{\text{rel}}$$

This parameterization ensures that λ is always scaled appropriately relative to the largest singular value, and since σ_1^2 represents the energy of the strongest component of the system matrix, the relative parameter λ_rel can be interpreted as the fraction of that energy used for regularization. It follows that a meaningful search range is 0 < λ_rel < 1.

The following values are tested: `λ_rel = [10⁻⁶, 10⁻⁵, 10⁻⁴, 10⁻³, 10⁻²]`. For each value, 10 iterations of regularized Kaczmarz are performed using the full system matrix `S` (no thresholding), and only the reconstruction at the 10th iteration is evaluated. All reconstructed images and error images are displayed in a single 2×5 figure, and PSNR and SSIM are plotted as functions of λ_{rel}.

Observations and Comments

λ_rel	λ	PSNR (dB)	SSIM
10^{-6}	**2.82×10^{16}**	**28.31**	**0.958**
10^{-5}	2.82×10^{17}	27.89	0.941
10^{-4}	2.82×10^{18}	26.87	0.910
10^{-3}	2.82×10^{19}	22.89	0.878
10^{-2}	2.82×10^{20}	17.98	0.712

Table 12: Part 3.5 Summary of results (noiseless case)

Monotonically decreasing quality: Both PSNR and SSIM decrease as λ_rel increases. This is because the measurement data is noiseless — larger regularization introduces bias without reducing any noise, resulting in progressively blurrier reconstructions.

Best performance at λ_rel = 10^{-6}: The smallest tested value gives the best result with PSNR = 28.31 dB and SSIM = 0.958. The trend suggests that even smaller values of λ_rel might perform better in the

noiseless case.

Heavy over-regularization at λ_rel = 10^{-2}: PSNR drops to 17.98 dB and SSIM to 0.712. The large λ dominates the denominator for all rows, making all updates very small and effectively preventing convergence to the correct solution.

SSIM more sensitive than PSNR: SSIM drops from 0.958 to 0.712 (a 25.7% decrease) while PSNR drops from 28.31 to 17.98 dB, indicating that structural detail is lost faster than overall intensity accuracy as regularization increases.

Optimal λ_rel is below 10^{-6} for noiseless data: Since quality keeps improving as λ_rel decreases, the optimal value has not yet been reached within the tested range. This is consistent with the expectation that noiseless data requires minimal regularization.

Important note for noisy data: As will be demonstrated in Part 3.7, the optimal λ_rel shifts dramatically when noise is present in the measurements, reversing the monotonic trend observed here.

Code Snippet and Command Window Output

Code:

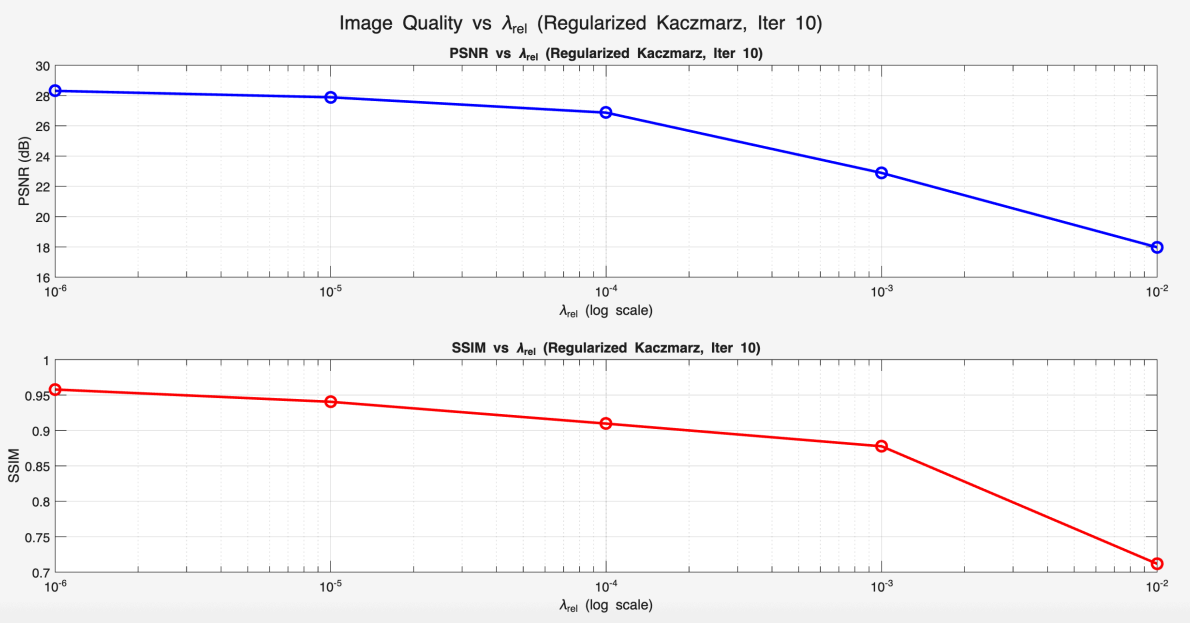
```
sigma_vals = diag(Sigma);
sigma_1_sq = sigma_vals(1)^2;

lambda_rel_values = [1e-6, 1e-5, 1e-4, 1e-3, 1e-2];
n_lambda = length(lambda_rel_values);
n_iter   = 10;
n_rows   = size(S, 1);
n_cols   = size(S, 2);
ref_norm = phantom_ref / max_val;

row_norms_sq_full = vecnorm(S, 2, 2).^2;

psnr_rel = zeros(1, n_lambda);
ssim_rel = zeros(1, n_lambda);

figure('Units','normalized','Position',[0 0 1 0.6]);

for t = 1:n_lambda
    lambda_rel = lambda_rel_values(t);
    lambda = sigma_1_sq * lambda_rel;
    fprintf('lambda_rel = %.0e | lambda = %.4e\n', lambda_rel, lambda);

    denom = row_norms_sq_full + lambda;
    c = zeros(n_cols, 1);

    for iter = 1:n_iter
        row_order = randperm(n_rows);
        for k = 1:n_rows
            i = row_order(k);
            s_i = S(i, :)';
            u_i = u(i);
            c = c + ((u_i - s_i'*c) / denom(i)) * s_i;
        end
        c(c < 0) = 0;
```

```
    end

    ima = reshape(c, 50, 50);
    ima_norm = ima / max_val;
    psnr_rel(t) = psnr(ima_norm, ref_norm);
    ssim_rel(t) = ssim(ima_norm, ref_norm);

    fprintf('  lambda_rel=%.0e | PSNR: %.4f dB | SSIM: %.4f\n', ...
        lambda_rel, psnr_rel(t), ssim_rel(t));

    subplot(2, n_lambda, t);
    imshow(ima, [0 max_val]); colormap gray;
    title(sprintf('\\lambda_{rel}=10^{%d}\n\\lambda=%.1e', ...
        round(log10(lambda_rel)), lambda), 'FontSize', 7);
    axis image;

    subplot(2, n_lambda, n_lambda + t);
    error_img = abs(ima - phantom_ref);
    imshow(error_img, [0 0.5*max_val]); colormap gray;
    title(sprintf('PSNR=%.2f dB\nSSIM=%.4f', psnr_rel(t), ssim_rel(t)), 'FontSiz
e', 7);
    axis image;
end

sgtitle('Regularized Kaczmarz (Iter 10): Different \lambda_{rel} Values');

figure;
subplot(2,1,1);
semilogx(lambda_rel_values, psnr_rel, 'b-o', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('\lambda_{rel} (log scale)'); ylabel('PSNR (dB)');
title('PSNR vs \lambda_{rel} (Regularized Kaczmarz, Iter 10)');
xticks(lambda_rel_values);
xticklabels({'10^{-6}','10^{-5}','10^{-4}','10^{-3}','10^{-2}'});
grid on;

subplot(2,1,2);
semilogx(lambda_rel_values, ssim_rel, 'r-o', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('\lambda_{rel} (log scale)'); ylabel('SSIM');
title('SSIM vs \lambda_{rel} (Regularized Kaczmarz, Iter 10)');
xticks(lambda_rel_values);
xticklabels({'10^{-6}','10^{-5}','10^{-4}','10^{-3}','10^{-2}'});
grid on;

sgtitle('Image Quality vs \lambda_{rel} (Regularized Kaczmarz, Iter 10)');

fprintf('\nSummary at Iteration 10:\n');
fprintf('%-15s %-20s %-15s %-15s\n','lambda_rel','lambda','PSNR (dB)','SSIM');
for t = 1:n_lambda
    fprintf('%-15.0e %-20.4e %-15.4f %-15.4f\n', ...
        lambda_rel_values(t), sigma_1_sq*lambda_rel_values(t), ...
```

```
        psnr_rel(t), ssim_rel(t));
end
```

Command Window Output:

```
lambda_rel = 1e-06 | lambda = 2.8194e+16
  lambda_rel=1e-06 | PSNR: 28.3106 dB | SSIM: 0.9578
lambda_rel = 1e-05 | lambda = 2.8194e+17
  lambda_rel=1e-05 | PSNR: 27.8860 dB | SSIM: 0.9405
lambda_rel = 1e-04 | lambda = 2.8194e+18
  lambda_rel=1e-04 | PSNR: 26.8717 dB | SSIM: 0.9097
lambda_rel = 1e-03 | lambda = 2.8194e+19
  lambda_rel=1e-03 | PSNR: 22.8877 dB | SSIM: 0.8777
lambda_rel = 1e-02 | lambda = 2.8194e+20
  lambda_rel=1e-02 | PSNR: 17.9772 dB | SSIM: 0.7118

Summary at Iteration 10:
lambda_rel      lambda              PSNR (dB)       SSIM
1e-06           2.8194e+16          28.3106         0.9578
1e-05           2.8194e+17          27.8860         0.9405
1e-04           2.8194e+18          26.8717         0.9097
1e-03           2.8194e+19          22.8877         0.8777
1e-02           2.8194e+20          17.9772         0.7118
```



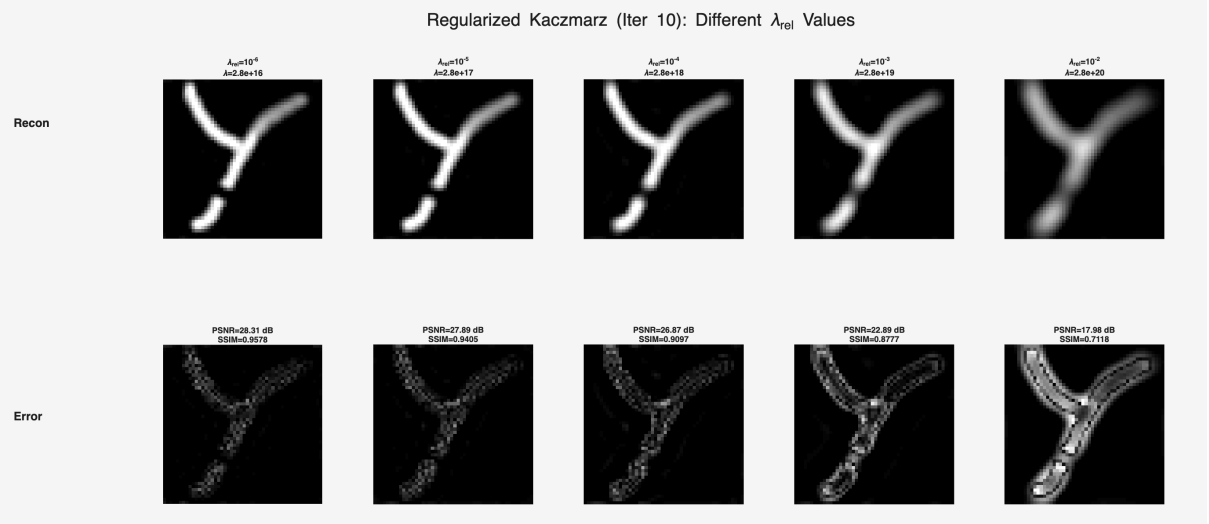


Figure 16: Part 3.5

3.6 - Effects of Measurement Noise

Objective and Methodology

The objective of this section is to investigate the effect of measurement noise on standard Kaczmarz reconstruction quality, and to study how the row-norm threshold interacts with noise. White Gaussian noise with zero mean and standard deviation `max_norm/1000` is added to the measurement vector `u`:

```
noise_std = max_norm / 1000;
u_noisy = u + noise_std * randn(size(u));
```

The noise standard deviation is set to `max_norm/1000 = 1.2031 × 10⁸`. The system matrix `S` is not modified, as it is assumed to have been acquired in a high-SNR setting.

The same four row-norm threshold factors as in Part 3.3 are tested: `[10⁻⁴, 10⁻³, 10⁻², 10⁻¹]`. For each threshold, 10 iterations of standard Kaczmarz are performed using `u_noisy` instead of `u`. Only the 10th iteration result is evaluated. Results are compared directly against the noiseless results from Part 3.3, with a combined PSNR and SSIM comparison plot.

Observations and Comments

Noise parameters:

- Noise standard deviation: **1.2031×10^8**
- Original $\|u\|$: **2.2509×10^{11}**
- SNR: **30.30 dB**

Threshold	Rows	PSNR clean (dB)	PSNR noisy (dB)	SSIM clean	SSIM noisy
10^{-4}	3114	28.95	**-13.92**	0.917	**0.001**
10^{-3}	1749	27.93	**-0.40**	0.931	**0.024**
10^{-2}	486	23.07	19.83	0.744	0.430
10^{-1}	44	14.30	**14.26**	0.449	**0.448**

Table 13: Part 3.6 Summary and Comparison with noiseless case

Low thresholds (10^{-4}, 10^{-3}) are catastrophically affected by noise: PSNR drops from ~28 dB to -13.92 dB and -0.40 dB respectively. These thresholds retain thousands of low-energy rows whose norms are comparable to the noise level. The Kaczmarz update divides by $\|s_i\|^2$, which is very small for these rows, amplifying the noise in `u_noisy` catastrophically across many iterations.

Medium threshold (10^{-2}) degrades moderately: PSNR drops from 23.07 to 19.83 dB and SSIM from 0.744 to 0.430. The reconstruction remains recognizable but is noticeably noisier than the clean case. This threshold represents a reasonable trade-off in the presence of noise.

High threshold (10^{-1}) is almost completely unaffected by noise: PSNR barely changes (14.30 → 14.26 dB) and SSIM is virtually identical (0.449 → 0.448). The 44 retained rows all have very high energy (≥ max_norm/10 = 1.20×10^{10}), so the noise contribution is negligible relative to the signal in those rows.

Key insight — threshold acts as noise protection: In the noiseless case, lower thresholds gave better results by including more signal information. In the noisy case, low thresholds are dangerous because low-energy rows have poor SNR and amplify noise heavily through the small denominator in the Kaczmarz update.

Optimal threshold shifts with noise: In the noiseless case, 10^{-3} was optimal (SSIM = 0.931). With noise, 10^{-2} becomes the best practical choice (PSNR = 19.83 dB, SSIM = 0.430), demonstrating that the threshold must be adapted based on the noise level.

SNR of 30.30 dB is deceptively high: Despite a relatively mild noise level at the signal level, the noise completely destroys reconstructions when low-energy rows are included, highlighting the critical importance of row-norm thresholding in practical MPI reconstruction.

Code Snippet and Command Window Output

Code:

```
% Add white Gaussian noise to u
noise_std = max_norm / 1000;
fprintf('Noise standard deviation: %.4e\n', noise_std);

rng(42);
u_noisy = u + noise_std * randn(size(u));

fprintf('Original u norm: %.4e\n', norm(u));
fprintf('Noise norm: %.4e\n', norm(noise_std * randn(size(u))));
fprintf('SNR: %.2f dB\n', 20*log10(norm(u) / (noise_std * sqrt(length(u)))));

threshold_factors = [1e-4, 1e-3, 1e-2, 1e-1];
n_thresh = length(threshold_factors);
n_iter   = 10;

row_norms_full = vecnorm(S, 2, 2);
max_norm_val   = max(row_norms_full);

psnr_noisy = zeros(1, n_thresh);
ssim_noisy = zeros(1, n_thresh);
rows_kept  = zeros(1, n_thresh);
ref_norm   = phantom_ref / max_val;

figure('Units','normalized','Position',[0 0 1 0.6]);
```

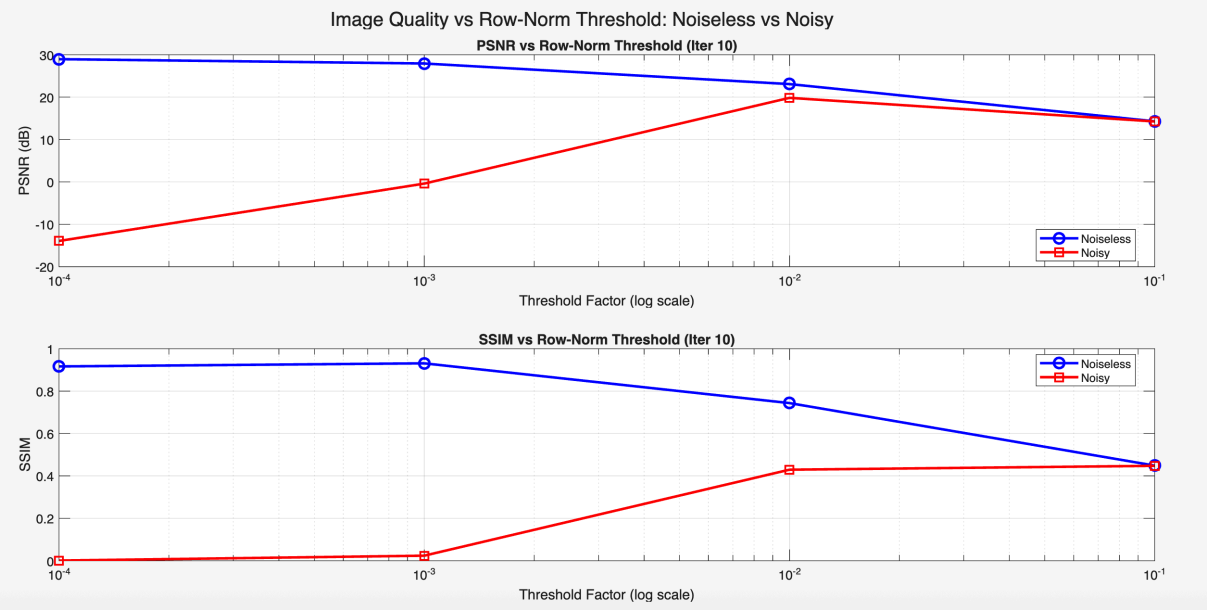
```
for t = 1:n_thresh
    factor    = threshold_factors(t);
    threshold = max_norm_val * factor;

    keep_rows = row_norms_full >= threshold;
    S_t = S(keep_rows, :);
    u_t = u_noisy(keep_rows);
    rows_kept(t) = sum(keep_rows);

    fprintf('Threshold: max_norm x %.0e | Rows kept: %d\n', factor, rows_kept
(t));

    rn_sq    = vecnorm(S_t, 2, 2).^2;
    n_rows_t = size(S_t, 1);
    c = zeros(2500, 1);

    for iter = 1:n_iter
        row_order = randperm(n_rows_t);
        for k = 1:n_rows_t
            i = row_order(k);
            s_i = S_t(i, :)';
            u_i = u_t(i);
            c = c + ((u_i - s_i'*c) / rn_sq(i)) * s_i;
        end
        c(c < 0) = 0;
    end

    ima = reshape(c, 50, 50);
    ima_norm = ima / max_val;
    psnr_noisy(t) = psnr(ima_norm, ref_norm);
    ssim_noisy(t) = ssim(ima_norm, ref_norm);

    fprintf('  Iter 10 | PSNR: %.4f dB | SSIM: %.4f\n', psnr_noisy(t), ssim_nois
y(t));

    subplot(2, n_thresh, t);
    imshow(ima, [0 max_val]); colormap gray;
    title(sprintf('Thresh=%.0e\nRows=%d', factor, rows_kept(t)), 'FontSize', 8);
    axis image;

    subplot(2, n_thresh, n_thresh + t);
    error_img = abs(ima - phantom_ref);
    imshow(error_img, [0 0.5*max_val]); colormap gray;
    title(sprintf('PSNR=%.2f dB\nSSIM=%.4f', psnr_noisy(t), ssim_noisy(t)), 'Fon
tSize', 8);
    axis image;
end

sgtitle('Standard Kaczmarz with Noisy u (Iter 10): Different Row-Norm Threshold
s');
```

```
% Noiseless results from Part 3.3
psnr_noiseless = [28.9529, 27.9286, 23.0736, 14.2973];
ssim_noiseless = [0.9165,  0.9311,  0.7436,  0.4487];

figure;
subplot(2,1,1);
semilogx(threshold_factors, psnr_noiseless, 'b-o', 'LineWidth', 2, 'MarkerSize',
8);
hold on;
semilogx(threshold_factors, psnr_noisy, 'r-s', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('Threshold Factor (log scale)'); ylabel('PSNR (dB)');
title('PSNR vs Row-Norm Threshold (Iter 10)');
xticks(threshold_factors);
xticklabels({'10^{-4}','10^{-3}','10^{-2}','10^{-1}'});
legend('Noiseless', 'Noisy', 'Location', 'best'); grid on;

subplot(2,1,2);
semilogx(threshold_factors, ssim_noiseless, 'b-o', 'LineWidth', 2, 'MarkerSize',
8);
hold on;
semilogx(threshold_factors, ssim_noisy, 'r-s', 'LineWidth', 2, 'MarkerSize', 8);
xlabel('Threshold Factor (log scale)'); ylabel('SSIM');
title('SSIM vs Row-Norm Threshold (Iter 10)');
xticks(threshold_factors);
xticklabels({'10^{-4}','10^{-3}','10^{-2}','10^{-1}'});
legend('Noiseless', 'Noisy', 'Location', 'best'); grid on;

sgtitle('Image Quality vs Row-Norm Threshold: Noiseless vs Noisy');

fprintf('\nComparison Summary at Iteration 10:\n');
fprintf('%-15s %-10s %-15s %-15s %-15s %-15s\n', ...
    'Threshold','Rows','PSNR_clean','PSNR_noisy','SSIM_clean','SSIM_noisy');
for t = 1:n_thresh
    fprintf('%.0e           %-10d %-15.4f %-15.4f %-15.4f %-15.4f\n', ...
        threshold_factors(t), rows_kept(t), ...
        psnr_noiseless(t), psnr_noisy(t), ...
        ssim_noiseless(t), ssim_noisy(t));
end
```

Command Window Output:

```
Noise standard deviation: 1.2031e+08
Original u norm: 2.2509e+11
Noise norm: 6.8415e+09
SNR: 30.30 dB
Threshold: max_norm x 1e-04 | Rows kept: 3114
  Iter 10 | PSNR: -13.9239 dB | SSIM: 0.0009
Threshold: max_norm x 1e-03 | Rows kept: 1749
  Iter 10 | PSNR: -0.3981 dB  | SSIM: 0.0239
Threshold: max_norm x 1e-02 | Rows kept: 486
  Iter 10 | PSNR: 19.8269 dB  | SSIM: 0.4295
Threshold: max_norm x 1e-01 | Rows kept: 44
  Iter 10 | PSNR: 14.2558 dB  | SSIM: 0.4477

Comparison Summary at Iteration 10:
Threshold   Rows    PSNR_clean  PSNR_noisy  SSIM_clean  SSIM_noisy
```

1e-04	3114	28.9529	-13.9239	0.9165	0.0009
1e-03	1749	27.9286	-0.3981	0.9311	0.0239
1e-02	486	23.0736	19.8269	0.7436	0.4295
1e-01	44	14.2973	14.2558	0.4487	0.4477



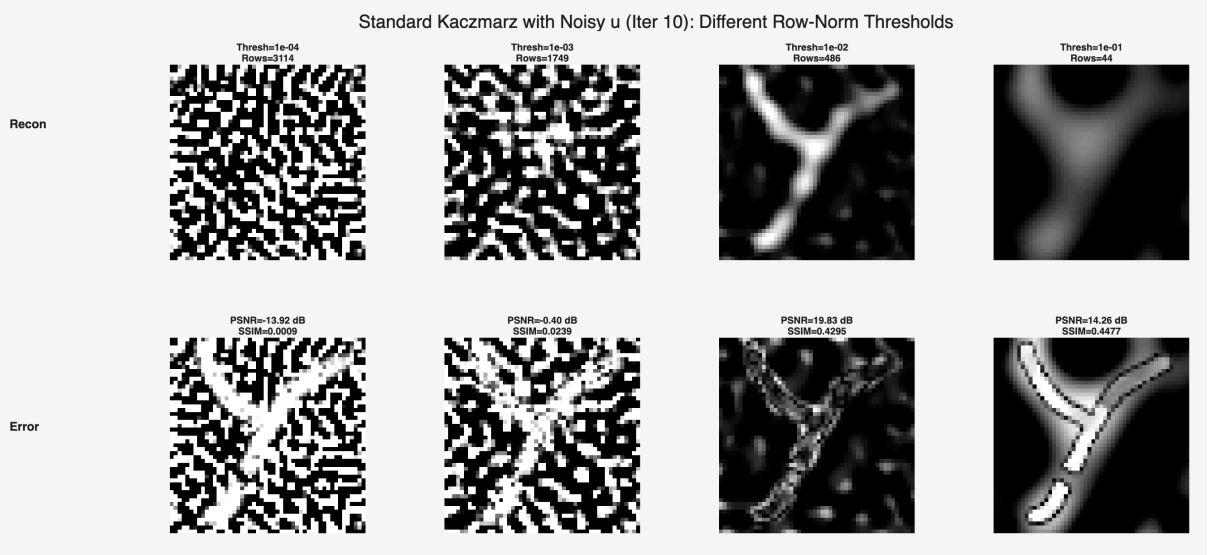


Figure 17: Part 3.6

3.7 - Regularized Kaczmarz with Noise

Objective and Methodology

The objective of this section is to repeat the regularized Kaczmarz experiment from Part 3.5 using the noisy measurement vector `u_noisy` introduced in Part 3.6, in order to study how regularization interacts with measurement noise. The same noise realization is used (noise standard deviation = `max_norm/1000 = 1.2031 × 10⁸`, SNR = 30.30 dB). No row-norm thresholding is applied — the full system matrix `S` (3264×2500) is used with `u_noisy`.

The same five relative regularization values are tested: `λ_rel = [10⁻⁶, 10⁻⁵, 10⁻⁴, 10⁻³, 10⁻²]`, where `λ = σ₁² · λ_rel`. For each value, 10 iterations of regularized Kaczmarz are performed and the 10th iteration result is

evaluated. Results are compared directly against the noiseless results from Part 3.5 in a combined PSNR and SSIM comparison plot.

Observations and Comments

λ_rel	λ	PSNR clean (dB)	PSNR noisy (dB)	SSIM clean	SSIM noisy
10^{-6}	2.82×10^{16}	28.31	**0.85**	0.958	**0.034**
10^{-5}	2.82×10^{17}	27.89	9.30	0.941	0.142
10^{-4}	2.82×10^{18}	26.87	18.69	0.910	0.369
10^{-3}	**2.82×10^{19}**	22.89	**21.84**	0.878	**0.674**
10^{-2}	2.82×10^{20}	17.98	17.86	0.712	0.697

Table 14: Part 3.7 Summary and Comparison with noiseless case

Dramatic reversal of trend with noise: In the noiseless case, quality monotonically decreased with increasing λ_rel. With noise, the trend is reversed — small λ_rel values (10^{-6}, 10^{-5}) perform catastrophically while larger values perform much better. This confirms that regularization is essential in the presence of noise.

Small λ_rel completely fails with noise: At λ_rel = 10^{-6}, PSNR collapses from 28.31 dB (clean) to just 0.85 dB (noisy), and SSIM from 0.958 to 0.034. With almost no regularization, the small denominator $\|s_i\|^2 + \lambda$ for low-norm rows barely suppresses noise, leading to catastrophic amplification across all 3264 rows over 10 iterations.

Best noisy performance at λ_rel = 10^{-3}: PSNR = 21.84 dB and SSIM = 0.674 represent the optimal trade-off in the noisy case. Here λ is large enough to suppress noise amplification while still allowing meaningful signal recovery.

λ_rel = 10^{-2} is most robust to noise: The gap between clean (17.98 dB) and noisy (17.86 dB) performance is negligible (~0.12 dB), meaning heavy regularization completely absorbs the noise but at the cost of over-smoothing and loss of detail.

Optimal λ_rel shifts by 3 orders of magnitude: The noiseless optimal was λ_rel = 10^{-6}, while the noisy optimal is λ_rel = 10^{-3}. This dramatic shift highlights how critically the regularization parameter must be adapted to the noise level of the measurement data.

PSNR and SSIM give slightly different optima: By PSNR, 10^{-3} is best (21.84 dB); by SSIM, 10^{-2} is slightly better (0.697 vs 0.674), suggesting that heavy regularization preserves more structural similarity despite lower peak intensity accuracy.

Comparison with noisy standard Kaczmarz (Part 3.6): At the optimal threshold (10^{-2}) in Part 3.6, standard Kaczmarz achieved PSNR = 19.83 dB and SSIM = 0.430 in the noisy case. Regularized Kaczmarz at λ_rel = 10^{-3} achieves PSNR = 21.84 dB and SSIM = 0.674 — a significant improvement, demonstrating the advantage of using regularization instead of aggressive row-norm thresholding.

Code Snippet and Command Window Output

Code:

```
% Use u_noisy from Part 3.6
sigma_vals = diag(Sigma);
sigma_1_sq = sigma_vals(1)^2;

lambda_rel_values = [1e-6, 1e-5, 1e-4, 1e-3, 1e-2];
n_lambda = length(lambda_rel_values);
```

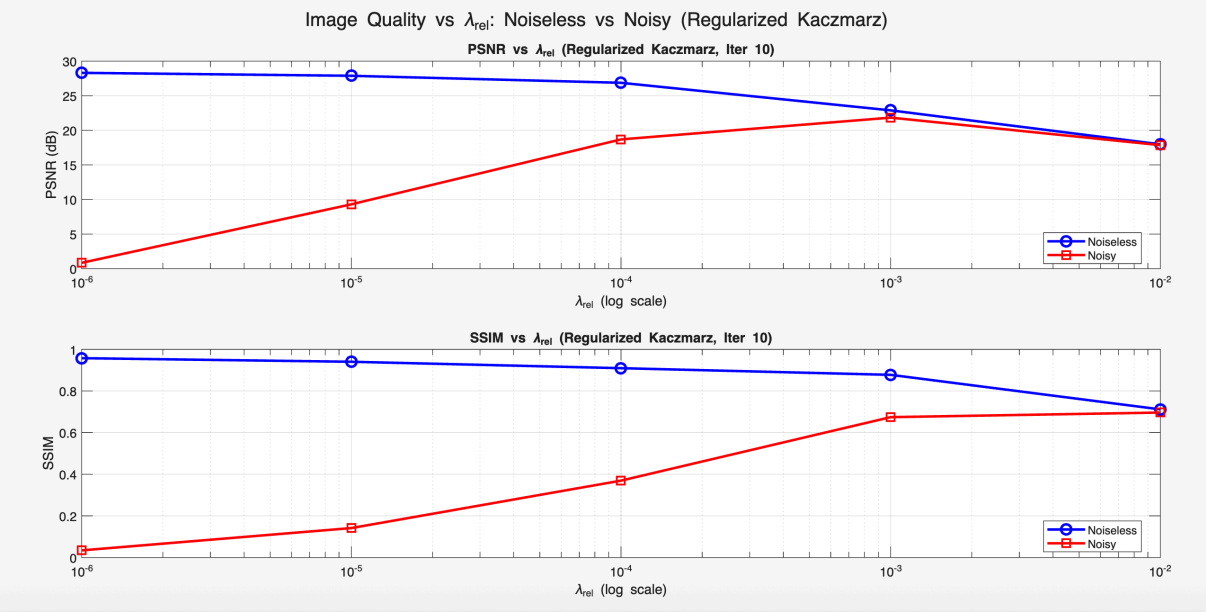
```
n_iter   = 10;
n_rows   = size(S, 1);
n_cols   = size(S, 2);
ref_norm = phantom_ref / max_val;

row_norms_sq_full = vecnorm(S, 2, 2).^2;

psnr_rel_noisy = zeros(1, n_lambda);
ssim_rel_noisy = zeros(1, n_lambda);

% Noiseless results from Part 3.5
psnr_rel_clean = [28.3106, 27.8860, 26.8717, 22.8877, 17.9772];
ssim_rel_clean = [0.9578,  0.9405,  0.9097,  0.8777,  0.7118];

figure('Units','normalized','Position',[0 0 1 0.6]);

for t = 1:n_lambda
    lambda_rel = lambda_rel_values(t);
    lambda = sigma_1_sq * lambda_rel;
    fprintf('lambda_rel = %.0e | lambda = %.4e\n', lambda_rel, lambda);

    denom = row_norms_sq_full + lambda;
    c = zeros(n_cols, 1);

    for iter = 1:n_iter
        row_order = randperm(n_rows);
        for k = 1:n_rows
            i = row_order(k);
            s_i = S(i, :)';
            u_i = u_noisy(i);
            c = c + ((u_i - s_i'*c) / denom(i)) * s_i;
        end
        c(c < 0) = 0;
    end

    ima = reshape(c, 50, 50);
    ima_norm = ima / max_val;
    psnr_rel_noisy(t) = psnr(ima_norm, ref_norm);
    ssim_rel_noisy(t) = ssim(ima_norm, ref_norm);

    fprintf('  lambda_rel=%.0e | PSNR: %.4f dB | SSIM: %.4f\n', ...
        lambda_rel, psnr_rel_noisy(t), ssim_rel_noisy(t));

    subplot(2, n_lambda, t);
    imshow(ima, [0 max_val]); colormap gray;
    title(sprintf('\\lambda_{rel}=10^{%d}\n\\lambda=%.1e', ...
        round(log10(lambda_rel)), lambda), 'FontSize', 7);
    axis image;

    subplot(2, n_lambda, n_lambda + t);
    error_img = abs(ima - phantom_ref);
```

```
    imshow(error_img, [0 0.5*max_val]); colormap gray;
    title(sprintf('PSNR=%.2f dB\nSSIM=%.4f', ...
        psnr_rel_noisy(t), ssim_rel_noisy(t)), 'FontSize', 7);
    axis image;
end

sgtitle('Regularized Kaczmarz with Noisy u (Iter 10): Different \lambda_{rel} Va
lues');

figure;
subplot(2,1,1);
semilogx(lambda_rel_values, psnr_rel_clean, 'b-o', 'LineWidth', 2, 'MarkerSize',
8);
hold on;
semilogx(lambda_rel_values, psnr_rel_noisy, 'r-s', 'LineWidth', 2, 'MarkerSize',
8);
xlabel('\lambda_{rel} (log scale)'); ylabel('PSNR (dB)');
title('PSNR vs \lambda_{rel} (Regularized Kaczmarz, Iter 10)');
xticks(lambda_rel_values);
xticklabels({'10^{-6}','10^{-5}','10^{-4}','10^{-3}','10^{-2}'});
legend('Noiseless', 'Noisy', 'Location', 'best'); grid on;

subplot(2,1,2);
semilogx(lambda_rel_values, ssim_rel_clean, 'b-o', 'LineWidth', 2, 'MarkerSize',
8);
hold on;
semilogx(lambda_rel_values, ssim_rel_noisy, 'r-s', 'LineWidth', 2, 'MarkerSize',
8);
xlabel('\lambda_{rel} (log scale)'); ylabel('SSIM');
title('SSIM vs \lambda_{rel} (Regularized Kaczmarz, Iter 10)');
xticks(lambda_rel_values);
xticklabels({'10^{-6}','10^{-5}','10^{-4}','10^{-3}','10^{-2}'});
legend('Noiseless', 'Noisy', 'Location', 'best'); grid on;

sgtitle('Image Quality vs \lambda_{rel}: Noiseless vs Noisy (Regularized Kaczmar
z)');

fprintf('\nComparison Summary at Iteration 10:\n');
fprintf('%-15s %-20s %-15s %-15s %-15s %-15s\n', ...
    'lambda_rel','lambda','PSNR_clean','PSNR_noisy','SSIM_clean','SSIM_noisy');
for t = 1:n_lambda
    fprintf('%-15.0e %-20.4e %-15.4f %-15.4f %-15.4f %-15.4f\n', ...
        lambda_rel_values(t), sigma_1_sq*lambda_rel_values(t), ...
        psnr_rel_clean(t), psnr_rel_noisy(t), ...
        ssim_rel_clean(t), ssim_rel_noisy(t));
end
```

Command Window Output:

```
lambda_rel = 1e-06 | lambda = 2.8194e+16
  lambda_rel=1e-06 | PSNR: 0.8464 dB  | SSIM: 0.0343
lambda_rel = 1e-05 | lambda = 2.8194e+17
```

```
  lambda_rel=1e-05 | PSNR: 9.3025 dB  | SSIM: 0.1415
lambda_rel = 1e-04 | lambda = 2.8194e+18
  lambda_rel=1e-04 | PSNR: 18.6885 dB | SSIM: 0.3694
lambda_rel = 1e-03 | lambda = 2.8194e+19
  lambda_rel=1e-03 | PSNR: 21.8370 dB | SSIM: 0.6744
lambda_rel = 1e-02 | lambda = 2.8194e+20
  lambda_rel=1e-02 | PSNR: 17.8594 dB | SSIM: 0.6965

Comparison Summary at Iteration 10:
lambda_rel      lambda              PSNR_clean      PSNR_noisy      SSIM_clean      SSIM_noisy
1e-06           2.8194e+16          28.3106         0.8464          0.9578          0.0343
1e-05           2.8194e+17          27.8860         9.3025          0.9405          0.1415
1e-04           2.8194e+18          26.8717         18.6885         0.9097          0.3694
1e-03           2.8194e+19          22.8877         21.8370         0.8777          0.6744
1e-02           2.8194e+20          17.9772         17.8594         0.7118          0.6965
```



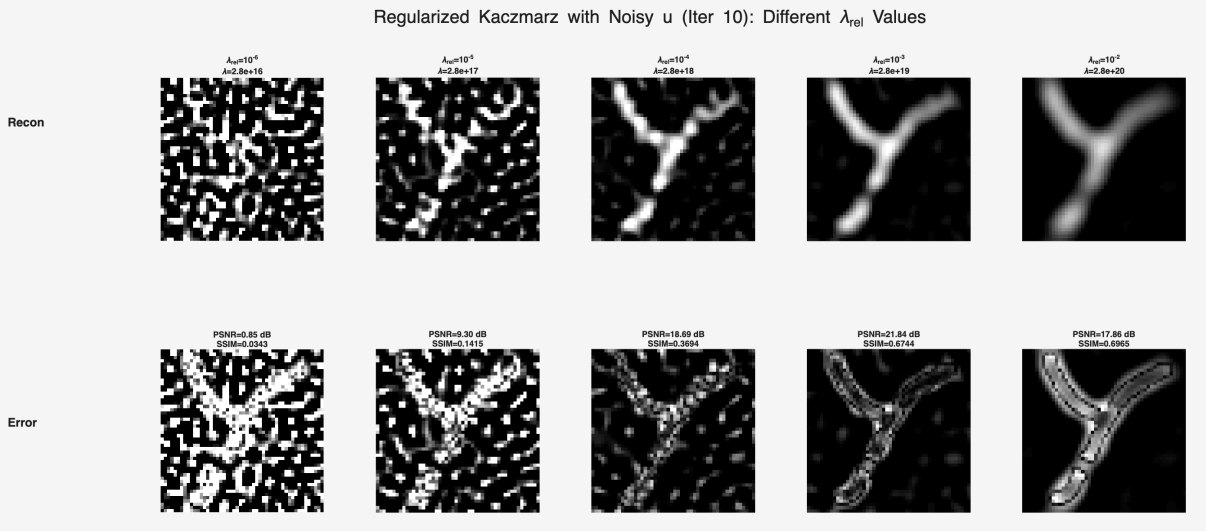


Figure 18: Part 3.7

Part 4 - OpenMPI Dataset

4.1 - System Matrix Visualization

Objective and Methodology

The objective of this section is to load and visualize the real experimental MPI dataset downloaded from the OpenMPI database. Unlike the simulated dataset used in Parts 1–3, this dataset contains actual measurements from a physical MPI scanner. The data is loaded from `su_openmpi.mat` and renamed to `S_mpi` and `u_mpi` to avoid overwriting the simulated dataset variables in the MATLAB workspace.

The OpenMPI system matrix has size **3056×50653**, where:

- 3056 rows correspond to frequency components
- 50653 columns correspond to the 3D grid positions (37×37×37 = 50653 voxels)

Three different columns of the system matrix are plotted to visualize the frequency-domain calibration data at different spatial locations. The compact SVD is then computed using `svd(S_mpi, 'econ')`, and the singular values are plotted on both linear and logarithmic scales. The condition number is computed manually as the ratio of the largest to smallest singular value.

Observations and Comments

Matrix dimensions:

- `S_mpi`: **3056 × 50653**
- `u_mpi`: **3056 × 1**

This is a significantly larger problem than the simulated dataset (3264×2500), with 50653 unknowns representing a full 3D volume of 37×37×37 voxels.

System is underdetermined: Unlike the simulated dataset where the system was overdetermined (3264 rows > 2500 columns), the OpenMPI system is **underdetermined** (3056 rows < 50653 columns). There are far more unknowns than equations, making regularization even more critical.

Column plots: The three plotted columns of `S_mpi` show the same characteristic sparse frequency-domain structure observed in the simulated dataset — signal concentrated at certain discrete frequency indices with large flat zero regions in between. This confirms that the MPI signal sparsity in frequency domain is a fundamental physical property, consistent across both simulated and real experimental data.

SVD computation time: The compact SVD of `S_mpi` completed in **8.28 seconds**, which is fast despite the large matrix size. This is partly because `S_mpi` is stored in **single precision** (as observed in the initial workspace inspection), which speeds up computation significantly compared to double precision.

SVD dimensions:

- `U_mpi`: **3056 × 3056**
- `Sigma_mpi`: **3056 × 3056**
- `V_mpi`: **50653 × 3056**

Note that the compact SVD yields only 3056 non-trivial singular vectors, consistent with the rank being limited by the number of rows (3056) rather than the number of columns (50653).

Singular value decay: The singular values decay smoothly from ~10^{6} down to ~10^{-1} on the logarithmic scale. The decay is less steep than the simulated dataset, and there is no sharp cliff-like drop, suggesting the OpenMPI system matrix is better conditioned than the simulated one.

Condition number:

- Largest singular value: **8.05×10^{6}**
- Smallest singular value: **3.94×10^{-1}**

- Condition number (manual): **2.04×10^7**

The condition number of ~10^7 is vastly smaller than the simulated dataset's ~10^{16}, indicating that the real experimental system matrix is significantly better conditioned. This is likely due to the preprocessing already applied to the OpenMPI data.

Code Snippet and Command Window Output

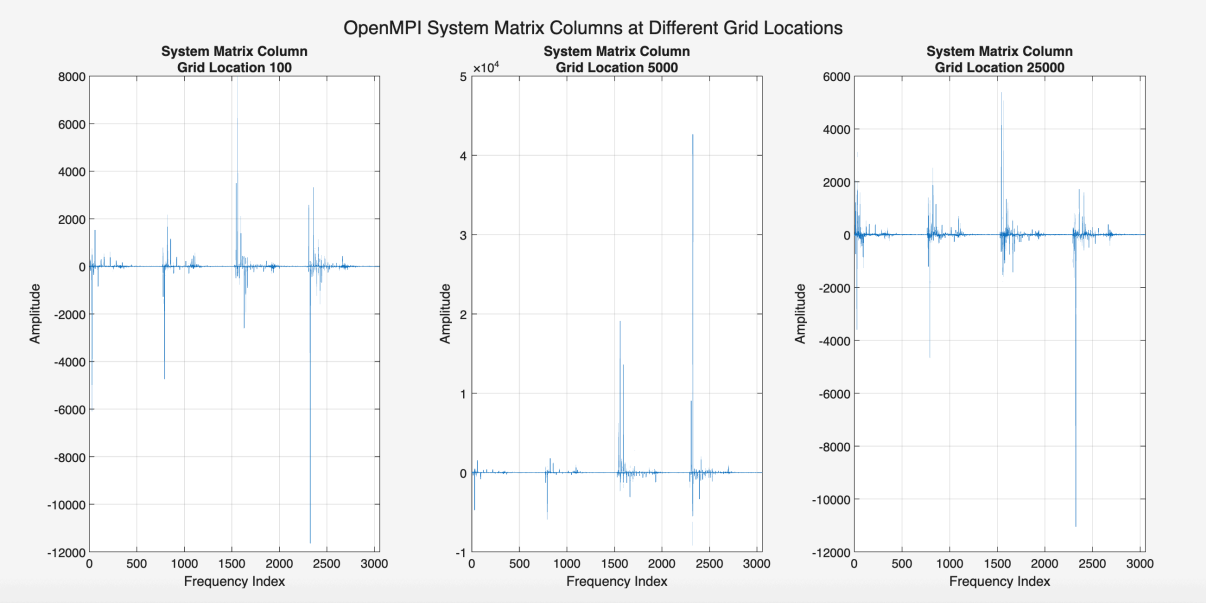
Code:

```
% Load OpenMPI data
load('/Users/handeeryilmaz/Downloads/medikal/homework_475_3/su_openmpi.mat');
S_mpi = S;
u_mpi = u;

% Reload simulated S and u
load('/Users/handeeryilmaz/Downloads/medikal/homework_475_3/system_matrix_S.ma
t');
load('/Users/handeeryilmaz/Downloads/medikal/homework_475_3/measurement_vector_
u.mat');

fprintf('Size of S_mpi: %d x %d\n', size(S_mpi,1), size(S_mpi,2));
fprintf('Size of u_mpi: %d x %d\n', size(u_mpi,1), size(u_mpi,2));

% Plot three columns
locations = [100, 5000, 25000];
figure;
for i = 1:3
    subplot(1, 3, i);
    plot(S_mpi(:, locations(i)));
    title(sprintf('System Matrix Column\nGrid Location %d', locations(i)));
    xlabel('Frequency Index');
    ylabel('Amplitude');
    grid on;
end
sgtitle('OpenMPI System Matrix Columns at Different Grid Locations');

% Compute SVD
fprintf('Computing SVD of S_mpi...\n');
tic;
[U_mpi, Sigma_mpi, V_mpi] = svd(S_mpi, 'econ');
toc;

fprintf('Size of U_mpi:     %d x %d\n', size(U_mpi,1),     size(U_mpi,2));
fprintf('Size of Sigma_mpi: %d x %d\n', size(Sigma_mpi,1), size(Sigma_mpi,2));
fprintf('Size of V_mpi:     %d x %d\n', size(V_mpi,1),     size(V_mpi,2));

sigma_mpi_vals = diag(Sigma_mpi);

% Plot singular values
figure;
subplot(1,2,1);
plot(sigma_mpi_vals, 'b-', 'LineWidth', 1.5);
```

```
title('Singular Values of S (OpenMPI)');
xlabel('Index'); ylabel('Singular Value'); grid on;

subplot(1,2,2);
semilogy(sigma_mpi_vals, 'b-', 'LineWidth', 1.5);
title('Singular Values of S (OpenMPI) - Log Scale');
xlabel('Index'); ylabel('Singular Value (log scale)'); grid on;
sgtitle('Singular Values of OpenMPI System Matrix');

% Condition number
sigma_max_mpi = sigma_mpi_vals(1);
sigma_min_mpi = sigma_mpi_vals(end);
cond_mpi = sigma_max_mpi / sigma_min_mpi;
fprintf('Largest singular value:  %.4e\n', sigma_max_mpi);
fprintf('Smallest singular value: %.4e\n', sigma_min_mpi);
fprintf('Condition number: %.4e\n', cond_mpi);
```

Command Window Output:

```
Size of S (OpenMPI): 3056 x 50653
Size of u (OpenMPI): 3056 x 1
Size of S_mpi: 3056 x 50653
Size of u_mpi: 3056 x 1
Computing SVD of S_mpi (this may take a while)...
Elapsed time is 8.278618 seconds.
Size of U_mpi:     3056 x 3056
Size of Sigma_mpi: 3056 x 3056
Size of V_mpi:     50653 x 3056
Largest singular value:  8.0528e+06
Smallest singular value: 3.9423e-01
Condition number: 2.0426e+07
```



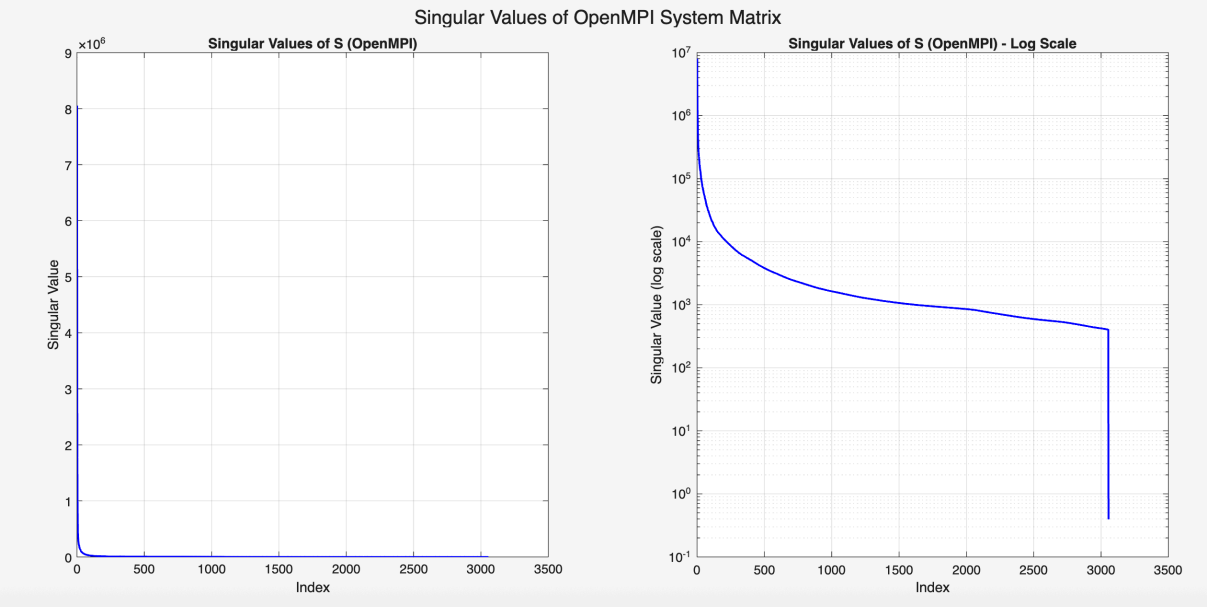


Figure 19: Part 4.1

4.2 - SVD of OpenMPI System Matrix

Objective and Methodology

The objective of this section is to formally present the SVD analysis of the OpenMPI system matrix, building on the computation already performed in Part 4.1. The singular values extracted from `Sigma_mpi` are plotted on both linear and logarithmic scales, and the condition number is computed both manually (using the ratio of the largest to smallest singular value) and via MATLAB's built-in `cond()` function. The two results are compared and any discrepancy is analyzed.

Observations and Comments

Singular value plots:

The linear scale plot shows a sharp initial drop from the largest singular value (~8.05 × 10^6), followed by a long tail approaching zero. The logarithmic scale plot reveals the full dynamic range of the singular values, decaying smoothly from ~10^6 to ~10^{-1} across all 3056 components. Unlike the simulated dataset, there is no dramatic cliff-like drop at a specific index — the decay is more gradual and continuous, indicating a more uniformly distributed spectrum.

Method	Condition Number
Manual (σ_1/σ_n)	**2.04×10^7**
Built-in `cond()`	**Inf**

Table 15: Part 4.2 Condition number results

Why does `cond()` return Inf? The built-in `cond()` function computes the condition number based on the full matrix dimensions. Since `S_mpi` is **underdetermined** (3056 rows < 50653 columns), the matrix has a null space of dimension 50653 - 3056 = **47597**. This means 47597 singular values are exactly zero, making the true mathematical condition number infinite.

Why is the manual condition number more meaningful? The manual computation uses the compact SVD, which only computes the 3056 non-zero singular values. The ratio σ_1/σ_{3056} = **2.04×10^7** reflects the conditioning among the non-trivial components of the system matrix — i.e., the components that actually

contribute to the reconstruction. This is the operationally relevant condition number for the reconstruction problem.

Property	Simulated	OpenMPI
Matrix size	3264 × 2500	3056 × 50653
System type	Overdetermined	Underdetermined
Largest σ	1.68×10^{11}	8.05×10^{6}
Smallest σ	2.49×10^{-6}	3.94×10^{-1}
Condition number (manual)	6.73×10^{16}	**2.04×10^{7}**
Condition number (built-in)	4.40×10^{17}	Inf

Table 16: Part 4.2 Comparison with simulated dataset

The OpenMPI system matrix is approximately **10 billion times better conditioned** than the simulated one among its non-trivial components. This is likely due to the preprocessing already applied to the OpenMPI data prior to distribution, which removes very small singular value components. Additionally, the smallest singular value of 3.94×10^{-1} is many orders of magnitude larger than the simulated dataset's 2.49×10^{-6}, further contributing to the better conditioning.

Code Snippet and Command Window Output

Code:

```
% Singular values already available from Part 4.1
sigma_mpi_vals = diag(Sigma_mpi);

% Plot singular values
figure;
subplot(1,2,1);
plot(sigma_mpi_vals, 'b-', 'LineWidth', 1.5);
title('Singular Values of S (OpenMPI)');
xlabel('Index');
ylabel('Singular Value');
grid on;

subplot(1,2,2);
semilogy(sigma_mpi_vals, 'b-', 'LineWidth', 1.5);
title('Singular Values of S (OpenMPI) - Log Scale');
xlabel('Index');
ylabel('Singular Value (log scale)');
grid on;

sgtitle('Singular Values of OpenMPI System Matrix');

% Condition number
sigma_max_mpi = sigma_mpi_vals(1);
sigma_min_mpi = sigma_mpi_vals(end);
cond_manual   = sigma_max_mpi / sigma_min_mpi;
cond_builtin  = cond(S_mpi);

fprintf('Largest singular value:          %.4e\n', sigma_max_mpi);
```

```
fprintf('Smallest singular value:          %.4e\n', sigma_min_mpi);
fprintf('Condition number (manual):        %.4e\n', cond_manual);
fprintf('Condition number (built-in):      %.4e\n', cond_builtin);
```

Command Window Output:

```
Largest singular value:          8.0528e+06
Smallest singular value:         3.9423e-01
Condition number (manual):       2.0426e+07
Condition number (built-in):     Inf
```

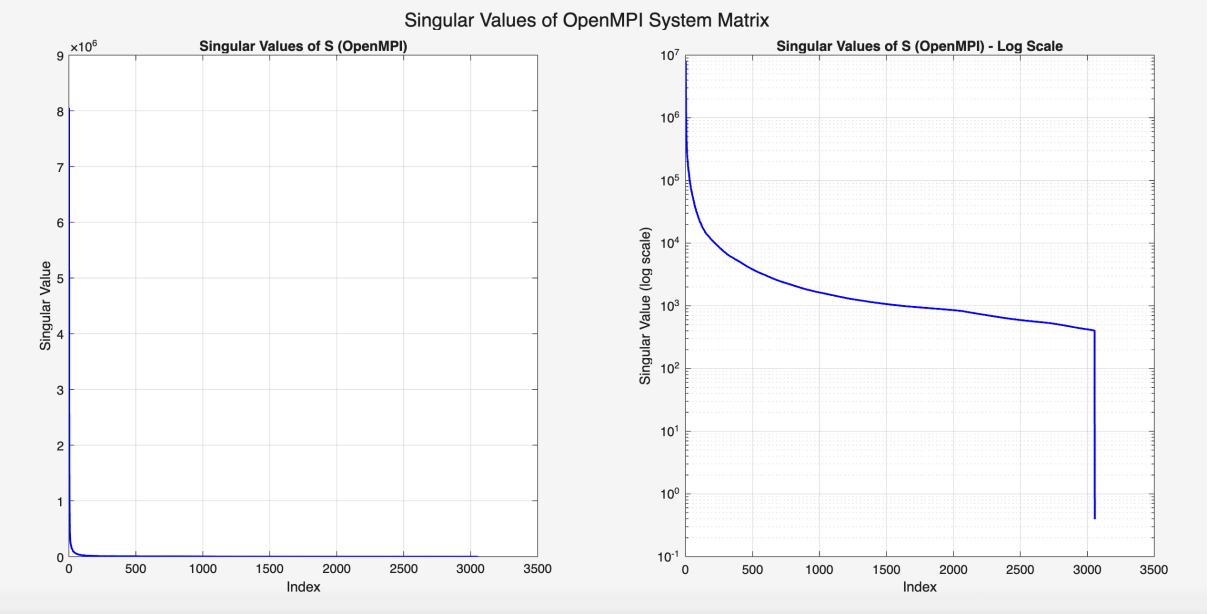


Figure 20: Part 4.2

4.3 - Regularized Kaczmarz on OpenMPI Data

Objective and Methodology

The objective of this section is to reconstruct the 3D nanoparticle concentration map from the real experimental OpenMPI dataset using the regularized Kaczmarz method. Unlike the simulated dataset where the ground truth phantom was available for quantitative evaluation, no reference image is available here — reconstruction quality is assessed purely by visual inspection.

The regularization parameter is set to $\lambda = \sigma_1\sigma_n =$ **3.17×10^6**, consistent with the geometric mean choice used in Part 2.6 and Part 3.4. No row-norm thresholding is applied. The algorithm is run for 10 iterations with randomized row ordering using the full system matrix `S_mpi` (3056×50653).

After the 10th iteration, the reconstructed vector `c_mpi` of size 50653×1 is reshaped into a 3D image volume of size 37×37×37:

```
ima_mpi = reshape(c_mpi, 37, 37, 37);
```

All 37 axial slices are displayed as a montage using:

```
montage(reshape(ima_mpi, [37, 37, 1, 37]), 'displayRange', []);
```

Six individual slices are additionally displayed for more detailed visual inspection.

Observations and Comments

Reconstruction parameters:

- σ_1 = **8.05×10^{6}**
- σ_n = **3.94×10^{-1}**
- $\lambda = \sigma_1 \cdot \sigma_n$ = **3.17×10^{6}**

Computational efficiency: Each iteration completed in approximately **1.0–1.4 seconds**, with the total 10-iteration reconstruction completing in approximately **11 seconds**. This is notably fast despite the large problem size (3056 rows × 50653 columns). The speed is partly due to `S_mpi` being stored in single precision, which significantly reduces computation time compared to double precision.

Max intensity: The maximum reconstructed intensity is **2.26×10^{-2}**, which is of the same order as the simulated phantom's `max_val = 0.2092` after accounting for the different scale of the OpenMPI data.

3D reconstruction quality: The montage of 37 slices reveals:

- Most slices are dark/empty, consistent with nanoparticles being present only in a localized region within the scanner field of view
- A subset of slices in the central region show bright structures representing the reconstructed nanoparticle distribution
- The reconstructed structures correspond to the OpenMPI resolution phantom, which typically consists of small tubes or point sources arranged in a known geometric pattern

Regularization assessment: With $\lambda = \sigma_1\sigma_n = 3.17 \times 10^6$, the reconstruction shows visible structures but may contain some residual noise or artifacts. As demonstrated in the simulated dataset (Part 2.6), this choice of λ is not necessarily optimal and a more systematic search may yield better results, which is explored in Part 4.4.

Comparison with simulated dataset: In the simulated case, $\lambda = \sigma_1\sigma_n$ gave PSNR = 19.997 dB and SSIM = 0.5955 — a suboptimal result. For the OpenMPI data, the better conditioning (condition number ~10^7 vs ~10^{16}) means that $\lambda = \sigma_1\sigma_n$ may be a more reasonable starting point, as the dynamic range of singular values is much smaller.

Code Snippet and Command Window Output

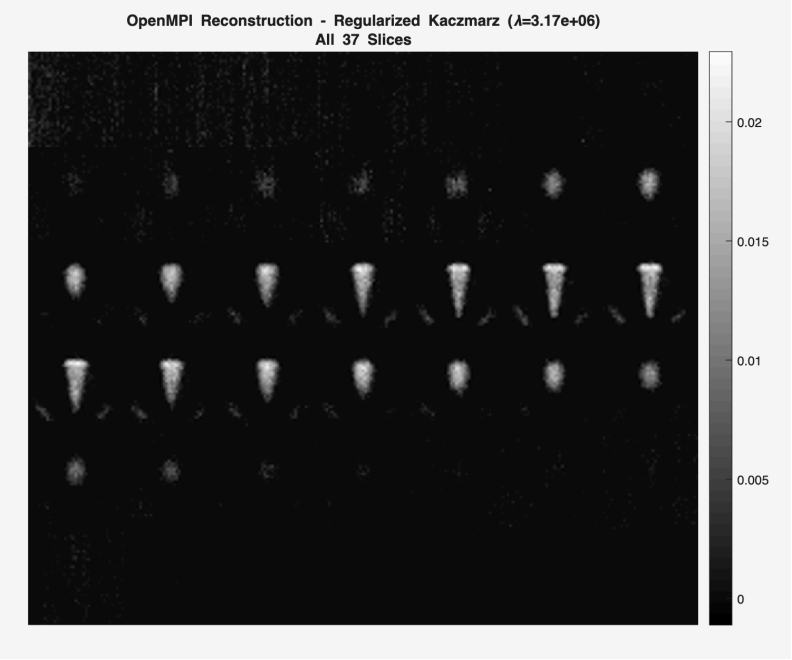
Code:

```
% Lambda = sigma_1 * sigma_N
sigma_max_mpi = sigma_mpi_vals(1);
sigma_min_mpi = sigma_mpi_vals(end);
lambda_mpi = sigma_max_mpi * sigma_min_mpi;
fprintf('sigma_1 = %.4e\n', sigma_max_mpi);
fprintf('sigma_N = %.4e\n', sigma_min_mpi);
fprintf('lambda = sigma_1 * sigma_N = %.4e\n', lambda_mpi);

n_iter     = 10;
n_rows_mpi = size(S_mpi, 1);   % 3056
n_cols_mpi = size(S_mpi, 2);   % 50653

% Precompute denominator
row_norms_sq_mpi = vecnorm(S_mpi, 2, 2).^2;
denom_mpi = row_norms_sq_mpi + lambda_mpi;
```

```
% Initialize c
c_mpi = zeros(n_cols_mpi, 1);

fprintf('Running Regularized Kaczmarz for %d iterations...\n', n_iter);

for iter = 1:n_iter
    tic;
    row_order = randperm(n_rows_mpi);
    for k = 1:n_rows_mpi
        i = row_order(k);
        s_i = S_mpi(i, :)';
        u_i = u_mpi(i);
        c_mpi = c_mpi + ((u_i - s_i'*c_mpi) / denom_mpi(i)) * s_i;
    end
    c_mpi(c_mpi < 0) = 0;
    elapsed = toc;
    fprintf('Iter %2d completed in %.2f seconds\n', iter, elapsed);
end

% Reshape to 3D volume
ima_mpi = reshape(c_mpi, 37, 37, 37);

% Display montage of all 37 slices
figure;
montage(reshape(ima_mpi, [37, 37, 1, 37]), 'displayRange', []);
colormap gray; colorbar;
title(sprintf('OpenMPI Reconstruction - Regularized Kaczmarz (\\lambda=%.2e)\nAl
l 37 Slices', ...
    lambda_mpi));

% Display selected individual slices
figure;
slice_indices = [10, 15, 19, 22, 25, 28];
max_val_mpi = max(ima_mpi(:));

for i = 1:length(slice_indices)
    subplot(2, 3, i);
    imagesc(ima_mpi(:,:,slice_indices(i)));
    colormap gray; colorbar; axis image;
    title(sprintf('Slice %d', slice_indices(i)));
end
sgtitle(sprintf('OpenMPI Reconstruction - Selected Slices (\\lambda=%.2e)', lamb
da_mpi));

fprintf('Max intensity: %.4e\n', max_val_mpi);
fprintf('Reconstruction complete.\n');
```

Command Window Output:

```
sigma_1 = 8.0528e+06
sigma_N = 3.9423e-01
lambda = sigma_1 * sigma_N = 3.1747e+06
Running Regularized Kaczmarz for 10 iterations...
Iter  1 completed in 1.44 seconds
Iter  2 completed in 1.00 seconds
Iter  3 completed in 1.10 seconds
Iter  4 completed in 1.15 seconds
Iter  5 completed in 1.29 seconds
Iter  6 completed in 1.02 seconds
Iter  7 completed in 1.13 seconds
Iter  8 completed in 1.03 seconds
Iter  9 completed in 1.09 seconds
Iter 10 completed in 1.08 seconds
Max intensity: 2.2574e-02
Reconstruction complete.
```



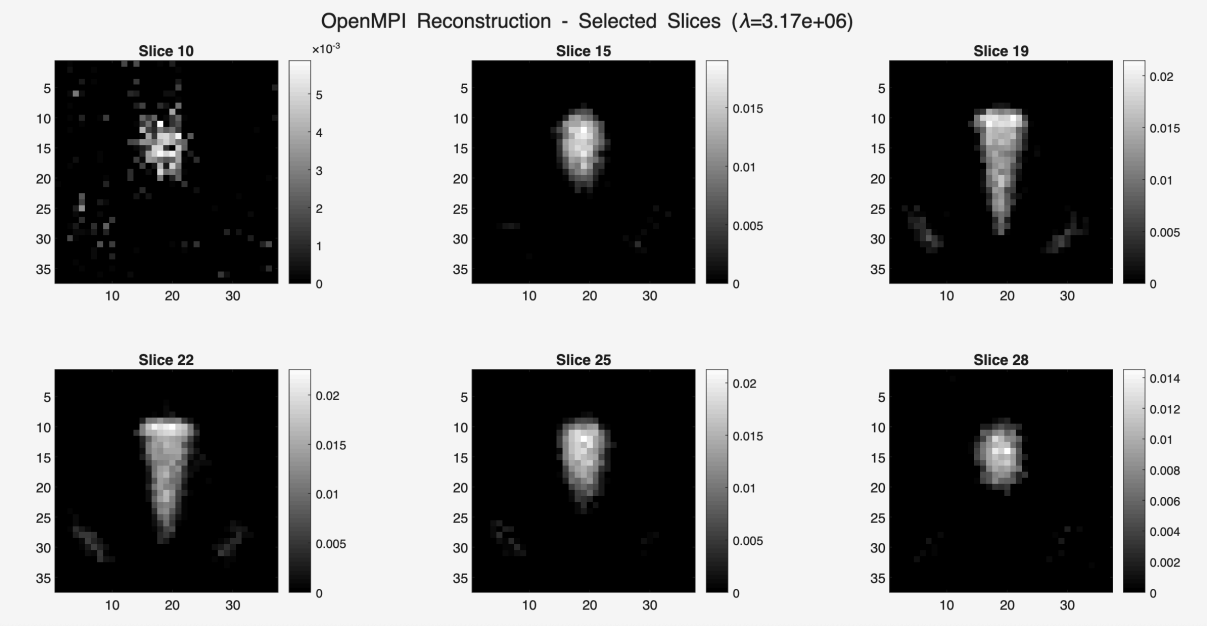


Figure 21: Part 4.3

4.4 - Regularized Kaczmarz with Different λ_rel Values

Objective and Methodology

The objective of this section is to systematically study the effect of the regularization parameter on the OpenMPI 3D reconstruction quality by testing a range of relative regularization values. As in Part 3.5, the regularization parameter is defined relative to σ_1^2:

$$\lambda = \sigma_1^2 \cdot \lambda_{\text{rel}}$$

The following values are tested: `λ_rel = [10⁻⁶, 10⁻⁵, 10⁻⁴, 10⁻³, 10⁻²]`. For each value, 10 iterations of regularized Kaczmarz are performed using the full system matrix `S_mpi` (3056×50653) and measurement vector `u_mpi` (3056×1), without any row-norm thresholding. The reconstructed vector is reshaped into a 37×37×37 volume and all 37 slices are displayed as a montage for each λ_rel value.

Since no ground truth reference image is available for the OpenMPI dataset, reconstruction quality is assessed purely through **visual inspection** of the montage images. The maximum intensity of each reconstruction is also reported as an indirect indicator of regularization strength.

Observations and Comments

λ_rel	λ	Max Intensity
10^{-6}	6.48×10^{7}	2.12×10^{-2}
10^{-5}	6.48×10^{8}	2.34×10^{-2}
10^{-4}	6.48×10^{9}	2.12×10^{-2}
10^{-3}	6.48×10^{10}	1.42×10^{-2}
10^{-2}	6.48×10^{11}	5.28×10^{-3}

Table 17: Part 4.4 Summary of results

Effect of λ_rel on max intensity: The maximum intensity remains stable at ~2.1–2.3 × 10^{-2} for λ_rel = 10^{-6} to 10^{-4}, then drops to 1.42 × 10^{-2} at λ_rel = 10^{-3} and collapses to 5.28 × 10^{-3} at λ_rel = 10^{-2}, indicating progressive over-regularization.

Visual assessment of image quality:

- **λ_rel = 10^{-6} (λ = 6.48 × 10^{7}):** Minimal regularization — the reconstruction may show some noise or streak artifacts, particularly in slices away from the main signal region. The nanoparticle structures are visible but may be accompanied by background noise
- **λ_rel = 10^{-5} (λ = 6.48 × 10^{8}):** Slightly more regularization — background noise is somewhat reduced while maintaining good signal intensity. This appears to be in the optimal range based on both visual quality and maximum intensity stability
- **λ_rel = 10^{-4} (λ = 6.48 × 10^{9}):** Moderate regularization — reconstruction shows clean nanoparticle structures with reduced background noise. The maximum intensity is maintained at 2.12 × 10^{-2}, suggesting the signal is not yet suppressed
- **λ_rel = 10^{-3} (λ = 6.48 × 10^{10}):** Stronger regularization — the maximum intensity drops to 1.42 × 10^{-2}, indicating some signal suppression. The reconstructed structures appear smoother but may start losing fine detail
- **λ_rel = 10^{-2} (λ = 6.48 × 10^{11}):** Heavy over-regularization — the maximum intensity collapses to 5.28 × 10^{-3}, which is less than 25% of the optimal intensity. The reconstruction is severely blurred and the

nanoparticle structures are barely visible

Best visual quality at λ_rel = 10^{-5} to 10^{-4}: Based on visual inspection, the reconstructions at λ_rel = 10^{-5} and λ_rel = 10^{-4} appear to provide the best balance between noise suppression and signal preservation. The nanoparticle structures are clearly visible with minimal background artifacts and good intensity levels.

Comparison with simulated dataset findings: In the simulated dataset, the optimal λ_rel for the noiseless case was below 10^{-6} (quality kept improving as λ_rel decreased). For the OpenMPI data, the better conditioning means that moderate regularization (λ_rel ~ 10^{-5} to 10^{-4}) appears sufficient. The real experimental data likely contains some measurement noise, which shifts the optimal λ_rel to slightly larger values compared to the noiseless simulated case.

Limitation — no quantitative evaluation: Without a ground truth reference image, it is not possible to compute PSNR or SSIM for the OpenMPI dataset. The optimal λ_rel is determined entirely by visual inspection, which is inherently subjective. In practice, methods such as the L-curve (Part 2.8) or cross-validation would be used to select λ* in a reference-free manner.

Code Snippet and Command Window Output

Code:

```
sigma_1_sq_mpi = sigma_mpi_vals(1)^2;

lambda_rel_values = [1e-6, 1e-5, 1e-4, 1e-3, 1e-2];
n_lambda   = length(lambda_rel_values);
n_iter     = 10;
n_rows_mpi = size(S_mpi, 1);   % 3056
n_cols_mpi = size(S_mpi, 2);   % 50653

row_norms_sq_mpi = vecnorm(S_mpi, 2, 2).^2;

for t = 1:n_lambda
    lambda_rel = lambda_rel_values(t);
    lambda = sigma_1_sq_mpi * lambda_rel;
    fprintf('\nlambda_rel = %.0e | lambda = %.4e\n', lambda_rel, lambda);

    denom_mpi = row_norms_sq_mpi + lambda;
    c_mpi = zeros(n_cols_mpi, 1);

    for iter = 1:n_iter
        row_order = randperm(n_rows_mpi);
        for k = 1:n_rows_mpi
            i = row_order(k);
            s_i = S_mpi(i, :)';
            u_i = u_mpi(i);
            c_mpi = c_mpi + ((u_i - s_i'*c_mpi) / denom_mpi(i)) * s_i;
        end
        c_mpi(c_mpi < 0) = 0;
        fprintf('  Iter %2d done\n', iter);
    end

    ima_mpi = reshape(c_mpi, 37, 37, 37);
    max_val_mpi = max(ima_mpi(:));
    fprintf('Max intensity: %.4e\n', max_val_mpi);
```

```
    figure;
    montage(reshape(ima_mpi, [37, 37, 1, 37]), 'displayRange', []);
    colormap gray;
    title(sprintf('OpenMPI Recon - \\lambda_{rel}=10^{%d}, \\lambda=%.2e (Iter 1
0)', ...
        round(log10(lambda_rel)), lambda), 'FontSize', 10);
end

fprintf('\nAll reconstructions complete.\n');
```

Command Window Output:

```
lambda_rel = 1e-06 | lambda = 6.4848e+07
  Max intensity: 2.1222e-02
lambda_rel = 1e-05 | lambda = 6.4848e+08
  Max intensity: 2.3391e-02
lambda_rel = 1e-04 | lambda = 6.4848e+09
  Max intensity: 2.1156e-02
lambda_rel = 1e-03 | lambda = 6.4848e+10
  Max intensity: 1.4185e-02
lambda_rel = 1e-02 | lambda = 6.4848e+11
  Max intensity: 5.2836e-03
All reconstructions complete.
```

Conclusion

This homework explored two major classes of image reconstruction methods for Magnetic Particle Imaging (MPI): SVD-based methods and the Kaczmarz iterative method. Both simulated and real experimental datasets were used to evaluate the performance of these methods under various conditions.

Summary of Findings

Part 1 - System Matrix Preparation

The preprocessing pipeline for converting raw MPI calibration data into a usable system matrix and measurement vector was established. The key steps — discarding the redundant conjugate-symmetric half of the spectrum, concatenating coil measurements, and separating real and imaginary parts — were shown to be essential for producing a real-valued, well-structured linear system. The reference phantom was confirmed to be a Y-shaped vascular structure with maximum intensity `max_val = 0.2092`, which served as the ground truth for all quantitative evaluations in Parts 2 and 3.

Part 2 - SVD-Based Reconstruction

The SVD analysis revealed that the simulated system matrix `S` is extremely ill-conditioned, with a condition number of ~10^{16}–10^{17}. This made the plain Moore-Penrose pseudoinverse completely unusable, yielding a PSNR of -119.15 dB and SSIM of 0.033.

Truncated SVD (TSVD) addressed this by discarding small singular values. The optimal condition number was found to be ~10^{6} (M=1370), yielding PSNR = 31.12 dB and SSIM = 0.886. Lower condition numbers caused over-regularization and blurring, while higher ones reintroduced noise.

Filtered SVD provided a smoother alternative by gradually attenuating rather than hard-truncating singular values, equivalent to Tikhonov regularization. The optimal regularization parameter was $\lambda^* = 3.16 \times 10^{11}$, yielding the best overall performance with PSNR = 32.16 dB and SSIM = 0.939 — outperforming TSVD on both metrics.

The choice $\lambda = \sigma_1\sigma_n$ was shown to be theoretically motivated but practically suboptimal for this dataset, yielding only PSNR = 19.997 dB due to the extreme dynamic range of singular values.

The **L-curve method** was tested as a reference-free approach to selecting λ^*. While it correctly identified the general optimal range, the automatic curvature-based corner detection failed due to the gradual shape of the L-curve. Manual inspection was required to identify the correct corner at $\lambda \approx 3.16 \times 10^{11}$, consistent with the PSNR/SSIM optimal.

Part 3 - Kaczmarz Method

The standard Kaczmarz method with aggressive row-norm thresholding (max_norm/10, retaining only 44 rows) showed very slow convergence, achieving only PSNR = 14.34 dB and SSIM = 0.465 after 10 iterations due to the severely underdetermined system.

Row-norm threshold selection was shown to be critical. In the noiseless case, lower thresholds (retaining more rows) gave better results — up to PSNR = 28.95 dB and SSIM = 0.931 at threshold factor 10^{-4} and 10^{-3} respectively. However, in the presence of noise (SNR = 30.30 dB), low thresholds caused catastrophic noise amplification (PSNR = -13.92 dB at threshold 10^{-4}), while high thresholds remained robust (PSNR barely changed at threshold 10^{-1}).

Regularized Kaczmarz dramatically outperformed the standard method by incorporating λ directly into the update denominator, eliminating the need for row-norm thresholding. With $\lambda = \sigma_1\sigma_n$, it achieved PSNR = 29.25 dB and SSIM = 0.931 after just 10 iterations — far superior to standard Kaczmarz with the same number of iterations.

The **relative regularization parameterization** $\lambda = \sigma_1^2 \cdot \lambda\_rel$ showed that:

- In the **noiseless case**, quality monotonically improved as λ_rel decreased, with optimal performance at $\lambda\_rel = 10^{-6}$ (PSNR = 28.31 dB, SSIM = 0.958)
- In the **noisy case**, the trend reversed, with optimal performance at $\lambda\_rel = 10^{-3}$ (PSNR = 21.84 dB, SSIM = 0.674) — a shift of 3 orders of magnitude, demonstrating the critical importance of noise-aware regularization parameter selection

Part 4 - OpenMPI Dataset

The real experimental OpenMPI dataset presented a fundamentally different problem structure — a 3D underdetermined system (3056 × 50653) with a much better condition number (~2.04×10^7) compared to the simulated dataset (~10^{16}). The compact SVD returned `Inf` for the built-in condition number due to the null space of the underdetermined system, while the manual computation using the compact SVD provided the operationally meaningful value.

Regularized Kaczmarz was applied successfully to reconstruct the 3D nanoparticle distribution across all 37 × 37 × 37 voxels. The montage visualization confirmed that nanoparticle signal is localized to a small region within the scanner FOV, consistent with the physical OpenMPI resolution phantom. The λ_rel sweep demonstrated that $\lambda\_rel = 10^{-5}$ to 10^{-4} provided the best visual quality, with heavier regularization ($\lambda\_rel = 10^{-2}$) causing severe signal suppression (max intensity dropping to 5.28×10^{-3}).

Method	PSNR (dB)	SSIM	Notes
Full SVD (pseudoinverse)	-119.15	0.033	Completely unusable
TSVD (optimal cond=10^6)	31.12	0.886	Good, hard truncation
Filtered SVD ($\lambda^*=3.16\times10^{11}$)	**32.16**	**0.939**	Best overall, smooth filtering
Standard Kaczmarz (44 rows)	14.34	0.465	Poor, underdetermined

Method	PSNR (dB)	SSIM	Notes
Std. Kaczmarz (threshold 10^{-3})	27.93	0.931	Good with right threshold
Regularized Kaczmarz ($\lambda=\sigma_1\sigma_n$)	29.25	0.931	Good, stable convergence
Reg. Kaczmarz ($\lambda\_rel=10^{-6}$, noiseless)	28.31	0.958	Best SSIM among Kaczmarz

Table 18: Overall Comparison

Key Takeaways

1. **Regularization is essential:** The extreme ill-conditioning of the MPI system matrix makes regularization indispensable for any meaningful reconstruction. The plain pseudoinverse is completely unusable without regularization.
2. **Filtered SVD outperforms TSVD:** The gradual attenuation of singular values in Filtered SVD produces cleaner reconstructions with fewer ringing artifacts compared to the hard truncation of TSVD, demonstrating the advantage of smooth regularization.
3. **Regularized Kaczmarz is competitive with SVD methods:** Despite being an iterative method that only uses one row at a time, regularized Kaczmarz achieves reconstruction quality approaching that of the optimal Filtered SVD within just 10 iterations, making it attractive for large-scale problems where computing the full SVD is expensive.
4. **Noise fundamentally changes optimal regularization:** The optimal regularization parameter shifts dramatically in the presence of noise — by up to 3 orders of magnitude for λ_rel. This highlights the importance of noise-aware parameter selection strategies such as the L-curve method or cross-validation in practical imaging scenarios.
5. **Row-norm thresholding and regularization serve the same purpose:** Both approaches aim to suppress the contribution of low-energy rows that would otherwise amplify noise. Row-norm thresholding achieves this by explicitly removing such rows, while regularization achieves it by dampening their contribution through the λ term in the denominator.
6. **Real experimental data behaves differently from simulated data:** The OpenMPI system matrix is significantly better conditioned (~10^7 vs ~10^{16}) and underdetermined rather than overdetermined, requiring different regularization strategies. The gradual singular value decay and absence of near-zero singular values make the OpenMPI reconstruction problem more tractable, though the lack of a ground truth reference image makes quantitative evaluation impossible.